XPLM®

Installation & Administration Guide

Autodesk Inventor integration
for
Aras Innovator

Valid for product version: 4.7.0

Published: 27.02.2026 | Build: 1285 | Revision: 4bb116954

XPLM Solution GmbH

Altmarkt-Galerie

Altmarkt 25

01067 Dresden

Office: +49 351 82658-0

Fax: +49 351 82658-88

Web: www.xplm.com

Legal information

Non-disclosure

The information in this document is to be treated confidentially.

Licensing

XPLM Solution GmbH (XPLM) does not issue licenses for the software components to be integrated. These licenses must be obtained from the respective software manufacturer. To license the integration software from XPLM, a software usage and maintenance contract must be concluded.

Support

In the event of problems with the integration software, access the XPLM Support Portal at https://support.xplm.com and file a ticket. If you don't have an account yet, contact support@xplm.com and request one.

Feedback

If you discover any errors or inconsistencies in this documentation, write to documentation@xplm.com and let us know what we can improve.

Handling personal data

Against the background of the EU General Data Protection Regulation (GDPR), XPLM informs about the processing of personal data in its contracts and privacy policies. The extent to which XPLM comes into contact with personal data from the customer environment while providing services depends on the customer's IT systems and the data transmitted to XPLM. XPLM is generally not interested in gaining access to personal data from the customer environment. If this cannot be avoided, the customer is responsible for such personal data and must enter into a GDPR-compliant data processing agreement with XPLM.

Information exchange: The integration software exchanges information between the systems involved in the form of attributes. What is contained in these attributes is left to the customer's definition. For ECAD integrations, the exchanged attribute information is saved in the file `metadata.xml` in the configured project directory or directly in the project files.
For other integrations, the exchanged attribute information is saved in files or objects of the source system.

Log files: To record activities of integration functions, the integration software stores log files on the computer on which it is installed. These log files can also contain information from Windows environment variables, for example the user and computer name. For ECAD integrations, log files are stored in the directory `%TEMP%\integrate` by default. This directory can be deleted, but new log files will be created when the integration software is used again. For other integrations, logging is deactivated by default. Activation, protocol level and storage location must be set up in advance. Directories with log files can be deleted, but new log files will be created when the integration software is used again.

Support tickets: When submitting a support ticket in the event of problems with the integration software, information such as first and last name, email address, etc. can be transmitted to XPLM support. For more information on this topic, see *Legal Notices* > *Privacy Policy* in the support portal.

Other policies: XPLM also refers to the privacy policies of the software manufacturers involved for the handling of personal data in their systems.

Table of Contents

Glossary

Application Programming Interface (API)

Defines a set of routines, communication protocols and tools for building software. In general, they are clearly defined methods for communication between different components.

Aras Markup Language (AML)

Defines the backbone language of Aras Innovator. Almost every action a user performs in the client sends an AML request to the Aras Innovator server to process the business logic.

Bill of Materials (BOM)

Defines a list of assemblies, sub-assemblies, parts and their quantities needed to produce a final product.

BOM position

Defines a position in the BOM with unique identification, name, quantity and other characteristics.

Component Object Model (COM)

Defines a binary-interface standard for software components introduced by Microsoft.

Connector

Defines a central interface component of each XPLM integration. The integration uses connectors for each participating application to exchange data via their API.

Datamodel

Defines objects and their relationships in a PLM system that are managed by the integration to store data from an authoring application.

Dynamic Link Library (DLL)

Defines a file with a library of functions and other information that can be accessed by a Windows program.

Generation

Defines an incremental counter of each object modification in Innovator on check-in.

Payload

Defines the data contained within an API request. The description is borrowed from the transportation industry, where a truck carries its cargo (its payload) to a location. The truck, as with the API request, is always the same, but the payload changes with each request.

Product Lifecycle Management (PLM)

Defines systems and processes for managing data during the development of a product from creation through manufacturing to maintenance and disposal.

Revision

Defines a released object state in Innovator that cannot be modified.

Script engine

Defines the central component in each integration. It contains the integration logic for processing and forwarding the information and data coming from the connectors.

User Interface (UI)

Defines a (usually) graphical interface through which a user interacts with the computer.

x86/x64

Defines the processor architecture in a computer and thus also the performance of applications. x86 corresponds to 32-bit and x64 corresponds to 64-bit.

1 Introduction

1.1 About this document

You are reading the Installation and Administration Guide of the Autodesk Inventor integration for Aras Innovator.

Purpose

This document describes how to install, set up and configure the integration.

Target audience

This document is intended for system administrators in the company who are responsible for the operation of the integration. In addition, system administrators should also read the User Guide to familiarize themselves with the user interface and integration functions.

How to read this document

This document is structured chronologically and you should read it in the order of the chapters described. If you skip chapters, you will miss important information.
Where appropriate, cross-references to other chapters are listed. To quickly return to where you came from after clicking such a link, click the **Back** button in your PDF viewer. Try it right now with this link!

Notes used


This note highlights additional information about the current content.


This note highlights important instructions.


This note highlights the integration version from which a particular feature or change was introduced or updated.

Searching for text strings

Long text strings, for example variables, are sometimes manually wrapped in this document to fit in a table cell. When you search for such terms in this document, Acrobat Reader and Acrobat cannot interpret a line break, which results in the entire term not being found. Therefore, always open the PDF in one of the above browsers and search there.

Copying code

You can read this document in all supported PDF viewers, including PDF browser plugins. Check the following recommendations about copying code and searching for text strings.

```
<XPlmConnector>
  <Properties>
    ...
  </Properties>
</XPlmConnector>
```

Not all PDF viewers support proper copying of code. If you open this document in Firefox, Acrobat Reader, or Acrobat and copy the above code, the lines will be pasted without indents and even in one line only, depending on the version of the PDF viewer.

Browsers that copy code correctly are Chrome, Edge and Brave. Keep your browsers and Acrobat tools always up to date.

1.2 Naming conventions

In addition to the terms already listed in the glossary, the following naming conventions are used in this document. Familiarize yourself with these names. You will find them throughout the document.

Form Used	Generally refers to
Innovator	Aras Innovator
Inventor	Autodesk Inventor
Part	Part in Innovator
Attribute	Object property in Innovator
Library Part	Standard Part

1.3 Whats new

Check out what's new in this release! This section provides a summary of the latest documentation updates based on new features and improvements to the integration. Refer to the official release notes for detailed release information.

Version 4.7.0

Low-Code Configuration for Dialog Options and Check Instructions

You can now configure Check Instructions and Dialog Options directly in Innovator, without editing connector XML files.

Highlights:

- Check Instructions are now configured via an XML snippet stored in an ItemType in Innovator.
- Each Check Instruction can be linked to specific dialogs (for example, *Save dialog, Copy dialog, Load dialog*)
- Configure Load and Save Options for visibility, editability, and default values.
- Default configurations for Save and Load are provided in a new Innovator database setup

Learn more: Configuring connector dialogs (p. 31)

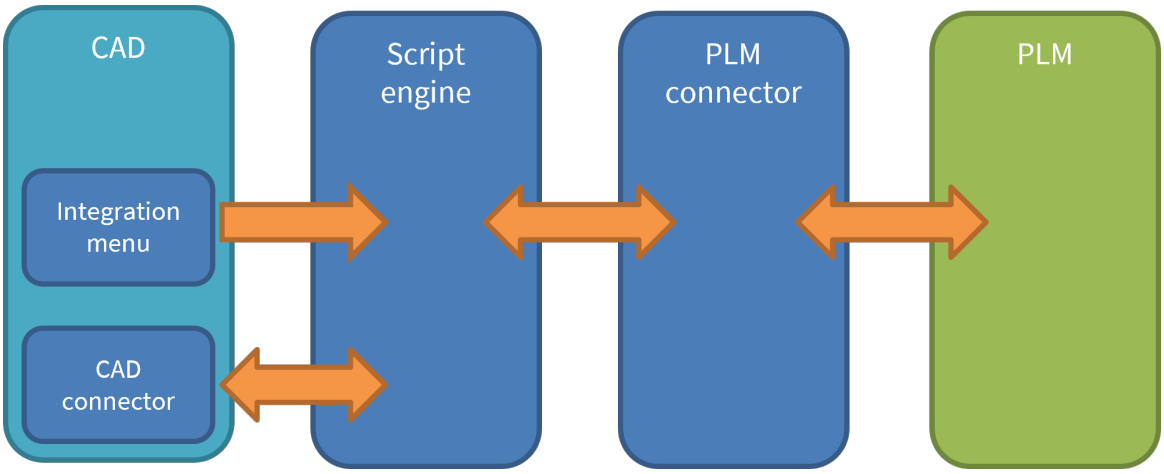
1.4 Integration overview

The integration is the interface between Inventor and Innovator and is designed for seamless and efficient data exchange between both applications.

The integration focuses on a Inventor-centric workflow, enabling users to interact with Innovator directly within Inventor. The main integration functions include saving, loading, and performing object operations. When saving CAD data to Innovator, corresponding objects and structures are created in Innovator and linked to the CAD data. The original CAD files are attached to these objects, allowing for centralized data management within the company.

To support concurrent engineering and avoid access conflicts, the integration utilizes the object reservation capabilities (checkin/checkout) from Innovator.

The integration functionalities are accessible through additional toolbars or extended menus in Inventor.



1.5 Supported file types

The Innovator - Inventor integration supports the following file types:

File type	File extension
Part	.ipt
Assembly	.iam
Drawing	.idw .dwg

2 Setup overview

This overview serves as the starting point for the integration setup.

Step 1: Preparation

The following chapters help you to understand key aspects of the integration, verify system requirements, and to organize licenses and installation media.

- System requirements (p. 14)
- Required integration licenses (p. 14)
- Required installation media (p. 14)

Step 2: Installation

To ensure the Innovator server is properly configured, the integration is correctly installed, the integration menus are available, and the applications are ready for use, complete all installation steps described in the following chapters.

- Installing the Server Components (p. 16)
- Installing the Client Application (p. 18)

Step 3: Initial setup

Complete all setup tasks in chapter Initial setup (p. 20) . This will ensure that the integration is prepared for data exchange between the applications.

Step 4: Innovator-based Configuration

In the Innovator-based configuration, you manage all system and application configuration directly in Innovator. This new approach has the advantage that configuration is held centrally in Innovator and can be deployed to all clients automatically when connecting to Innovator with the Inventor connector. This is the preferred method going forward.

See Innovator-based configuration (p. 24) for more information.

Step 4: Legacy XML-based configuration

In the legacy configuration method, you mainly work in configuration files. This approach allows a high degree of flexibility and customer-specific adaptations. This method is still supported for backward compatibility but is being phased out in favor of the Innovator-based configuration.

See Legacy XML-based configuration (p. 35) for more information.

3 System requirements

Before installing the integration, check that all requirements are met.

Supported versions and special requirements

- Official support: The version is supported by the latest integration. Issues are fixed immediately.
- Limited support: The version is generally supported by the latest integration, but has not been explicitly tested. Issues are fixed only if they occur in the official versions.

Name	Official support	Limited support	Special requirements
Aras Innovator	14-36+	12 SP0-18	An administrator account is required during installation and setup of the integration. Access to the Innovator server as administrator is required. A compatible browser for using Innovator is required.
Autodesk Inventor	2023-26	2018-22	The user has been granted any necessary privileges to use the application.

Client-side requirements

To install and run the integration on the client computer, the following is needed:

- Windows 10/11 x64 with local administrator rights
- Inventor in a supported version

  Starting with Inventor 2026, Apprentice Server is no longer included in the installer. Users must install it separately using the standalone installer. For more information, refer to Inventor 2026 documentation
- Microsoft .NET Framework 4.7.2+

3.1 Required integration licenses

The integration functions and connectors require corresponding licenses. Make sure you have them available.

Used license mechanism

The integration uses the license mechanism from Aras. Get the required licenses directly via Aras or an authorized reseller.

3.2 Required installation media

Prior to installation, make sure you have all installation media available.

What you need

- Installer `Aras-MCAD-*.7z.exe`
- Other installation media as listed in System requirements (p. 14)
- AML definition package `Aras-SolutionsImportAML-*.7z.exe`
- Conversion Server Plugin `Aras-ConversionServer_*.7z.exe` (optional)

Download location

Access https://support.xplm.com and download the installation media from the **Download Portal** tab and the following directories. If you don't have an account yet, contact support@xplm.com and request one.

- Official product releases: `Products/<plm>/<domain>`
- Customer-specific product releases: `Customer/<customer>`

4 Installation

4.1 Installing the Server Components

4.1.1 Importing AML definitions for connector settings

XPLM provides a preconfigured AML package to import required object definitions and other settings for the connector into the Innovator database. To import these definitions, use the import tool included in the Innovator installation media.

Before you start

See the official Aras documentation *Package Import Export Utilities* for more information on the import tool.

About this task

To import the package, complete the following steps.

Procedure

1. Double-click the archive `Aras-SolutionsImportAML_*.7z.exe` and extract it to a directory.
2. **If you are using Innovator 12 SP18 or lower**: In the extracted archive, edit the manifest file `Aras\imports.mf`. Uncomment the line with `... path="CAD_Exchange\Import_V12"` and comment the line with `... path="CAD_Exchange\Import"`. See also the note in this file. Save the file.
3. Start the import tool `import.exe`.
4. Enter the connection information for Innovator including administrator credentials and click **Login** to test the connection.
5. Enter a target release for the database and a description for the import.
6. In the section **Manifest File**, define the path to the manifest file `Aras\imports.mf` in the extracted archive.
7. In the section **Available for Import**, select all available packages.
8. In the section **Type**, select the option **Merge**.
9. In the section **Mode**, select the option **Thorough**.
10. Click **Import**.
11. **Only when using Batch Server**: Repeat the import for the manifest file in the directory `BatchQueue\imports.mf`.
12. Repeat the import for the manifest files in the directory `Collaboration`.

Result

Definitions are imported.

What to do next

Log in to Innovator as administrator and select **Administration** > **Configuration** > **Package Definitions**. Click **Search PackageDefinitions** and check for entries starting with `com.xplm*`, corresponding to the imported packages.
Select **Administration** > **Variables** and search for `xplm*` in the field **Name**. Check if the versions of the entries correspond to the version in the file name of the imported package.

4.1.2 Importing AML definitions for Innovator-based configuration

The required basic AML definitions for allowing Innovator-based configuration were already imported after completing the chapter Importing AML definitions for connector settings (p. 16). In addition, you have to import the connector-specific Innovator-based configuration settings that are available at the time of this release.

Before you start

See the official Aras Innovator documentation *Package Import Export Utilities* for more information on the import tool.

You can keep track of the version of the AML package installed using the internal version identifier file `XPLM ArasConnector LowCode Package Version.xml` available in the extracted archive `Aras-SolutionsImportAML_*\LowCode\xplm\cax_configuration_data\ArasConnector\Import\Variable`.

About this task

This task is optional and only required if you want to explore the capabilities of configuring connector settings in Innovator.

Procedure

1. Start the Aras import tool `import.exe` again and define the connection to Innovator.
2. In the section **Manifest File**, define the path to the manifest file `LowCode\imports.mf` in the extracted archive `Aras-SolutionsImportAML_*`.
3. In the section **Available for Import**, select all available packages.
4. In the section **Type**, select the option **Merge**.
5. In the section **Mode**, select the option **Thorough**.
6. Click **Import**.

Result

Connector-specific settings are imported.

What to do next

Log in to Innovator as administrator and select **Administration** > **Connector Configuration** > **Settings**. Click **Search** and check for entries corresponding to Inventor.

If the icons for **Mapping Definitions**, **Settings** and **Resources** are missing in Innovator, you can subsequently copy them from the extracted archive `Aras-SolutionsImportAML_*\icons` into the Aras tree `$Aras-Installation\Innovator\Client\images`.

4.1.3 Installing licenses in Innovator

To use the integration, you require special feature licenses from Aras which you install in Innovator.

Before you start

Make sure the feature licenses for the integration are available, see Required integration licenses (p. 14) for more information.

Procedure

1. Log in to Innovator with an administrator account.
2. In the top-right corner, click the user icon and select **Activate Feature** from the dropdown list.
3. Enter the activation key and click **Activate Feature**.

4. Repeat for all required activation keys.

Result

The feature licenses are installed.
Complete tasks for initial setup.

4.2 Installing the Client Application

4.2.1 Installing integration

Make sure that all installation media and licenses are available, and then start the installation.

About this task

This unified installer allows you to install multiple XPLM products in the same installation directory. You can update all components individually without affecting the functionality of other installed products.

The following procedure applies to a new installation. To update an existing installation, see Updating installation (p. 82) for more information.

You can install, modify, repair, or uninstall also in silent-mode. See Silent-mode installation (p. 136) for more information.

Procedure

1. Copy the installer archive `Aras-MCAD-*.7z.exe` to the client computer running Inventor.
2. Close all open applications related to the integration.
3. Extract the archive and start `Setup-*.exe` with administrator rights.
4. Install any Visual C++ runtimes that you are prompted for.
   ➔ The runtimes are installed, and the installation wizard appears.
5. Click **Next** to start the wizard.
   ➔ The step *License agreement for end-users* appears.
6. Accept the license agreement and click **Next**.
   ➔ The step *Installation path* appears.
7. Enter the installation directory and click **Next**.
   - The default installation directory `C:\Program Files\XPLM Solution GmbH` is protected by Windows and only administrators can modify files in it.

     To avoid later file access problems, it's good practice to always select a writable location not protected by Windows.
   - If another product is already installed, you cannot change the installation directory. The product installed first defines this path for other products.
   - If you have an overlay package you want to apply, enable the option **Apply custom files after installation** and enter the path to the overlay. Overlays contain customized files that overwrite the installed files as the last step. See Working with overlay packages (p. 143) for more information.
   - To backup the existing installation, enable the option **Backup current installation**. Define the backup scope first. Then click **Browse** and select a backup directory.

     Always create a backup when you update or change an installation. This makes it easier to compare and merge the files later.

   ➔ The step *MCAD components* appears.

8. Select the application(s) to be integrated. If required, select additionally a version or other settings. Click **Next**.
   ➔ The step *Tool components* appears.
9. Select additional tools or add-ons that you can use in the scope of this installation and click **Next**.
   - Batch Queue Admin
   - Batch Server
   - CADBAS Checkout Tool
   - Collaboration Client
   - Collaboration Server
   - Conversion Server
   - Encryption Helper
   - File Export Tool
   - License Test Tool

   ➔ The step *Ready to install* appears.
10. To start installation, click **Install**.

    In installer versions 24.3.3.606+, a progress dialog is shown with detailed information on each MSI currently installed. If there are installation issues, the process stops and the error is shown in red in the lower dialog section. Click **Save to log** and send the log file to support@xplm.com for further investigation.
    For older installers, a common CMD window with the progress is shown. Do not click into this window or installation cannot proceed. To continue if you clicked in, press **Enter** or **Esc**.
11. To close the wizard after installation, click **Finish**.
12. To finalize the installation, restart the system.

Result

Installation is complete and the environment variable `xPlmRootDir` points to the installation directory. You can find log files for all installed components in the directory `%ProgramData%\XPLM Solution GmbH\log`.

What to do next

Complete tasks for initial setup.

Related links
Tools and add-ons (p. 90)

5 Initial setup

5.1 Integration setup

Complete the following tasks on the client computer where Inventor is running and the integration is installed.

5.1.1 Setting up connection parameters for Innovator

The connector allows declaring several login environments. This can be used for example, to allow users to choose between production and test environment, or other separated instances.

- **Configuration file:** `XPlmArasLogonParameter.xml`

Login parameters are defined between `<logon>` blocks. You can create as many of these blocks as needed and name them using the tag `ENVIRONMENT`.

Since release 4.5.0, the authentication towards Innovator is handled by the *Aras login dialog* and no longer by the integration. This means, you only need to define the URL to theInnovator server in the XML file. All further authentication (such as Database and user credentials) happens in the *Aras login dialog*. See User Guide for more information.

Using Windows Authentication

To use Windows Authentication, edit the configuration file `XPlmArasConnector.xml` and set `ArasWindowsAuthentication` to `true`:

```
  <Field>
    <Name>ArasWindowsAuthentication</Name>
    <Value>true</Value>
  </Field>
...
```

If the default login is used, the flag `ArasWindowsAuthentication` must be set to `false`.

5.1.2 Setting up the root workspace

The integration sets the root workspace automatically but you can configure it to your desired location.

- **Configuration file:** `XPlmConnector.xml`
- **Property tag:** `LocalWorkDir`

Every file path on the local workstation can be used as an option setting. However, it is necessary that each user has full read and write access to the workspace directory and all its sub-directories on the local workstation.

The default workspace is set to `C:\caxtmp`.

Since release 4.1.1, it is possible to define the local working directory in `XPlmConnector.xml` using environment variables such as `%USERPROFILE%`.

Standard parts

Also the path to the checkout folder for standard parts can be configured. Executing **Synchronize Part Library** downloads standard parts from Innovator to defined path `StandardPartDir`. By default, the path is set to `C:\caxtmp\StandardParts\`.

5.1.3 Defining the integration language

The Inventor connector is supported in different languages hence the Add-in menu's language can be changed.

- **Configuration file:** `XPlmConnector.xml`
- **Property tag:** `Language`

The following languages are supported:

Language	Value
English	EN
German	DE

To change the language of the integration, set the `Language` tag in `XPlmConnector.xml` file to the desired language.

5.2 Starting the integration

No additional configuration is necessary.

Integration menu

When you start Inventor for the first time after installation, the *Add-in Manager Security Alert* pop-up dialog is displayed listing the *XplmInventorAddIn* as blocked. Click **Launch the Add-In Manager** to unblock the the add-in. Mark the **Load Automatically** check-box for the add-in to be loaded every time Inventor is started.

Environment

Make sure that a version of Excel is installed on the system if families of parts should be built. Otherwise the option **Save** of the integration will not properly work.

5.3 Aras Innovator

5.3.1 Authentication modes

About this task

There are different modes for login:

- Interactive: User/password provided interactively by the user
- Single Sign-On: User/password provided by Windows (SSO with Windows)

This use case describes how to configure SSO.

Procedure

1. Follow the *Windows Authentication Setup* of Innovator to enable SSO in Innovator.
2. For Innovator versions prior to Innovator 12: In the `OAuthServer\web.config` file:
   a) Set value for `ProtocolType` to `Custom`.
   b) Set value for `EnableWindowsAuthentication` to `true`.
3. Use the same SP version as the server when selecting Innovator version during client side installation of the XPLM integration (for example, Innovator 11 SP15 when the server runs at SP15).
4. When the SSO with the Innovator web client works correctly, apply the following settings in XPlmArasConnector.xml (p. 126):

```
<Field>
  <Name>ArasUseLegacyLogin</Name>
  <Value>true</Value>
</Field>
<Field>
  <Name>ArasWindowsAuthentication</Name>
  <Value>true</Value>
</Field>
```

Result

SSO is configured.

Related links
Setting up connection parameters for Innovator (p. 20)

5.3.2 Discussion panel

The **Discussion panel** is an easy-to-use social collaboration method using Aras Innovator Visual Collaboration discussions directly inside Inventor.

To use the discussion panel the package *MSO_Standard* of module Aras Office Connector needs to be imported into the used database. The package can be obtained from Aras.

The *Discussion Panel* is supported with Innovator 12.0 SP0 or higher.

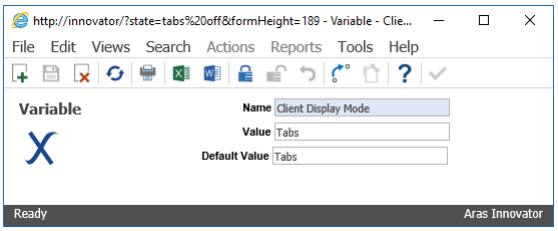
5.3.3 Innovator display mode

Innovator 11.0 SP9 has introduced a new tab viewing mode that allows tear-off windows to load as tabs on the main Innovator window.

The default mode for new databases is tab viewing mode, but windows viewing mode can still be used by configuring global settings. Windows and tabs mode are specified globally for all users by the *Client Display Mode*.

Please refer to *Release Notes* of Innovator 11.0 SP9 (Document #: 11.0.02015040601, section enhancements in Innovator 11.0 SP9) for more details.

Example: Variable client display mode - default setting



6 Configuration

6.1 Innovator-based configuration

6.1.1 Introduction

You can now define connector settings and mapping configurations in Innovator. This new Innovator-based approach has the advantage that configuration is held centrally in Innovator and settings are deployed to all clients automatically when connecting to Innovator. This improves accessibility and consistency across the system.

Introduced or updated with product version 4.5.0.

How it works

Connector settings and mapping configurations are stored in Innovator in own configuration sections. Each setting contains a description explaining its function. During login, the connector settings and mapping configurations are read from Innovator with a single bulk call.

Settings and mappings defined in Innovator override corresponding settings defined in XML files.

When you connect to Innovator for the first time via the integration, the following files are automatically written to disk in the temporary folder `%Temp%\xplm`;

- `XPlmPLMProperties.xml` - stores the settings retrieved from Innovator.
- `XPlmPLMResource.xml`
- `Customer_XPlmArasInventorTransaction.xml` - stores the mapping data retrieved from Innovator.
- `XPlmArasLogonParameter.xml` - stores the connection information to Innovator.
- `XPlmArasInventorDialogs.xml`
- `Customer_XPlmArasInventorDialogs.xml`
- `XPlmArasTransaction.xml`
- `Customer_XPlmArasTransaction.xml`
- `XPlmCopyArasConfiguration.xml`
- `Customer_XPlmCopyArasConfiguration.xml` - stores configurations of the *Copy dialog* retrieved from Innovator.

Once this process is complete, the normal operations of the integration resume seamlessly.

If you encounter any issues during configuration retrieval or synchronization, simply delete the temporary folder, `%Temp%\xplm`, and reconnect to Innovator. This will initiate a full reload of all configuration data from Innovator.

6.1.2 Connector configuration in Innovator

6.1.2.1 AdvancedLevel attribute in XML settings

A new attribute, `AdvancedLevel`, has been introduced in the property XML configuration files to indicate whether a property can be managed in Innovator.

The attribute has the following values:

- `AdvancedLevel="0"` - The property is configurable in Innovator.
- `AdvancedLevel="1"` - The property is not configurable in Innovator.

This enhancement provides clarity during the transition from legacy XML-based configuration to the Innovator-based approach, helping you to identify which settings should be maintained in Innovator and which remain file-based.

6.1.2.2 How to configure settings in Innovator

Complete these steps to configure basic settings in Innovator.

Before you start

Verify that connector settings are imported into the Innovator database, see Importing AML definitions for Innovator-based configuration (p. 17) for more information.

Procedure

1. Login to Aras Innovator web client with the appropriate administrator privileges.
2. Navigate to **Contents** > **Administration** > **Connector Configuration** > **Settings**
3. Click **Search Settings** to search for existing configurations. This opens a new **Settings** tab.
   a) Define the search criteria by entering values in the available field columns (for example, the name of the XML file or the setting itself.)
   b) Click **Search**
      ➔ The window is filled with query results. If no search criteria are defined, this will by default return all entries from the database until the defined search limit is reached.
   c) Right click on the desired setting and click **Open** from the context menu
      ➔ The configuration is open in a new tab
   d) Click **Edit** then change value as desired.
   e) Click **Save** > **Done**

Result

Configuration is set as desired.

What to do next

Test configurations to verify expected behaviour.

6.1.2.3 Defining a new configuration in Innovator

Complete these steps to create a new configuration in Innovator.

Before you start

Verify that connector settings are imported into the Innovator database, see Importing AML definitions for Innovator-based configuration (p. 17) for more information.

Procedure

1. Login to Aras Innovator web client with the appropriate administrator privileges.
2. Navigate to **Contents** > **Administration** > **Connector Configuration** > **Settings**
3. Click **Create New Setting** to define a new configuration.
   ➔ A new **Default** tab is open where you can the configuration.
4. To define the new configuration, fill in the following required parameters
   - **Site:** The location or environment where the settings applies.
   - **Name:** The name of the setting being configured.
   - **Component:** The corresponding XML file associated with the setting.

- **Value:** The assigned value of the setting (for example `true` / `false`, numerical values, or text).
- **Major_rev:** The major revision number of the configuration.
- **State:** The current state of the setting (for example `Preliminary`)
- **Comment:** Any additional notes or explanations regarding the setting.

5. Click **Save** > **Done**

Result

New configuration defined in Innovator

What to do next

Test configurations to verify expected behaviour.

6.1.3 Mapping configuration in Innovator

6.1.3.1 Mapping process

Mapping configurations are stored in Innovator. However, since mapping is performed on the client side, each CAD client needs to have the mapping information locally. Therefore, every time you log in to Innovator via the XPLM Inventor connector, the current valid configuration is downloaded to the client and applied accordingly.

Automatic distribution to Inventor clients

1. When you connect to Innovator using the integration, the mapping is read from Innovator in a single bulk call.
2. Any existing `Customer_XPlmArasInventorTransaction.xml` file is read, ensuring that existing customizations remain unmodified.
3. The mapping retrieved from Innovator is added to the `Customer_XPlmArasInventorTransaction.xml` file in the corresponding FieldCollections of the relevant Transactions.
4. The updated transaction file is written to disk in the temporary folder `%Temp%\xplm`.
5. Normal operation of the Inventor connector is resumed. The `Customer_XPlmArasInventorTransaction.xml` file is read locally during all connector actions such as save, create BOM and update property processes to ensure proper mapping.

   Performance is not impacted by PLM-side mapping. Communication is only done during login action.

The client only downloads the mapping configuration valid for;

- The currently used CAD system.
- The current site
- The `Production` state of the mapping.

Distinction between preliminary and production mapping

The XPLM mapping mechanism supports the use of versioning and life cycle states of mapping objects within Innovator in order to validate a mapping definition on a specific workstation before deploying it to all users.

By default, the Inventor client retrieves the latest mapping definition with the `Released` status.

When you edit the latest released mapping in Innovator, a new revision in `Preliminary` state is automatically created. This does not affect the Inventor clients since they are still using the previous revision in `Production` state.

You can force the XPLM connector to use the `Preliminary` revision of the mapping on a certain workstation by switching the field `State` within the transaction `ArasConfigurationMapping` in the local transaction file `XPlmArasInventorTransaction.xml`. Only this workstation will use the new mapping for testing purposes.

```
<!-- State = Preliminary|Production -->
     <Field>
      <Name>State</Name>
      <Value>Preliminary</Value>
     </Field>
```

After completing the tests and confirming that the mapping is ready for distribution to all clients, you can promote the mapping definition to the `Released` state within Innovator.

6.1.3.2 Defining mapping in Innovator

For each authoring tool, you can define mapping in Innovator in two ways;

1. **Per site**
   You can define mapping configurations based on specific sites. This allows different locations or branches to have unique mappings tailored to their Workflow and requirements.

   The default site named `Default`.
2. **Per action**
   You can also define mappings configurations based on specific actions, such as:
   - **CAD to PLM** - Mapping applied during create or save actions in the Inventor system.
     - **Create 3D** - Used during save from Inventor to Innovator to create new CAD Documents for all 3D formats.
     - **Create 2D** - Used during save from Inventor to Innovator to create new CAD Documents for all 2D formats.
     - **Update Save** - Used during save from Inventor to Innovator for existing CAD Documents.
   - **PLM to CAD** - Mapping applied when transferring data from Innovator to 2D or 3D Inventor files.
     - **Update Properties** - Used during the **Update Properties** command to fetch properties from Innovator and push them to the Inventor 3D model.
     - **Update Titleblock** - Used during the **Update Title Block** command to fetch properties from Innovator and push them to the Inventor 2D model.
   - **Parts/BOM mapping** - Mapping applied when creating or updating parts and Bill of Materials (BOM) in Innovator.
     - **Create Part/BOM** - Used during the **Create Part** or **Create Part and BOM** commands to create new Part items in Innovator.
     - **Update Part/BOM** - Used during the **Create Part** or **Create Part and BOM** commands to update existing Part items in Innovator.

6.1.3.3 Parameters for mapping configuration

When defining mapping configurations, the following parameters must be specified:

1. **CAD file property location**
   - Specifies the group from which the property is read from Inventor.
   - These include custom properties, physical properties or other metadata depending on the authoring tool system.
2. **CAD file property**
   - Defines the actual name of the property in Inventor to be mapped.
   - Examples include `Description`, `Name`, `Material`, `ID` and so on.
3. **Apply function before mapping**
   - Defines the transformations to be applied to the Inventor property before mapping it to Innovator.
   - Examples include `ToUpper`, `FixedValue`, `Left,30` or other predefined functions in Innovator. Refer to Formatting functions in Innovator (p. 74) for a list of all supported functions.

     Good practise is to limit the transferred value to the length of the property in Innovator with the `Left,` -function
4. **PLM item property**
   - Specifies the name of the attribute in Innovator where the mapped value will be stored.
   - Examples include `Description`, `Name`, `PartNumber` and so on.

6.1.3.4 Creating a new mapping definition in Innovator

Complete these steps to create a new mapping definition in Innovator.

About this task

This use case describes how to configure a new mapping definition in Innovator.

Procedure

1. Login to Aras Innovator web client with the appropriate administrator privileges.
2. Navigate to **Contents** > **Administration** > **Connector Configuration** > **Mapping Definitions** > **Create New Mapping Definition**

   ➔ A new **Default** tab is displayed
3. In the mapping definition section:
   a) Define the **Site** for which the mapping is valid. If you are unsure, use `Default`
   b) Choose the authoring tool to which the mapping will apply under the **CAD** drop down list.
   c) Select the direction of the mapping under the **Action** drop down list.
   d) **Optional:** Set a **Revision** for the mapping definition

   Default **State** is set to `Preliminary` in the `XPlmArasInventorTransaction.xml` within the Transaction `ArasConfigurationMapping`.
4. In the mappings section, click **New Mapping** to add a new row to the table
   a) Select the **CAD File Property Location** from the drop down list (for example `CAD File Properties`
   b) Define the **CAD File Property** (for example `Description)`
   c) **Optional:** Specify the transformation to be applied in **Apply Function Before Mapping**
   d) Set the **PLM Item Property** corresponding to the mapped CAD property.
   e) **Optional** add a **Comment**.

   Add as many mapping definitions as required by adding new rows and defining the required parameters.

   You can use the context menu functions **Mappings** > **Copy Row/Paste Row** to quickly add more lines or to copy the complete mapping definition for example from *Create 3D* to *Create 2D*.
5. Click **Save** > **Done**

Result

The new mapping definition is created and saved

What to do next

Test the mapping :

- By applying the mapping to a sample Inventor file
- Verify the mapped properties are correctly transferred to Innovator.

6.1.3.5 Predefined target locations for property mapping

The import package now contains new entries to the `cax_structures` list in Innovator which enable predefined target locations for mapping properties from Innovator to Inventor

Introduced or updated with product version 4.6.0.

The list is available in the extracted archive `Aras-SolutionsImportAML_*\Aras\xplm\cax_configuration\Import\List`.

After import to Innovator, the list is available under **Contents** > **Administration** > **Lists** > **cax_structures**

6.1.4 Custom transaction and property files

With the LowCode approach, two key files are stored in the temporary folder : `Customer_XPlmArasInventorTransaction.xml` and `XPlmPLMProperties.xml`. The integration follows a specific order when reading these files to ensure proper execution.

Introduced or updated with product version 4.5.0.

Reading mapping data

When you log in with the Inventor connector, a bulk call retrieves mapping definitions from Innovator. The order in which the files are read follows the following rules

Transaction file in `%xPlmRootDir%\xml`	**Transaction file in `%temp%\xplm`**	**Result**
NO	NO	Error file not found
YES	NO	use file from `%xPlmRootDir%\xml`
NO	YES	use file from `%temp%\xplm`
YES	YES	use file from `%temp%\xplm`
NO	YES (transaction not found)	error transaction not found
YES	YES (transaction not found)	error transaction not found

Transaction files in `%temp%\xplm` overwrite the copy found in `%xPlmRootDir%\xml`. If there is a transaction file in `%temp%\xplm` it will always be used. If no transaction is present in this file, this will result in an error message (transaction not found).

For the integration to use the `Customer_XPlmArasInventorTransaction.xml` file in `%temp%\xplm`, the main transaction file `XPlmArasInventorTransaction.xml` must also be present in the `%temp%\xplm` folder. This is how the checking routine works. Test for main file, look if there is `Customer_` file in same directory. This ensures that existing mapping definitions remain unmodified.

Processing the property file

When you log in with the Inventor connector, a bulk call retrieves all configurations from Innovator which are then written to `XPlmPLMProperties.xml` file in the `%temp%\xplm` folder.

The table below shows the order in which the property files are read.

Property file in `%xPlmRootDir%\xml`	**Property loader file in `%temp%\xplm`**	**Result**
NO	NO	No property mapping (strict=false)
YES	NO	property mapping from `%xPlmRootDir%\xml`
NO	YES	property replacement from `%temp%\xplm`
YES	YES	property replacement from `%temp%\xplm`

Any property written to the `XPlmPLMProperties.xml` overwrites properties from `%xPlmRootDir%\xml` property files.

6.1.5 Configuring connector dialogs

You can now configure connector dialogs in Innovator.

Introduced or updated with product version 4.7.0.

The following connector dialogs are supported;

- *Save dialog*
- *Load dialog*
- *BOM dialog*
- *Copy dialog*
- *Workspace dialog*

This section explains how to configure;

- Dialog options
- Check Instructions
- Column definitions

6.1.5.1 How to configure dialog options

Complete these steps to configure load and save dialog options in Innovator.

Before you start

Verify that connector settings are imported into the Innovator database, see Importing AML definitions for Innovator-based configuration (p. 17) for more information.

About this task

Dialog Options control the behavior of *Load* and *Save* dialogs.

You can configure the following properties:

- **Visibility** - shown or hidden
- **Editability** - read only or editable
- **Default value** - pre-selected option.

Configurable options for the *Save dialog* include:

- Create Parts for All Configurations
- New Generation
- Update BOM

Configurable options for the *Load dialog* include

- IsLoadFromClient
- Load Read-Only
- Load Top-Level Only
- Rename Files
- Resolution
- Set Reservation
- Show Load Preview
- Update Properties
- Use File Cache

Procedure

1. Login to Aras Innovator web client with the appropriate administrator privileges.
2. Navigate to **Contents** > **Administration** > **Connector Configuration** > **Dialog Options**
3. Click **Search Dialog Options** to search for existing configurations. This opens a new **Dialog Options** tab.
   a) Define the search criteria by entering values in the available field columns (for example, define the dialog in the **Dialog** column.)
   b) Click **Search**
   ➔ The window is filled with query results. If no search criteria are defined, this will by default return all available entries from the database until the defined search limit is reached.
   If a CAD system is specified in the respective column, the dialog option applies only to that CAD system. If no CAD system is specified, the option applies to all supported CAD systems.
   c) Right click on the desired option from the list and click **Open** from the context menu
   ➔ The configuration option is open in a new tab
   d) Click **Edit** then modify the following fields as desired.
   - **Visible** - Check box (checked = visible, unchecked = hidden).
   - **Editable** - Check box (checked = editable, unchecked = read-only).
   - **DefaultValue** - Text field. Values differ between *Save* and *Load* dialog.
     - *Save dialog*: Default values are specified using boolean values (`true/false`).
     - *Load dialog*: Default values are specified using numeric indicators (`0/1`).
     - Exceptions: *Load dialog* option **Resolution** uses predefined keywords, (`AsSaved/Current/Released`), instead of numeric values.

   Example:

   To make **Update Properties** on the *Load dialog* visible, editable and enabled by default:
   - Visible - checked
   - Editable - checked
   - Default Value - value set to `1`

   e) Click **Save** > **Done**

Result

Option is set as desired.

What to do next

Test the affected dialog in the connector to verify expected behaviour.

6.1.5.2 How to configure check instructions

Complete these steps to configure check instructions in Innovator.

Before you start

Verify that connector settings are imported into the Innovator database, see Importing AML definitions for Innovator-based configuration (p. 17) for more information.

About this task

Check Instructions are configured using an XML-snippet ItemType, where the XML defining a check instruction can be specified and the dialog in which it should be used can be selected.

Procedure

1. Login to Aras Innovator web client with the appropriate administrator privileges.
2. Navigate to **Contents** > **Administration** > **Connector Configuration** > **Check Instructions**
3. Click **Search Check Instructions** to search for existing configurations. This opens a new **Check Instructions** tab.
   a) Click **Search**
   ➔ The window is filled with all available entries from the database.
   If a CAD system is specified in the respective column, the configuration applies only to that CAD system. If no CAD system is specified, the option applies to all supported CAD systems.
   b) Right click on the desired setting from the list and click **Open** from the context menu
   ➔ The configuration option is open in a new tab
   c) Click **Edit** then modify the following fields as desired.
   - Adjust the **Name**, **CAD System** and **Site** values as desired.
   - Edit the XML snippet as desired.
   - Under **Add to Dialog**, mark the check box to select the dialog in which the configuration should be applied.

   d) Click **Save** > **Done**

Result

Check Instructions are set as desired.

What to do next

Test the affected dialog in the connector to verify expected behaviour.

6.1.5.3 How to configure column definitions

Complete these steps to configure column definitions for all integration dialogs.

Before you start

Verify that connector settings are imported into the Innovator database, see Importing AML definitions for Innovator-based configuration (p. 17) for more information.

About this task

For each of the integration dialogs, you can configure whether the column is:

- **Visible** - shown or hidden
- **Editable** - read only or editable
- **Property grid** - whether it is added to Property grid of the dialog.
- **Position** - the position of the column in the model table.

Procedure

1. Login to Aras Innovator web client with the appropriate administrator privileges.
2. Navigate to **Contents** > **Administration** > **Connector Configuration** > **Column Definitions**
3. Click **Search Column Definitions** to search for existing configurations. This opens a new **Column Definitions** tab.
   a) Click **Search**
   ➔ The window is filled with all available columns from the database in a table format. The check boxes selected under the dialog columns indicate which options are enabled.

b) Right click on the row of column name from the list and click **Open** from the context menu

➔ The column option is open in a new tab

c) Click **Edit** then modify the following fields as desired for each dialog.
Mark the check box for the dialog in which the column is to be configured and for each dialog, configure the following properties ;

- Visible - Check box (checked = visible, unchecked = hidden).
- Editable - Check box (checked = editable, unchecked = read-only).
- Property Grid - Check box (checked = visible on Property grid, unchecked = hidden from Property Grid).
- Position - number filed which determines the location of the column in the model table.

For the *Search dialog*, you can configure additional options such as **Search field Default Value**, **Sub Property Searches** (dialog tabs) and filter **Operators**

d) Click **Save** > **Done**

Result

The columns are set as desired.

What to do next

Test the affected dialog in the connector to verify expected behaviour.

6.2 Legacy XML-based configuration

6.2.1 Connector configuration settings

The configuration of the connector is done in XML files, which are in the `%xPlmRootDir%\xml` subdirectory of the integration.

These files are associated with a specific component and therefore their names are also affected by these components.

The integration is configured mainly using the following files:

- `XPlmConnector.xml` - Contains basic settings, for example integration language or local work directories.
- `XPlmArasConnector.xml` - contains mostly Innovator related settings
- `XPlmInventorConnector.xml` - base configuration of the Inventor connector .
- `XPlmInventorArasConnector.xml` - contains mostly Innovator related settings.
- `XPlmArasInventorDialogs.xml` - Customization of load and save preview
- `XPlmArasInventorTransaction.xml` - Customization of property mappings.

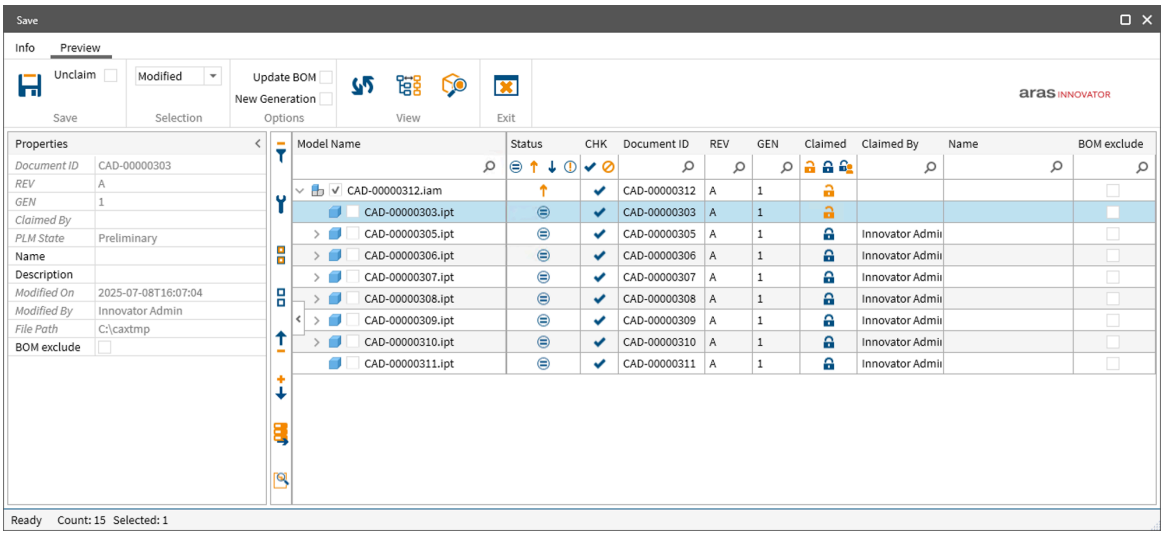
6.2.2 Configuring the Save dialog

The connector comes with default configurations of the *Save* dialog for the save function. However, the UI of the *Save dialog* is configurable to a certain extent.

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** `SavePreviewDialog`

There is only one transaction to configure the UI of the *Save* dialog (`SavePreviewDialog`)

Example *Save* dialog with default configuration:



6.2.2.1 Customizing the layout

You can customize the layout of the *Save* dialog depending on your company's needs.

Columns

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** `SavePreviewDialog`
- **Table:** `Columns`

To customize, only the table node called `Columns` must be modified.

```
<Transaction>
    <Aliasname>SavePreviewDialog</Aliasname>
        <Import>
            <Parameter>
            …
                <StructureCollection>
                …
                    <Structure>
                        <Name>Options</Name>    ← Options configuration
                        …
                    </Structure>
                </StructureCollection>
                <TableCollection>
                    <Table>
                        <Name>Columns</Name>
        ← Please modify only elements of FieldCollectionTable
                        <FieldCollectionTable>
                            <FieldCollection>…    ← FieldCollection of 1st column
                            <FieldCollection>…    ← FieldCollection of 2nd column
                            …    ← FieldCollections of remaining columns
                        </FieldCollectionTable>
…
```

Do not modify other nodes! This may result in unexpected behavior of the *Save dialog*.

Each column in *Save* dialog is assigned to a `FieldCollection` node. For example.

```
<FieldCollection Visible="True" PropertyGrid="False">
    <Field>
        <Name>RowID</Name> ← Column ID
        <Value>5</Value> ← Column position
    </Field>
    <Field>
        <Name>Caption</Name> ← Column label
        <Type>XPlmMessage</Type> ← Source is: XPlmResourceNETDialogs.xml
        <Attribut>NDL0116</Attribut> ← <NDL0116>Generation</NDL0116>
    </Field>
    <Field>
        <Name>Width</Name> ←  Column Width
        <Value>32</Value>
    </Field>
    <Field>
        <Name>Subtype</Name>
        <Value>StructureCollection</Value> ← Contains the structure MetaData
    </Field>
    <Field>
        <Name>Structurename</Name>
        <Value>MetaData</Value> ← Where the content is found
    </Field>
    <Field>
        <Name>Attribut</Name>
        <Value>generation</Value> ← Name of the Innovator property
    </Field>
    <Field>
        <Name>Edit</Name>
        <Value>false</Value> ← Do Not edit
    </Field>
    <Field>
        <Name>ColumnType</Name> ← Type of column
        <Value>FilterImages</Value>
    </Field>
</FieldCollection>
```

- If `Visible` is set to `true` for specific `FieldCollection`, column is visible in structure/list view of *Save* dialog.
- If `PropertyGrid` is set to `true`, row appears in property grid of the *Save dialog*.

Each row requires a unique numerical `RowID` and a unique name. `RowID`'s need to start with `1` and ongoing. Keep in mind there are column dependencies to table context menu items. If `RowID`'s change, menu functions need to be adjusted as well. `RowID`'s may not be changed!

Field `Caption` headlines first column with name `File`, whereas field `Width` defines its width. The information of generation can be found in structure named `Properties`. Usually this is the place where all properties of Innovator objects can be found.

Multi-language configuration

For multi-language issues, it is possible to headline columns in different languages by addressing them to the corresponding entry of the `XPlmResourceNETDialogs.xml` file, which is integrated in `XPlmConnector.xml` file.

It is possible to add a new language in `XPlmResourceNETDialogs.xml` or import own resource file in `XPlmConnector.xml`. For this, add file with a semicolon like shown in following code snippet. Language can be changed by modifying `Language` node of file `XPlmconnector.xml`.

```
<XPlmConnector>
    <Properties>
    …
        <Language>EN</Language>
        …
        <ResourceFiles>XPlmResource.xml;TheOwnResourceFile.xml</ResourceFiles> ← use
 semicolon
    …
```

Toggle buttons

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** `SavePreviewDialog`
- **Table:** `OptionsDialogRibbonToggleButtons`

You can configure the options **New Generation** and **Update BOM** on the *Save dialog* by either setting them to read-only, completely hiding them from the user or preselecting them.

- To hide the options, set `IsVisible` to `false` (Default value is `true`).
- To grey out the options, set `IsEnabled` to `false` (Default value is `true`).
- To preselect options, set `Value` to `false` (Default value is `true`).

6.2.2.2 Column types

Different column types are available in the *Save* dialog

About this task

The following column types are available in the *Save* dialog:

- `CheckText`: Column for text with an additional check box.
- `Image`: Column for images.
- `Text`: Column for strings.
- `FilterImage`: Column for image based values.
- `Checkbox`: Column for boolean values.
- `Combobox`: Column for static multiple choices per row.
- `DynamicCombobox`: Column for alternating multiple choices per row.
- `HyperLink`: Column for hyperlinks.

Following steps demonstrate how to set up combo boxes in *Save* dialog

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** `SavePreviewDialog`

Procedure

1. Define desired fields as shown in following example:

```
<TableCollection>
    <Table>
        <Name>ComboBoxTable1</Name>
        <FieldCollectionTable>
            <FieldCollection>
                <Field>
                    <Name>id</Name>
                    <Value>1</Value>
                </Field>
                <Field>
                    <Name>Caption</Name>
                    <Value>.ipt</Value>
                </Field>
            </FieldCollection>
            <FieldCollection>
                <Field>
                    <Name>id</Name>
                    <Value>2</Value>
                </Field>
                <Field>
                    <Name>Caption</Name>
                    <Value>.iam</Value>
                </Field>
            </FieldCollection>
        </FieldCollectionTable>
    </Table>
</TableCollection>
```

2. Add a column `ColumnType` with value `Combobox` and define mapping.

```
<Table>
    <Name>Columns</Name>
    <FieldCollectionTable>
        <FieldCollection Visible="True" PropertyGrid="False">
          .....
            <Field>
                <Name>ColumnType</Name>
                <Value>Combobox</Value>
            </Field>
            <Field>
                <Name>Width</Name>
                <Value>200</Value>
            </Field>
            <Field>
                <Name>Subtype</Name>
                <Value>FieldCollection</Value>
            </Field>
            <Field>
                <Name>Attribut</Name>
                <Value>CBxSelectedItemType</Value>
            </Field>
            <Field>
                <Name>Edit</Name>
                <Value>true</Value>
            </Field>

            <Field>
                <Name>ComboboxOptions</Name>
                <Value>ComboBoxTable1</Value>
            </Field>
            <Field>
                <Name>FilterType</Name>
                <Value>Distinct</Value>
            </Field>
        </FieldCollection>
    </FieldCollectionTable>
</Table>
```

Result

Combo boxes are set up successfully.

What to do next

Restart the *Save* dialog and verify the combo boxes are available in the specified column.

6.2.2.3 Customizing the context menu

Context menu functions can also be executed via a hot-key combination which you can configure as desired.

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** `SavePreviewDialog`
- **Table:** `ContextMenuItems`

To use a desired hot-key combination, a field needs to be added to the context menu operation. It is also possible to change existing hot-key combinations.

Following must be considered:

- Field must contain at least one or maximum four modifier keys. The different modifier keys attributes must end with an unique index like `1,2,3,4`, e.g. `<Field ModifierKey1="Control" ModifierKey2="Alt">`. Modifier key values can be: **Control**, **Alt**, **Shift** and **Windows**.
- Field name must be `Hotkey` and must not be changed.
- Field value must be a key enumeration (System.Windows.Input).
- Make sure not using an already used hot-key combination.

A successful defined hot-key combination appears in brackets behind context menu operation.

```
<Transaction>
    <Aliasname>SavePreviewDialog</Aliasname>
        <Import>
            <Parameter>
            …
                <TableCollection>
                  <Table>
                    <Name>ContextMenuItems</Name>
                    <FieldCollectionTable>
                      <FieldCollection>
                        ...
                        <Field>
                          <Name>PostActionUserExit</Name>
                          <Value></Value>
                        </Field>
                        <Field ModifierKey1="Control">  ← Modifier keys
                          <Name>Hotkey</Name> ← Name of the field (do not change)
                          <Value>Space</Value> ← key enumeration
                        </Field>
                      </FieldCollection>
                      ...
                    </FieldCollectionTable>
                ...
```

Context menu behavior on disabled rows

- Table: `ContextMenuItems`

A new Attribute Condition with value `CLB` has been introduced to govern the behaviour of the context menu for files without collaboration data (CLB files).

If the `CLB` condition is set, context menu functions are disabled for rows that do not contain collaboration data.

```
<Field Destination="SAVEPREVIEW" Condition1="PLM" Condition2="CLB">
   <Name>Callback</Name>
   <Value>ActivateWorkspaceModified</Value>
</Field>
```

6.2.2.4 Save dialog options

You can customize the behaviour and functionality of the *Save dialog*.

- **Configuration file:** `XPlmArasInventorDialogs.xml`

- **Transaction:** `SavePreviewDialog`
- **Structure:** `Options`

The following options are available for customization;

Name	Description
CheckInitialPLMConnection	If set to `true`, a check for an existing connection to Innovator is executed. It is set to true by default. **Default: true** **Possible values: true | false**
CacheRows	Defines the filling methods of the rows. If set to true, the rows are filled with values during initialization (increase of performance when scrolling). If set to false, the rows are filled with values during scrolling (for faster dialog loading in case of giant assemblies). **Default: false** **Possible values: true | false**
Logging	
MessageBuffer	Concerns logging: Defines the amount of the max. rows in system messages. **Default:** 50
LogFile	Concerns logging: Defines the path to the desired basic log file. **Default:** empty
TimeLogFile	Concerns logging: Defines the path to the desired time log file. **Default:** empty
SerializeDataModelFile	Concerns logging: Defines the path to the desired serialization file. **Default:** empty
Visual	
PropertyGridSupport	If set to `true`, row based values of text columns are shown in a property grid. **Default:** true **Possible values:** true | false
WindowBinding	If set to `true`, this options sets the calling WND as parent of the dialog and brings the dialog on top. **Default:** true **Possible values:** true | false
MessageBoxBehavior	If set to `Parent`, message box is attached to parent. If set to `Modal`, message box appears model. If set to `Free`, message box is attached to nothing. **Default:** Parent **Possible values:** Parent | Modal | Free
Width	Defines the initial dialog width. Is used if no user profile is available. **Default:** 1510

Name	Description
Height	Defines the initial dialog height. Is used if no user profile is available. **Default:** 750
AlternationCount	Defines the count of the alternate visual row style. **Default:** 2
AlternateRowBackground	Defines the alternate row color. **Default:** #FFF5F5F5
ShowColumnFooters	This option is currently not used. **Default:** false **Possible values:** true | false
ScrollMode	Defines which rows are shown when scrolling. Three values are available: ■ `RealTime`: All rows are visible (poor performance). ■ `Mixed`: Only freezed columns are visible (balanced performance). ■ `Deferred`: All rows are disabled (best performance). **Default:** RealTime **Possible values:** RealTime | Mixed | Deferred
UpdateProgressBarIntervall	Defines the percentage progress bar refresh interval. In case of giant assemblies, small values cause performance looses. **Default:** 10
AutoExpandToBeSavedItems	Toggles expansion of documents with an expected attribute value (on/off). **Default:** false **Possible values:** true | false
RowHeight	Defines the height of the rows. **Default:** 25
SideBar	Toggles the control panel on the left side of the dialog. **Default:** true **Possible values:** true | false
EnableSplashScreen	Option toggles the splash screen of dialog on or off. **Default:** true **Possible values:** true | false
Workspace	
WorkSpaceDelete FileAccessCheck	If set to `true`, access is checked before deleting a file. **Default:** false **Possible values:** true | false
Columns	

Name	Description
FrozenColumnCount	Defines the number of columns on the left side, which are expected when scrolling horizontally. **Default:** 2
CanUserFreezeColumns	Defines if the user is allowed to change the number of frozen columns. **Default:** true **Possible values:** true | false
FrozenColumnsSplitterVisibility	Defines the visibility of the separator between frozen and unfrozen columns. **Default:** visible
FilteringMode	Toggles filtering fields in the column headers (on/off). **Default:** true **Possible values:** true | false
UserProfileMode	Toggles user profiles (on/off). Controls column visibility, column width and dialog position. **Default:** true **Possible values:** true | false
Logic	
AutoOk	If set to `1`, the dialog starts hidden and closes automatically. If set to 0, the dialogs starts as window. **Default:** 0 **Possible values:** 0 | 1
SaveMode	Defines which files are selected for saving. The following values are available: ■ `0`: Nothing is selected/saved. ■ `1`: Only new/modified documents are selected/saved. ■ `2`: All documents are selected/saved. ■ `3`: Only locked documents are saved. **Default:** 1 **Possible values:** 0 | 1 | 2 | 3
AutoLock	If set to `true`, all modified documents are locked in PLM automatically on dialog startup. **Default:** false **Possible values:** true | false

Name	Description
GetMetadata	Defines the metadata update options on dialog start-up. The following values are available: ■ `None`: No documents are updated. ■ `Modified`: Only modified documents are updated. ■ `All`: All documents are updated. **Default:** Modified **Possible values:** None | Modified | All
GetMetadataPreActionUserExit	Defines if user exit is called before `GetMetadata` on dialog start-up. **Default:** empty
GetMetadataPostActionUserExit	Defines if user exit is called after `GetMetadata` on dialog start-up. **Default:** empty
EventTransactionFile	Defines the name of the corresponding transaction file. **Default:** XPlmArasInventorTransaction.xml
PreActionAddLineNotify	Toggles the `PreActionAddLine` user exit on/off. In case of giant assemblies `PreActionAddLineNotify` results in a descend performance. **Default:** false **Possible values:** true | false
ModificationLevelParent	Number of superordinate assembly documents that are marked for saving to Innovator. Can be overwritten by the `ModifyFather` property of the Innovator connectors. **Default:** 1
ModificationLevel ParentDisabled	Number of superordinate assembly documents after which `ModificationParent` should be aborted. Can be overwritten by the `ModifyFather` property of the PLM connectors. **Default:** 1
AllowSaveForLocked DocumentsOnly	If set to true, user can only save locked objects. **Default:** false **Possible values:** true | false
AllowSaveForFailed CheckInstructions	If set to `true`, the user can save documents when save checks fail. **Default:** false **Possible values:** true | false
UnlockOnSaveCheck	If set to `true`, locked and workspace modified objects are checked for unlock on initial opening of the *Save Preview*. The check can be triggered manually by opening the context menu of the unlock on save column (RM on column header). **Default: false** **Possible values: true | false**

Name	Description
BeforeDataLoadedUserExit	User exit called on dialog startup, syntax `ScriptEngine.CScriptEngine@CallbackMethod`. **Default:** [empty]
BeforeShowDialogUserExit	User exit called on dialog startup, syntax `ScriptEngine.CScriptEngine@CallbackMethod`. **Default:** empty
IsViewPreview	If set to `true`, dialog opens without function **Save**, toggle buttons, combo box. **Default:** false **Possible values:** true | false
WorkspaceModified UserLockCheck	Checks if a modified file is locked by another user. **Default: true** **Possible values: true | false**
WorkspaceModifiedStrict UserLockCheck	If this option is set to `true`, the user can only save objects reserved by himself. **Default: false** **Possible values: true | false**
DatabaseRecordExistenceCheck	If set to `true`, the integration checks if the database has changed. **Default: true** **Possible values: true | false**
GetAccessCheck	If set to `true`, the integration checks if the user has permissions to save the desired object. **Default: true** **Possible values: true | false**
IsCurrentCheck	If set to `true`, the *Save Preview* it determines if the Innovator `is_current` property set to 1. If it is not set to 1, the row is marked with failed check instruction **Default: true** **Possible values: true | false**
StatusCheck	Detects errors caused by objects modified in Innovator or locally. **Default: true** **Possible values: true | false**

CheckInstructions

- **Transaction:** `SavePreviewDialog`
- **Structure:** `CheckInstructions`

If one of these checks fails, the affected objects are not preselected for saving even though they have been modified. Manual reselection is possible, but not recommended.

Configuration of checks

`Resource`: The displayed text (mouse over) can be changed in `XPlmResourceNETDialogs.xml`.

`Icon`: Is the displayed icon for the failed check.

`ShowIcon`: If set to `true`, icon is shown for failed check. If set to `false` check is executed but the icon is not displayed.

`Name`: Do not change the name of the check.

`Value`: Described in the following rows.

```
<Field Resource="NDL0134" Icon="CheckOutUser_transparent_16x16.png" ShowIcon="True">

    <Name>WorkspaceModifiedUserLockCheck</Name>
    <Value>True</Value>
</Field>
```

6.2.3 Configuring the Load dialog

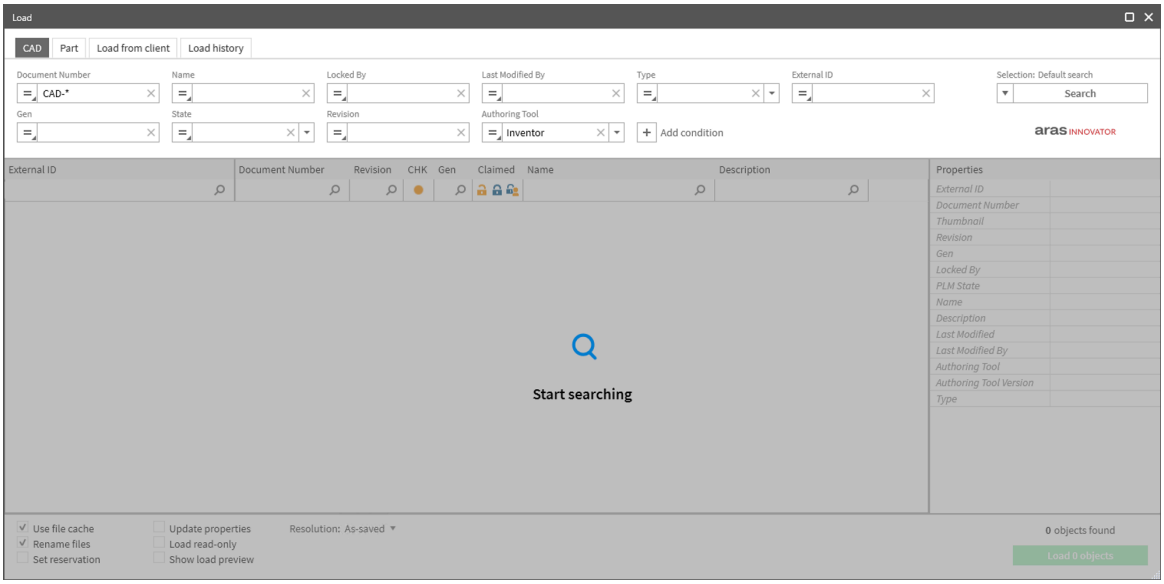
The connector comes with default configurations of the *Load dialog* for the load function. With this, the user can explore active objects in Innovator.

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** `SearchDialog`, `SearchDialogCAD`, `SeachDialogPart`, `SearchDialogAssign`, `SearchDialogCADLoadFromClient`
- **Annotation:** One transaction to configure the UI, several transactions to control the displayed UI elements

One transaction to configure the UI of the *Search* dialog (`SearchDialog`) and several smaller transactions (for example, `SearchDialogCAD`, `SearchDialogPart`, …) that control the displayed UI configuration.

First the main `SearchDialog` transaction must be configured and after that the control transactions. The data of the control transaction are mapped in the `SearchDialog` transaction. For example, if a tab is visible or not or which is the preselected tab and search.

Example *Load dialog* with default configuration, active tab: CAD



For customer specific UI configuration there are ten predefined callback methods in the CMainModule of the script engine (`arasLoadFromDocumentCustomer[1 - 5]` and `arasLoadFromPartCustomer[1 - 5]`).

To use them a control transaction must be added for the specific callback.

- For `arasLoadFromDocumentCustomer[1 - 5]` the control transaction must be named `SearchDialogCADCustomer[1 - 5]`.

- For `arasLoadFromPartCustomer[1 - 5]` the control transaction must be named `SearchDialogPartCustomer[1 - 5]`.

6.2.3.1 Tab configuration

The *Load dialog* consists of several tabs which can be configured individually.

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** SearchDialog transactions
- **Table:** `DialogTabs`

The connector comes with default configuration for the tabs *CAD*, *Part*, *Load From Client*, *Load History* and *Assign*.

To customize, only the table node `DialogTabs` must be modified.

Each tab in the *Load dialog* is assigned to a `FieldCollection` node. A sequence of this `FieldCollection` is shown in following example.

```
<FieldCollection>
  <Field>
    <Name>ID</Name> ← ID of tab
    <Value>CAD</Value> ← unique identifier string
  </Field>
  ...
  <Field>
    <Name>IsEnabled</Name>  ← indicates if the tab is enabled
    <Type>ParameterList</Type>
    <Subtype>FieldCollection</Subtype>
    <Attribut>CAD_IsEnabled</Attribut> ← mapping value from the control transaction

  </Field>
  <Field>
    <Name>IsVisible</Name> ← indicates if the tab is visible
    <Type>ParameterList</Type>
    <Subtype>FieldCollection</Subtype>
    <Attribut>CAD_IsVisible</Attribut> ← mapping value from the control transaction

  </Field>
  <Field>
    <Name>AutoSearch</Name> ← defines if search is triggered automatically
    <Type>ParameterList</Type>
    <Subtype>FieldCollection</Subtype>
    <Attribut>CAD_ AutoSearch </Attribut> ← mapping value from the control transaction
  </Field>
  ...
</FieldCollection>
```

- If `IsVisible` is set to `true` for specific `FieldCollection`, tab is visible in the search dialog. The value is mapped from the control transaction.
- If `IsEnabled` is set to `true` for specific `FieldCollection`, tab is enabled in the *Search Dialog*. The value is mapped from the control transaction.
- If `AutoSearch` is set to `true` for specific `FieldCollection`, the search is triggered automatically when the tab is selected. The value in this example is mapped from the control transaction.

In the `ColumnDefinition` field the name of the table where the columns are defined in the configuration is set. Every tab can have his own column definition. The same behavior is working for the `ContextMenuDefinition`.

6.2.3.2 Load options

The load options in the lower part of the *Load dialog* can be configured separately for each tab.

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** SearchDialog transactions
- **Table:** `LoadOptions`

For customizing purposes, only the table node `LoadOptions` must be modified. Each load option is assigned to a `FieldCollection` node.

A load option can be a check box or a combo box. To define a combo box the `Type` node must be set to `ComboBox` and the name of the combo box table must be set to the `Items` node.

Alternatively, the data for the combo box can come from a script engine. For this purpose, the `Items` node must be set to `ScriptEngine.CScriptEngine@CallbackMethod`. The method must have an `XPlmDocument` as input and output. The data for the combo box must be a `Table` in the `TableCollection` of the output and must be the same structure.

Related links
Combo box configuration (p. 49)

6.2.3.3 Search fields

For each item type, you can define a list of search fields.

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** SearchDialog transactions
- **Table:** `SearchFields_<ItemType>`

After definition, these search fields are available in the selection list of the tabs which have the same *ItemType* value. In addition, they can be used to define predefined searches.

For customizing purpose, only the table nodes called `SearchFields_<ItemType>` must be modified. Each search field is assigned to a `FieldCollection` node.

A search field can be a text box, date picker or a combo box. To define a combo box the `Type` node must be set to `ComboBox` and an `Attribute` to this `Field` with name `DataSource` must be added. This `Attribute` has the name of the combo box table.

Alternatively, the data for the combo box can come from a script engine. For this purpose, the `DataSource Attribute` must be set to `ScriptEngine.CScriptEngine@CallbackMethod`. The method must have an `XPlmDocument` as input and output. The data for the combo box must be a `Table` in the `TableCollection` of the output and have to be the same structure.

Related links
Combo box configuration (p. 49)

6.2.3.4 Combo box configuration

About this task

Following steps show how to set up combo boxes in the *Load dialog* using the example of a load option.

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** SearchDialog transactions

Procedure

1. Define desired fields as extra `Table` as shown in following example:

```
<Table>

  <Name>ComboBoxLoadOptionsResolution</Name>
  <FieldCollectionTable>
    <FieldCollection>

      <Field>
        <Name>Caption</Name>
        <Type>XPlmMessage</Type>
        <Attribut>NDL0192</Attribut>
      </Field>
      <Field>
        <Name>ID</Name>
        <Value>AsSaved</Value>
      </Field>
    </FieldCollection>
   ......
  </FieldCollectionTable>
</Table>
```

2. Add a load option with `Type` `ComboBox` and set `Items` to `ComboBoxLoadOptionsResolution`:

```
<FieldCollection>
 ....
  <Field>
    <Name>Type</Name>
    <Value>ComboBox</Value>
  </Field>
  <Field>
    <Name>Items</Name>
    <Value>ComboBoxLoadOptionsResolution</Value>
  </Field>
  ....
</FieldCollection>
```

Result

After restarting the *Load dialog*, the combo box is available.

6.2.3.5 Predefined searches

A list of predefined searches can be defined for each *ItemType*.

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** SearchDialog transactions
- **Table:** `Searches_<ItemType>`

After definition, these predefined searches are available in the drop-down menu under the search button.

For customizing purpose, only the table nodes called `Searches_<ItemType>` must be modified. Each predefined search is assigned to a `FieldCollection` node. You can only use search fields from the same *ItemType* to define a predefined search.

6.2.3.6 Columns

Each tab of the *Load dialog* can be configured with its own column definition.

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** SearchDialog transactions

For customizing purposes, only the table node that is filled at the Tab configuration (p. 48) must be modified. Each column is assigned to a `FieldCollection` node.

Different column types are available in *Load dialog*:

- `CheckTextImage`: Column for text with an additional check box and image.
- `Image`: Column for images.
- `Text`: Column for strings.
- `FilterImage`: Column for image-based values.
- `Checkbox`: Column for boolean values.
- `Combobox`: Column for static multiple choices per row.
- `Decimal`: Column for decimals.

To define a combo box the `ColumnType` node must be set to `ComboBox` and the name of the combo box table must be set to the `ComboboxOptions` node.

Alternatively, the data for the combo box can come from a script engine. For this purpose, the `ComboboxOptions` node must be set to `ScriptEngine.CScriptEngine@CallbackMethod`. The method must have an `XPlmDocument` as input and output. The data for the combo box must be a `Table` in the `TableCollection` of the output and must be the same structure.

Related links
Combo box configuration (p. 49)

6.2.3.7 Context menu

Context menu functions can also be executed via a hot-key combination which you can configure as desired..

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** `SearchDialog`
- **Table:** `ContextMenuItems`

For customizing purposes, only the table node that is filled at the Tab configuration (p. 48) must be modified. Each column is assigned to a `FieldCollection` node.

To use a desired hot-key combination, a field needs to be added to the context menu operation. It is also possible to change existing hot-key combinations.

Following must be considered:

- Field must contain at least one or maximum four modifier keys. The different modifier keys attributes must end with an unique index like `1,2,3,4`, e.g. `<Field ModifierKey1="Control" ModifierKey2="Alt">`. Modifier key values can be: **Control**, **Alt**, **Shift** and **Windows**.
- Field name must be `Hotkey` and must not be changed.
- Field value must be a key enumeration (`System.Windows.Input`).
- Make sure not using an already used hot-key combination.

A successful defined hot-key combination appears in brackets behind context menu operation.

```
<Transaction>
  <Aliasname>SearchDialog</Aliasname>
    <Import>
      <Parameter>
      …
        <TableCollection>
          <Table>
            <Name>ContextMenuItems</Name>
            <FieldCollectionTable>
              <FieldCollection>
              ...
                <Field>
                  <Name>PostActionUserExit</Name>
                  <Value></Value>
                </Field>
                <Field ModifierKey1="Control">  ← Modifier keys
                  <Name>Hotkey</Name> ← Name of the field (do not change)
                  <Value>Space</Value> ← key enumeration
                </Field>
              </FieldCollection>
              ...
            </FieldCollectionTable>
  ...
```

6.2.3.8 Load dialog options

You can customize the behaviour and functionality of the *Load dialog*.

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction tags:** `SearchDialog`
- **Structure:** `Options`

The following options are available for customization;

Name	Description
Logging	
LogFile	Concerns logging: Defines the path to the desired basic log file. **Default:** empty
Visual	
PropertyGridSupport	If set to true, row based values of text columns are shown in a property grid. **Default:** true **Possible values:** true | false
Width	Defines the initial dialog width. Is used if no user profile is available. **Default:** 1510

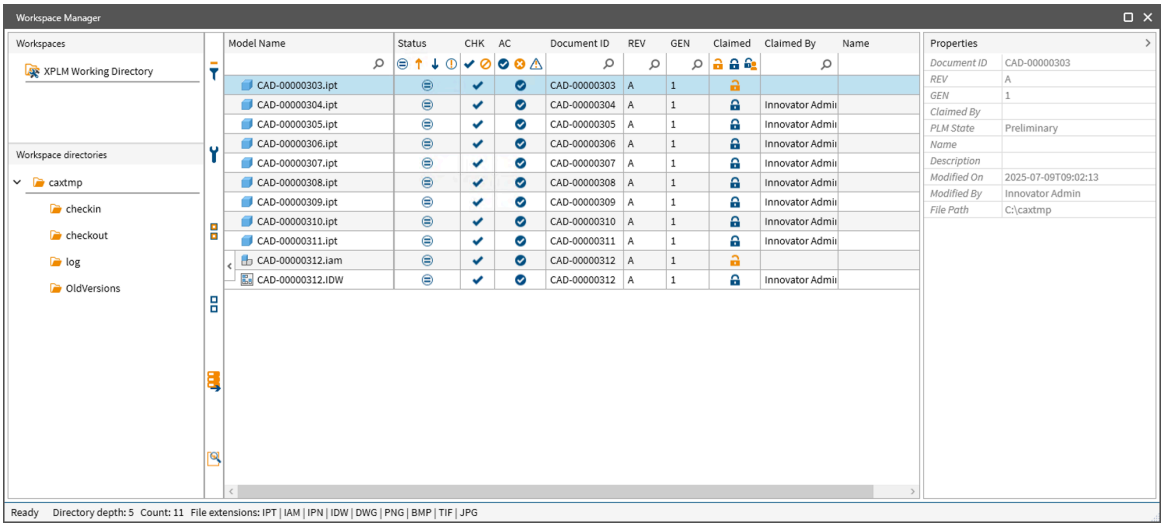
Name	Description
Height	Defines the initial dialog height. Is used if no user profile is available. **Default:** 750
AlternationCount	Defines the count of the alternate visual row style. **Default:** 2
AlternateRowBackground	Defines the alternate row color. **Default:** #FFF5F5F5
RowHeight	Defines the height of the rows. **Default:** 25
Columns	
FrozenColumnCount	Defines the number of columns on the left side, which are expected when scrolling horizontally. **Default:** 2
CanUserFreezeColumns	Defines if the user is allowed to change the number of frozen columns. **Default:** true **Possible values:** true | false
FrozenColumnsSplitterVisibility	Defines the visibility of the separator between frozen and unfrozen columns. **Default:** visible
Logic	
EventTransactionFile	Defines the name of the corresponding transaction file. **Default:** XPlmArasInventorTransaction.xml

6.2.4 Configuring the Workspace manager dialog

The connector comes with default configurations of the *Workspace manager dialog*. However, you can configure the UI of the *Workspace manager dialog* to a certain extent.

- **Configuration file:** `XPlmArasInventorDialogs.xml`, `XPlmWorkspaceConfiguration.xml`
- **Transaction:** `WorkspaceDialog`

Example *Workspace manager dialog* dialog with default configuration:



6.2.4.1 Customize the layout

You can customize the layout of the *Workspace browser* depending on your company's needs.

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** `WorkspaceDialog`
- **Table:** `Columns`

To customize, only the table node called `Columns` must be modified.

```
<Transaction>
    <Aliasname>WorkspaceDialog</Aliasname>
        <Import>
            <Parameter>
            …
                <StructureCollection>
                …
                    <Structure>
                        <Name>Options</Name>  ← Options configuration
                        …
                    </Structure>
                </StructureCollection>
                <TableCollection>
                    <Table>
                        <Name>Columns</Name>
          ← Please modify only elements of FieldCollectionTable
                        <FieldCollectionTable>
                            <FieldCollection>…    ← FieldCollection of 1st column
                            <FieldCollection>…    ← FieldCollection of 2nd column
                            …    ← FieldCollections of remaining columns
                        </FieldCollectionTable>
…
```

Do not modify other nodes! This may result in unexpected behavior of the *Workspace Browser*.

Each column in *Workspace Browser* is assigned to a `FieldCollection` node. For example,

```
<FieldCollection Visible="True" PropertyGrid="False">
    <Field>
        <Name>RowID</Name> ← Column ID
        <Value>5</Value> ← Column position
    </Field>
    <Field>
        <Name>Caption</Name> ← Column label
        <Type>XPlmMessage</Type> ← Source is: XPlmResourceNETDialogs.xml
        <Attribut>NDL0116</Attribut> ← <NDL0116>Generation</NDL0116>
    </Field>
    <Field>
        <Name>Width</Name> ← Width of the column
        <Value>32</Value>
    </Field>
    <Field>
        <Name>Subtype</Name>
        <Value>StructureCollection</Value> ← Contains the structure MetaData
    </Field>
    <Field>
        <Name>Structurename</Name>
        <Value>MetaData</Value> ← Where the content is found
    </Field>
    <Field>
        <Name>Attribut</Name>
        <Value>generation</Value> ← Name of the Innovator property
    </Field>
    <Field>
        <Name>Edit</Name>
        <Value>false</Value> ← Not possible to edit
    </Field>
    <Field>
        <Name>ColumnType</Name> ← Type of column
        <Value>FilterImages</Value>
    </Field>
</FieldCollection>
```

- If `Visible` is set to `true` for specific `FieldCollection`, column is visible in structure/list view of *Workspace browser*.
- If `PropertyGrid` is set to `true`, row appears in property grid of the *Workspace browser*.

Each row requires a unique numerical `RowID` and a unique name. `RowID`'s need to start with `1` and ongoing. Keep in mind there are column dependencies to table context menu items. If `RowID`'s change, menu functions need to be adjusted as well. `RowID`'s may not be changed!

Field `Caption` headlines first column with name `File`, whereas field `Width` defines its width. The information of generation can be found in structure named `Properties`. Usually this is the place where all properties of Innovator objects can be found.

For multi-language issues, it is possible to headline columns in different languages by addressing them to the corresponding entry of the `XPlmResourceNETDialogs.xml` file, which is integrated in `XPlmConnector.xml` file.

It is possible to add a new language in `XPlmResourceNETDialogs.xml` or import own resource file in `XPlmConnector.xml`. For this, add file with a semicolon like shown in following code snippet. Language can be changed by modifying `Language` node of file `XPlmconnector.xml`.

```
<XPlmConnector>
    <Properties>
    …
        <Language>EN</Language>
        …
        <ResourceFiles>XPlmResource.xml;TheOwnResourceFile.xml</ResourceFiles> ← use
 semicolon
    …
```

6.2.4.2 Customize the context menu

Context menu functions can also be executed via a hotkey combination which you can configure as desired..

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** `WorkspaceDialog`
- **Table:** `ContextMenuItems`

To use a desired Hotkey combination, a field needs to be added to the context menu operation. It is also possible to change existing hotkey combinations.

Following must be considered:

- Field must contain at least one or maximum four modifier keys. The different modifier keys attributes must end with an unique index like `1,2,3,4`, e.g. `<Field ModifierKey1="Control" ModifierKey2="Alt">`. Modifier key values can be: **Control**, **Alt**, **Shift** and **Windows**.
- Field name must be `Hotkey` and must not be changed.
- Field value must be a key enumeration (System.Windows.Input).
- Make sure not using an already used Hotkey combination.

A successful defined Hotkey combination appears in brackets behind context menu operation.

```
<Transaction>
    <Aliasname>WorkspaceDialog</Aliasname>
        <Import>
            <Parameter>
            …
                <TableCollection>
                  <Table>
                    <Name>ContextMenuItems</Name>
                    <FieldCollectionTable>
                      <FieldCollection>
                        ...
                        </Field>
                        <Field ModifierKey1="Control">  ← Modifier keys
                          <Name>Hotkey</Name> ← Name of the field (do not change)
                          <Value>Space</Value> ← key enumeration
                        </Field>
                      </FieldCollection>
                      ...
                    </FieldCollectionTable>
                  ...
```

6.2.4.3 User-defined workspaces

Within the *Workspace Browser*, it is possible to define custom workspaces. This allows to specify which files types are shown in the *Workspace Browser*.

- **Configuration file:** `XPlmWorkspaceConfiguration.xml`
- **Tables:** `CAXTypes`, `Workspaces`

CAXTypes

In table `CAXTypes`, the file extensions are divided in groups, named `FieldCollection`. Each `FieldCollection` represents one CAD system and its file types. Other file types are also defined, for example, documents and viewables. See code block below.

The `Name` of the `FieldCollection` is used to configure user-defined workspaces.

```
<Table>
 <Name>CAXTypes</Name>
 <FieldCollectionTable>
  <FieldCollection Name="-"> ← Name of the CAXType
   <Field>
    <Name>FileExtensions</Name>
    <Value></Value>  ← CAD file extensions
   </Field>
   <Field>
    <Name>ForceApplicationLaunch</Name>
    <Value>False</Value>
   </Field>
   <Field>
    <Name>ProgID</Name>
    <Value></Value>
   </Field>
   <Field>
    <Name>CadIdentifier</Name>
    <Value></Value>  ← CAD identifier
   </Field>
  </FieldCollection>
  ....
  </FieldCollection>
  <FieldCollection Name="Documents">
   <Field>
    <Name>FileExtensions</Name>
    <Value>PDF|TXT|DOC|DOCX|XML</Value>
   </Field>
  </FieldCollection>
              ....
 </FieldCollectionTable>
</Table>
```

Workspaces

In table `Workspaces`, multiple user-defined workspaces can be configured. A sample is already given in the configuration file. To define a custom workspace, uncomment the example and change the following options:

Name	Value
MaxDepth	This option sets the folder structure depth. **Default value: 5** **Possible value: Any valid integer** Higher values can lead to performance loss.
IsActive	To display the configured workspace in *Workspace Browser*, set this option to `true`. **Possible values: true | false**
DirectoryImage	Image shown in the folder structure in *Workspace Browser*. This change is optional.
Name	Name of the custom workspace displayed in *Workspace Browser*.
Directory	Path to the custom workspace, e.g. `C:\Custom Workspace`. ■ Network drives as UNC (uniform naming convention) paths are supported. ■ Root drives are not supported.
CAXTypes	Here the file types are defined that should be displayed in *Workspace Browser*. Multiple values are possible, separate them with a pipe. **Example value: SolidWorks|NX|Documents**
Image	Image shown in the list of workspace in *Workspace*. This change is optional.

The following code block shows the defined workspace. This workspace shows only files of Dassault Systèmes SOLIDWORKS, Siemens NX and file of type documents and has the name *Custom Workspace*.

```
<FieldCollection IsActive="true" DicretoryImage="OpenWorkspace_transparent_32x32.png">
 <Field>
  <Name>Name</Name>
  <Value>Custom Workspace</Value> ← Name of the workspace
 </Field>
 <Field>
  <Name>Directory</Name>
  <Value>C:\Custom Workspace</Value> ← Path of the workspace
 </Field>
 <Field>
  <Name>CAXTypes</Name>
  <Value>SolidWorks|NX|Documents</Value> ← Names of the CAXTypes
 </Field>
 <Field>
  <Name>Image</Name>
  <Value>WorkspaceManager_transparent_32x32.png</Value>
 </Field>
</FieldCollection>
```

To add further user-defined workspaces, copy the `FieldCollection` and make necessary adjustments.

Default workspace

It is also possible to deactivate the default workspace (`C:\caxtmp`). To do so, set `XPlmWorkDirInclude` to `false` in table `Workspaces`.

It is not possible, to specify displayed file types for default workspace.

6.2.5 Configuring the Copy dialog

The connector comes with default configurations of the *Copy* dialog to copy existing documents/ document structures.

- **Configuration file:** `XPlmCopyArasConfiguration.xml`
- **Transaction:** `CopyDialog`

The transaction `CopyDialog` is the main transaction which configures the top ribbon and the layout of the different grids (tree list views). Each tree list view is configured via a single transaction. The main `CopyDialog` transaction refers to the single transactions of the tree list views in the table `Layout` to configure them.

The main tree list view for the CAD documents is configurable via the transaction `DocumentsConfig`. The tree list view which displays the related drawings is configurable via the transaction `DrawingConfig`. The tree list view which displays the related parts is configurable via the transaction `PartConfig`.

The configuration of the tree list views is the same as the configuration of the tree list view in the *Save* dialog. The columns, property grid, check instructions, file extension images and context menus can be configured for each tree list view separately. In the `Options` structure, the styling and functionality of the tree list view can be set individually.

The general settings of the dialog are set via the main `CopyDialog` transactions e.g., the logging, the dialog dimensions, enable/disable the user profile or splash screen or set the event transaction file. Also, the `DataTypeNames` structure is only set there.

6.2.5.1 Property columns

The configuration of the tree list views is the same as what you see in the *Save* dialog. To configure a column that values will be used and added to the newly created copied document, you must bind it to the document's `UpdatePropertiesCopy` structure. Only the properties within this structure will be considered.

Below is an example of how to configure such a column;

```
<FieldCollection Visible="True" PropertyGrid="True">
<Field>
  <Name>RowID</Name>
  <Value>NewDocNum</Value>
 </Field>
 <Field>
  <Name>Caption</Name>
  <Value>New Doc Number</Value>
 </Field>
 <Field>
  <Name>ToolTip</Name>
  <Value>New document number for copied document</Value>
 </Field>
 <Field>
  <Name>Subtype</Name>
  <Value>StructureCollection</Value>
 </Field>
 <Field>
  <Name>Structurename</Name>
  <Value>UpdatePropertiesCopy</Value> # The mapping to this structure is mandatory
 </Field>
 <Field>
  <Name>Attribut</Name>
  <Value>item_number</Value>
 </Field>
 <Field>
  <Name>Width</Name>
  <Value>150</Value>
 </Field>
 <Field>
  <Name>Edit</Name>
  <Value>true</Value> # To edit the properties this field needs to be true
 </Field>
 <Field>
  <Name>ColumnType</Name>
  <Value>Text</Value>
 </Field>
 <Field>
  <Name>FilterType</Name>
  <Value>Text</Value>
 </Field>
</FieldCollection>
```

6.2.5.2 Excluding elements from copying

Excluding documents

It is possible to exclude customer specific documents (e.g templates or standard parts) from the *Copy* dialog.

For this purpose, the user exit `OnBeforeInvokeDialog` must be enabled. In the customer script engine, each document of the pInput document structure which should be excluded needs the following field in the `FieldCollection`:

```
<Field>
 <Name>ExcludeFromCopy</Name>
 <Value>1</Value>   # with Value = 1 the document is excluded from copy
</Field>
```

Excluding standard and library parts

Library parts are greyed out in the *Copy* dialog by default. The transactions `ExcludeDocumentStructureDefinition`, `ExcludeRelatedDrawingsDefinition` and `ExcludeRelatedPartsDefinition` in `XPlmCopyArasConfiguration.xml` influence this behavior

Innovator fields can be added to these three transactions. If a Innovator document has the same property name and the same property value as described in the transaction, it will be greyed out in the dialog and can't be selected for copying. `ExcludeDocumentStructureDefinition` will be evaluated in the structure view of the dialog, `ExcludeRelatedDrawingsDefinition` will be evaluated in the related drawings view of the dialog and `ExcludeRelatedPartsDefinition` will be evaluated in the related parts view of the dialog.

The field `ExludeChilds` in transaction `Config` defines, if child elements of standard parts are also greyed out. It is set to `true` by default.

The *Copy* dialog is designed so that if you select an item for copying, all parent items must also be copied. This property ensures that this behavior is mapped correctly.

6.2.6 Configuring the BOM dialog

The connector comes with default configurations of the *BOM* dialog to create and update parts and BOMs in Innovator.

The configuration is the same as the *Save* dialog.

6.2.6.1 BOM dialog options

You can customize the behaviour and functionality of the *BOM* dialog.

The dialog can be switched off with option setting `InventorArasBomDialogEnabled` in XPlmInventorArasConnector.xml.

XPlmArasInventorDialogs.xml

- **Configuration file:** `XPlmArasInventorDialogs.xml`
- **Transaction:** `PartsViewConfig`
- **Structure:** `Options`

The following options are available for customization;

Name	Description
GlobalHideFilters/ OccurrenceFilter	The occurrence filter is enabled to hide multiple instances of a part item in the same level. The filter key depends on the type of the item: ■ No CAD document available -> ModelFullName ■ `cax_bom_exclude` _flag is set to true -> CAD ITEM_ID ■ All others -> PART ITEM_ID **Default: true** **Possible values: true | false**
GetAccessCheck	If set to `true`, the integration checks if part is in release state. **Default: true** **Possible values: true | false**
IsCurrentCheck	If set to `true`, the integration checks if a newer generation of the part exists. **Default: true** **Possible values: true | false**
NoCAD	If set to `true`, the integration checks if a CAD document exists in Innovator to the local CAD file. **Default: true** **Possible values: true | false**
BomExclude	If set to `true`, the integration checks if the BOM exclude flag is set. **Default: true** **Possible values: true | false**

Query part numbers

To query a new part number the transaction `QueryPartNumber` in file `XPlmArasInventorTransaction.xml` is used. With the transaction, additional informations of the specific part can be transferred to the server method. The `XplmDocument` for the mapping can be viewed via function **Show selected documents** in the side bar of the dialog. By default, the Innovator server method **xPLM Generate Number** is used.

External positions

External positions are displayed in read-only in the BOM structure of the *BOM* dialog.

Via option setting `InventorArasBomDialogShowExternParts` in file `XPlmInventorArasConnector.xml`, displaying them can be switched off.

The transaction `MapExternPosition` in file `XPlmArasInventorTransaction.xml` is used to map Innovator properties to the corresponding CAD properties via the traverse process. The dialog con only bind

- These elements are read-only à context menu disabled, editing properties disabled
- Via the switch `InventorArasBomDialogShowExternParts` the external positions can be enabled/ disabled.
- Rows of external part items are highlighted in a different style.
- Transaction `MapExternPosition` is used to map Innovator properties to the corresponding CAD properties from the traverse process

- The dialog can only be bound to one property name per column. See following examples:

Innovator property	CAD property
quantity	ModelConfigurationQuantity
name	ModelName
cax_config	ModelActiveConfigurationName

6.2.7 Configuring dialog colors

The XPLM integration now supports customizable color themes for dialogs to enhance visual consistency and brand alignment. By default, the integration colors have been updated to match the color scheme of Aras Innovator ensuring a seamless and unified user experience.

Introduced or updated with product version 4.5.0.

- **Configuration file**: CustomDialogColors.xml (p. 135)

The default color palette of the integration now mirrors Innovator's primary grey color.

You can configure the integration dialog to adapt your organization's custom branding by editing `CustomDialogColors.xml` file. In this XML file, you can define the;

- Primary, secondary and accent colors.
- Component-specific color settings such as headers and borders in the integration dialogs.

Both HTML names and HEX values are supported.

Example:

```
<Transaction>
  <Aliasname>ColorTheme_Gray</Aliasname>
  <Import>
   <Parameter>
    <FieldCollection>
     <Field>
      <Name>XPLM_Main_1</Name>
      <Value>DimGray</Value> <!-- Borders of certain dialog and UI elements -->
     </Field>
     ......
    </FieldCollection>
    <StructureCollection/>
    <TableCollection/>
   </Parameter>
  </Import>
 </Transaction>
```

Related links
CustomDialogColors.xml (p. 135)

6.2.8 Property mapping

Mappings from Inventor to Innovator are triggered during the save process (initial or update), whereas the mappings from Innovator to Inventor takes place by the **Update Properties** or **Update Title Block** functions of the integration.

The property exchange configuration transaction files are read every time a property exchange is executed, making it possible to change property exchanges without needing to restart the integration (unlike other configuration files)

Since XML files are used to define property exchanges, it is important to understand their logical structure. This structure is called *transaction*. Transactions basically consist of three substructures; `<FieldCollection>`, `<StructureCollection>` and `<TableCollection>`) and a name (`<Aliasname>`). The name is used to identify the transaction, while the other substructures contain values that are important to the underlying definition of the property exchange. All together transactions are independent unified data containers.

The real data container is the `<Field>`. It is like a dictionary with name-value-pairs.

Fields are structured in a `FieldCollection`. `FieldCollection` can be seen as a single row of a table. To structure simple data a structure which can hold a single `FieldCollection` can be used. `StrucutreCollection` itself groups multiple structures. The representation for tables is grouped in the `TableCollection`.

So depending on the needs of transaction, the fields are either defined in the `FieldCollection`, the `StructureCollection` or the `TableCollection`, or in all of these XML elements.

6.2.8.1 Custom properties

The custom properties have different appearances and names inside Inventor. Their names are used for the XPLM connector to make the property mapping work.

Custom property exchange

- **Configuration file:** `XPlmArasInventorTransaction.xml`

The above named file (also referred to as **standard file**) contains all configurations concerning property exchange.

Customized property exchange is possible by directly manipulating the standard file or by creating a custom property-exchange file that overwrites the settings of the standard file. It is recommended to derive customized transactions from existing ones since an example configuration of every transaction or transaction template is given in the standard file.

A custom exchange file can be created by creating a new XML file that must be named `Customer_XPlmArasInventorTransaction.xml` (also referred to as **customer file**). The customer file can contain any transaction the standard file contains as well. If the customer file contains a transaction structure, this one overwrites the corresponding structure of the standard file.

Transactions that are not included in the customer file are taken from the standard file. Therefore, a customer exchange file can contain all transaction structures of the standard file, or just a subset that is relevant to the end user. However, when overwriting XML structures using a customer file it is always necessary to overwrite complete transaction structures. Subsets of transactions (for example, a single `FieldCollection`) cannot be overwritten since these XML structures cannot be identified without the corresponding transaction.

It is highly recommended not to change the *standard file* at all. Customization should be performed solely using the *customer file* whenever possible.

Property tabs

Autodesk Inventor manages different *Tabs* of iProperties with different internal structures depending on the language of the Autodesk Inventor installation. For English installations, the following mapping applies.

Structure Name	iProperties Tab
Summary Information	Tab Summary
Document Summary Information	Tab Summary
Design Tracking Properties	Tab Project, Tab Status, Tab Physical
User Defined Properties	Tab Custom

The following code snippet maps the volume parameter from within the physical iProperties to the `plm_volume` attribute within Innovator upon each save of the document.

```
<Transaction>
 <Aliasname>ChangeDocument1</Aliasname>
....
              <Field>
                <Name>plm_volume</Name>
                <Type>XPlmDocument</Type>
                <Subtype>StructureCollection</Subtype>
                <Structurename>Design Tracking Properties</Structurename>
                <Attribute>Volume</Attribute>
              </Field>
...
</Transaction>
```

6.2.8.2 Property exchange during initial save

You can define mapping of Inventor properties to Innovator during the initial save of PLM unknown objects.

- **Configuration file:** `XPlmArasInventorTransaction.xml`
- **Transaction tags:** CreateDocument1_[CAD file extension]
- **Annotation:** One transaction for each Inventor file type.

Each Inventor file type (part, assembly or drawing) is assigned its own transaction. These transactions are differentiated by the appended CAD file extension (in capital letters) for example `CreateDocument1_PRT`

The following example shows the layout of the transaction. The structure is identical for the other CAD objects. It is recommended not to modify other structures within the transaction. This may cause unexpected behavior of the XPLM connector.

```
<Transaction>
 <Aliasname>CreateDocument1_[CADObjectExtension]</Aliasname>
 <Import>
  <Parameter>
   <FieldCollection>
   …
   <StructureCollection>
   …
    <Structure> ← begin of Aras properties-structure
     <Name>Properties</Name> ← name of this structure
     <FieldCollection>
      <Field>
       <Name>name</Name> ← Innovator property
       <Type>XPlmDocument</Type>
       <Subtype>StructureCollection</Subtype> ← CAD system property
       <Structurename>MetaData</Structurename>
       <Attribut>ModelName</Attribut> ← assign value of ModelName
      </Field>
      …
      <Field>
       <Name>arasTestProperty</Name> ← Innovator property
       <Type>XPlmDocument</Type>
       <Subtype>StructureCollection</Subtype>
       <Structurename>[CADCustomPropertySection]</Structurename> ← depends on CAD
       <Attribut>[CADCustomPropertyName]</Attribut> ← assigns value from this source
      </Field>
     </FieldCollection>
    </Structure>    ← end of Properties-structure
…
```

Only available properties of the XPLM item types CAD can be exchanged.

Mapping of drawing sheet count

- **Configuration file:** `XPlmArasInventorTransaction.xml`
- **Transaction:** CreateDocument1_[CAD drawing extension]

To map the drawing sheet count to Innovator, add field `cax_sheet_count` to `Properties` of transaction `CreateDocument1_[CAD drawing extension]`, see code block below.

```
<Transaction>
  <Aliasname>CreateDocument1_[CAD drawing extension]</Aliasname>
  <Import>
    <Parameter>
      <FieldCollection>
        ...
      </Structure>
      <Structure>
        <Name>Properties</Name>
        <FieldCollection>
          ...
          <Field>
            <Name>cax_sheet_count</Name>
            <Type>XPlmDocument</Type>
            <Subtype>StructureCollection</Subtype>
            <Structurename>Properties</Structurename>
            <Attribut>DrawingSheetCount</Attribut>
          </Field>
          ...
        </FieldCollection>
      </Structure>
    </StructureCollection>
    ...
    </Parameter>
  </Import>
</Transaction>
```

6.2.8.3 Property exchange during update save

You can also define mapping of Inventor properties to Innovator during the update save of PLM known objects.

- **Configuration file:** `XPlmArasInventorTransaction.xml`
- **Transaction:** ChangeDocument1
- **Annotation:** One transaction for all Inventor objects

The exchange of properties is not only possible for creating new documents in Innovator. This can also be done by saving a modified document. Since the layout of the corresponding `ChangeDocument1` transaction is the same as the previous one, the field node can be added in the same way. Make sure, that the values of the `Name` and `Attribute` nodes matches the property names in Inventor and Innovator. Also, it is not necessary to modify other nodes in order to keep the exchange working.

6.2.8.4 Property exchange during update properties

You can define mapping of Innovator properties to Inventor.

- **Configuration file:** `XPlmArasInventorTransaction.xml`
- **Transaction:** UpdateProperties1
- **Annotation:** One transaction for all Inventor objects

In this case, properties are loaded into the Inventor from the Innovator metadata with the **Update Properties** function of the XPLM integration. Only changed properties are updated. If no properties changed, the local files are not marked as modified after **Update Properties**.

Since release 4.5.0, Part Data is also available for **Update Properties** by default. MetaData of the Innovator Part (PARTNUMBER and PARTNAME) are set in the same location where the other Properties are set.

The following example shows the layout of the transaction.

```
[---XPlmArasInventorTransaction.xml---]
<Transaction>
 <Aliasname>UpdateProperties1</Aliasname>
 <Import>
  <Parameter>
   <StructureCollection>
    <Structure>
     <Name>[CADCustomPropertySection]</Name> ← name of CADCustomPropertiesSection (API)
     <FieldCollection>
      <Field>
       <Name>TYPE</Name>
       <Type>ParameterList</Type>
       <Subtype>FieldCollection</Subtype>
       <Attribut>ITEM_TYPE</Attribut>
      </Field>
      …
      <Field>
       <Name>[CADCustomPropertyName]</Name> ← name of the CAD custom property
       <Type>ParameterList</Type> ← Do not change
       <Subtype>FieldCollection</Subtype> ← Do not change
       <Attribut><PLMPropertyName></Attribut> ← assign here name of an Aras prop.
      </Field>
     </FieldCollection>
…
```

The `Field` node has to be inserted as a sub node in the `FieldCollection` node. The nodes `Name` and `Attribute` must be modified with the corresponding values from Innovator and Inventor for the exchange to be working. The other nodes remain as they are.

6.2.8.5 Property exchange during update title block

You can define the contents of a title block in Inventor with custom properties. Additionally, you can also define mapping of Innovator properties to the title block in Inventor.

- **Configuration file:** `XPlmArasInventorTransaction.xml`
- **Transaction:** UpdateTitleBoxDocument_All
- **Annotation:** One transaction for all drawing templates

Since release 4.5.0, a single mapping transaction can be used for all drawing formats. You can customize exchange of properties for all drawing formats from Innovator to Inventor in the same transaction. This adds Innovator metadata to the Title Block in Inventor.

The transfer of metadata is invoked by **Update Title Block** integration function. You can define different title block fill rules.

Since release 4.5.0, Part Data is also available for **Update Title Block** by default. MetaData of the Innovator Part (PARTNUMBER and PARTNAME) are set in the same location where the other Properties are set.

Example:

```
<Transaction>
  <Aliasname>UpdateTitleBoxDocument_All</Aliasname>
  <Import>
    <Parameter>
      <StructureCollection>
        <Structure>
          <Name>Inventor User Defined Properties</Name>
          <FieldCollection>
            <Field>
              <Name>ID</Name>                  ← name of CAD custom property
              <Type>XPlmDocument</Type>
              <Subtype>FieldCollection</Subtype>
              <Attribut>ITEM_ID</Attribut>      ← name of a PLM property
            </Field>
            .....
            <!-- Part Properties -->
            <Field>
              <Name>PARTNAME</Name>
              <Type>XPlmDocument</Type>
              <Subtype>StructureCollection</Subtype>
              <Structurename>MetaData_Part</Structurename>
              <Attribut>name</Attribut>
            </Field>
            ....
          </FieldCollection>
        </Structure>
      </StructureCollection>
    </Parameter>
  </Import>
</Transaction>
```

6.2.8.6 Mapping extended properties

You can define mapping of Extended Properties (xProperties) to and from Innovator.

Since release 4.2.0, it is now possible to set attributes on Innovator properties.

- **Configuration file:** `XPlmArasInventorTransaction.xml`
- **Transaction:** `CreateDocument1_[CAD file extension]`

Example transaction:

```
<Transaction>
    <Aliasname>CreateDocument1_[CADObjectExtension]</Aliasname>
    <Import>
        <Parameter>
            <FieldCollection>
                <Field>
                    <Name>ITEM_TYPE</Name>
                    <Value>CAD</Value>
                </Field>
                <Field>
                    <Name>ITEM_ACTION</Name>
                    <Value>add</Value>
                </Field>
            </FieldCollection>
              .......
                <Structure>
                    <Name>Properties</Name>
                    <FieldCollection>
                        ...
     <!-- This example defines an xProperty and sets its value. -->
                        <Field set="value|explicit" explicit="1">
                            <Name>xp-testcad</Name>
                            <Value>newtest2</Value>
                        </Field>
     <!-- This example sets the value for an already defined xProperty. -->
                        <Field set="value">
                            <Name>xp-size</Name>
                            <Value>56</Value>
                        </Field>
                        ...
                    </FieldCollection>
                </Structure>
            </StructureCollection>
            <TableCollection>
                <Table>
                <Name>Relationships</Name>
                 .....
```

This transaction will result in the following AML call which is sent to Innovator. The XPLM-Field Attributes are mapped directly to the Innovator-Property Attributes.

```
<Item type="CAD" action="add">
    ...
    <xp-testcad set="value|explicit" explicit="1">newtest2</xp-testcad>
    <xp-size set="value">56</xp-size>
    ...
</Item>
```

If the xProperty is part of a xClass the xClass must be set within the transaction or before the transaction is executed.

If the xClass is not set, this leads to an error.

An example how to set the xClass within the transaction is the following FieldCollection that must be added to the Relationships Table in the transaction.

```
<FieldCollectionTable>
    <FieldCollection>
        <Field>
            <Name>relation_type</Name>
            <Value>CAD_xClass</Value>
        </Field>
        <Field>
            <Name>relation_action</Name>
            <Value>add</Value>
        </Field>
        <Field>
            <Name>related_item_type</Name>
            <Value>xClass</Value>
        </Field>
        <Field>
            <Name>related_item_action</Name>
            <Value>get</Value>
        </Field>
        <Field>
            <Name>RelatedItem@name</Name>
            <Value>Washer</Value>
        </Field>
    </FieldCollection>
</FieldCollectionTable>
```

6.2.8.7 Dependency classification mapping

The Inventor connector provides additional Reference link type information.

This feature was introduced in version 4.2 of the Inventor connector.

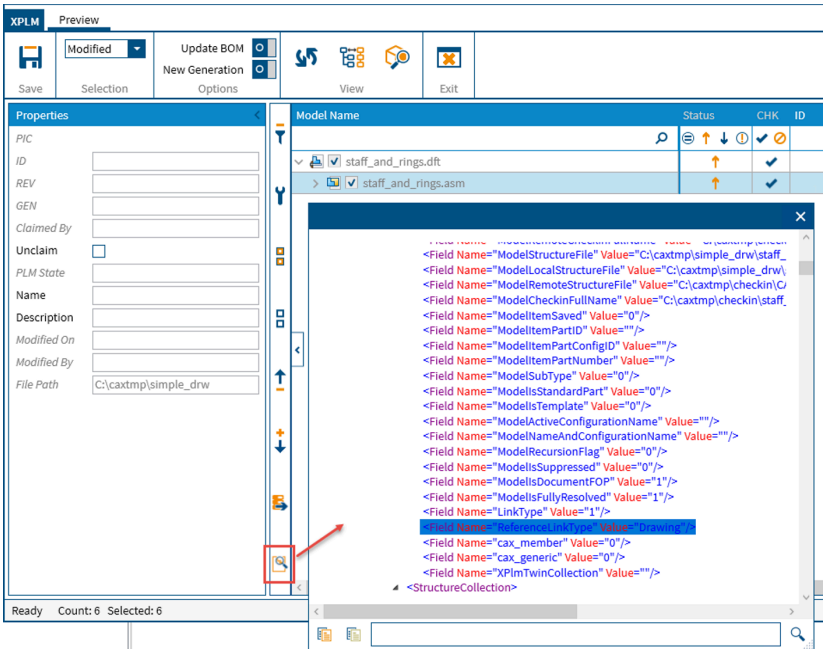
Reference link type refers to the type of CAD models relationship.

Reference link type information is stored in the XPlmDocument attribute `ReferenceLinkType` and is derived from the Inventor connector.

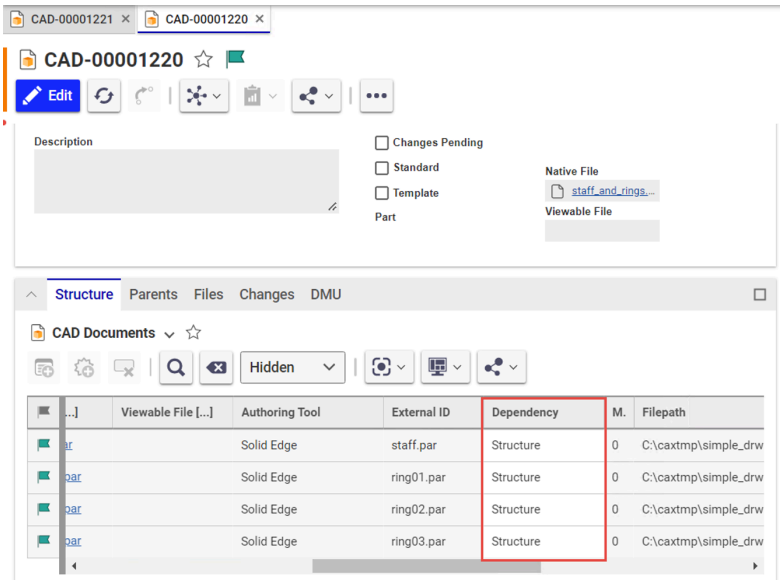
Possible values depending on the component relationship include:

- **Structure** - for native 3D models
- **Reference** - describes external references like inter part copy, split part, mirror part, part families, assembly driven geometry and so on.
- **Drawing** - for each model, which is directly included in a drawing.

The `XPlmDocument` is stored in the *Save dialog* **Side bar** > **Show selected documents**.



This relationship is also shown in Innovator in the CAD **Structure** > **Dependency** field.



Dependency Mapping definition

- **Configuration file:** `XPlmArasInventorTransaction.xml`
- **Transaction:** AddRelation1

The mapping of Reference link type is defined in the standard installation and values are transferred to Innovator during the save process.

The field `Relation@classification` is added to the transaction `AddRelation1`.

```
 <Transaction>
    <Aliasname>AddRelation1</Aliasname>
    <Import>
      <Parameter>
        <TableCollection>
          <Table>
            <Name>Relationships</Name>
            <FieldCollectionTable>
              <FieldCollection>
...
                <Field>
                  <Name>Relation@classification</Name>
                  <Type>XPlmDocument</Type>
                  <Subtype>FieldCollection</Subtype>
                  <Attribut>ReferenceLinkType</Attribut>
                </Field>
...
```

6.2.9 Standard parts

The Inventor connector provides additional functionality to manage Standard Parts.

Standard parts are managed in several ways. During check in, the CAD specific directory is used to identify standard parts. Usually, only few users perform check in on standard parts. Additionally, a marker can be used (property in the file).

Identifying standard parts

- **Configuration file:** `XPlmConnector.xml`
- **Parameter:** `StandardPartDir`, `StandardPartCheckoutDir`

In Innovator, the standard parts are flagged by the value `true` on the `CAD.is_standard` property.

During check out, the property `StandardPartCheckoutDir` is used to identify the directory to which standard parts are locally loaded usually set to `C:\cadlib` by default.

Additionally, there might be flags to determine to rename standard parts, to prohibit check in of standard parts and so on.

Also the path to the checkout folder for standard parts can be configured. Executing **Synchronize Part Library** downloads standard parts from Innovator to defined path `StandardPartDir`. By default, the path is set to `C:\caxtmp\StandardParts\`.

6.2.10 Activating Thumbnails

The connector supports the thumbnail functionality of Innovator.

About this task

This use case describes how to activate Thumbnails on the client side.

Procedure

To enable BITMAP thumbnail generation, configure the following files as described:

a) Open file in a text editor.

b) Save the configuration file.

c) Open the `XPlmArasInventorTransaction.xml` file in a text editor.

d) Enable `Thumbnail_Fullname` mapping field in transaction `CheckInFile1`

e) Save the configuration file.

Result

Thumbnails are activated.

6.2.11 Formatting functions in Innovator

There are a number of functions that can be used to manipulate the values transferred during property exchange.

The following tables gives a list of all available formatting functions.

System functions

Function Call	Description	Example Input	Example Output
`SetLangugages``,en-US,de-DE` [1]	Sets the language for parsing and output of numbers and dates If not set the current system setting is used.	`-`	`-`
`GetSystemLanguage` [1]	Returns the current system language. It is not affected by `SetLanguages`!	`-`	`de_DE`

String functions

Function Call	Description	Example Input	Example Output
`Convert``,True,X`	Replaces input `True` with `X` if it is an exact match.	`True` `true`	`X` `true`
`Insert``,7, not`	Inserts ` not` at position `7` in the input string.	`This is fun!`	`This is not fun!`
`Insert``,10,track`	Inserts `track` at position 10 If the index is out of bounds, the text will be added in front or at the end of the input string	`Short`	`Shorttrack`
`Left``,5`	Retrieves the first 5 characters of the input.	`Trainstation`	`Train`
`PadLeft``,10,0`	Pads the input string on the left with `0` characters until the total length is 10.	`123456`	`0000123456`
`PadRight``,10,0`	Pads the input string on the right with `0` characters until the total length is 10.	`123456`	`1234560000`

[1] Culture Relevant

Function Call	Description	Example Input	Example Output
PadLeftRegexp,^[0-9]*$,10,0	Pads the input string on the left with 0 characters until it reaches a total length of 10, but only if the input consists entirely of digits (matches the expression ^[0-9]*$	123456	0000123456
PadRightRegexp,^[0-9]*$,10,0	Pads the input string on the right with 0 characters until it reaches a total length of 10, but only if the input consists entirely of digits (matches the expression ^[0-9]*$	123456	123456
Postfix,-POST	Adds -POST at the end of the input string.	123456	123456-POST
Prefix,PRE-	Adds PRE- in front of the input string.	123456	PRE-123456
Replace,BB,12	Replaces all occurrences of BB with 12 in the input string (one or multiple characters).	AABBCC	AA12CC
ReplaceRegexp,[aeiou],_	Replaces all occurrences of the regular expression [aeiou] with _ in the input string.	Sentence with vocals!	S_nt_nc_ w_th v_c_ls!
Right,5	Retrieves the last 5 characters from the input string when 5 is smaller or equal to the length of the string.	Houseguest	guest
Substring,2,3	Extracts a portion of the input string starting at index 2 with the length of 3 characters.	ABCDEFG	CDE
ToLower [1]	Returns a copy of the input in lowercase, using the casing rules of the specified culture.	Test 123	test 123
ToUpper [1]	Returns a copy of the input in uppercase, using the casing rules of the specified culture.	Test 123	TEST 123
Translate,ABC,123	Returns a copy of the input string where each character found in the first set is replaced by the corresponding character at the same position in the second set.	0A0A-B9B-CC	0101-292-33
Trim,0	Removes all trailing or leading white space from the input string by default, or removes the specified character 0	000113300	1133

Function Call	Description	Example Input	Example Output
`TrimEnd,0`	Removes all trailing white space from the input string by default, or removes the specified character `0`	`000113300`	`0001133`
`TrimStart,0`	Removes all leading white space from the input string by default, or removes the specified character `0`	`000113300`	`113300`

Number functions

Function Call	Description	Example Input	Example Output
`NumberFormat,E` [1]	Formats the input number according to scientific (exponential) notation.	`133.22`	`1,332200E+002`
`NumberFormat,C` [1]	Formats the input number as currency based on current language settings.	`133.22`	`133,22 €`
`NumberFormat,D6` [1]	Formats the input number as decimal integer padded with leading zeros to ensure a total of 6 digits.	`133.22`	`000133`

Date functions

Function Call	Description	Example Input	Example Output
`DateAddDays,23,F` [1]	Returns the formatted input date after adding the number of days.	`1.12.2012 16:30:00`	`Montag, 24. Dezember 2012 16:30:00`
`DateAddMilliseconds,2000,s` [1]	Returns the formatted input date after adding the number of milliseconds.	`1.12.2012 16:30:00`	`2012-12-01T 16:30:02`
`DateAddSeconds,15,G` [1]	Returns the formatted input date after adding the number of seconds.	`1.12.2012 16:30:00`	`01.12.2012 16:30:15`
`DateAddMinutes,95,s` [1]	Returns the formatted input date after adding the number of minutes.	`1.12.2012 16:30:00`	`2012-12-01T 18:05:00`
`DateAddMonths,12,d` [1]	Returns the formatted input date after adding the number of months.	`1.12.2012 16:30:00`	`01.12.2013`
`DateAddYears,3,d` [1]	Returns the formatted input date after adding the number of years.	`1.12.2012 16:30:00`	`1.2.2015`

Function Call	Description	Example Input	Example Output
DateFormat,s [1] **DateFormat**,\Heu\tei\s\t dddd. [1]	Parses the input string as a date using the current language settings and returns it formatted according to the specified format.	16. April 2012 3.12.2012	16.04.2012 Heute ist Montag.
Now,yyyy-MM-dd [1]	Returns the current date and time in the format defined overwriting the input string.	-	2024-12-03

File name functions

Function Call	Description	Example Input	Example Output
FileChangeExtension,pdf	Interprets the input string as a Windows file name and replaces its current extension with the specified extension.	C:\temp\file.ext	C:\temp\file.pdf
FileCombine,subdir\file.ext	Interprets the input string as a Windows path and creates a new file name or path by adding the specified path.	c:\temp	C:\temp\file.ext
FileGetDirectoryName	Interprets the input string as windows file name and returns only the path information.	D:\long\path\to\file.ext	D:\long\path\to
FileGetExtension	Interprets the input string as windows file name and returns only the file extension (including the dot).	C:\temp\file.ext	.ext
FileGetFileName	Interprets the input string as windows file name and returns only the file name.	C:\temp\file.ext	file.ext
FileGetFileNameWithoutExtension	Interprets the input string as windows file name and returns only the file name.	C:\temp\file.ext	file

Info functions

Function Call	Description	Example Input	Example Output
InfoGetEnvironmentVariable,SystemRoot	Returns the environment variable visible to the current process.	-	C:\windows
InfoHostName	Returns the name of the current host (network name).	-	xplm-kap
InfoMachineName	Returns the name of the current machine (computer name).	-	XPLM-KAP
InfoUserDomainName	Returns the name of the current user as domain name.	-	xplm-kap

Function Call	Description	Example Input	Example Output
`InfoUserName`	Returns the name of the current user.	`-`	`kap`

6.2.11.1 Using functions

Functions are called by using their name and adding parameters separated by commas in a `<Function>` element within a `<Field>` element. Functions can be combined by concatenating them using the pipe `|`. Wrong parameters or functions in combined calls are ignored and they do not change the result.

```
<Field>
    <Name>Test</Name>
    <Value>Input</Value>
    <Function>Replace,e,oa</Function>
</Field>
```

- Single function call: `<Function>Replace,e,oa</Function>`
- Combined function call: `<Function>Replace,e,oa|Prefix,PRE</Function>`
- Escaping characters: `<Function>Translate,\,\|\\,123 </Function>`
- Ignoring errors: `<Function>Prefix,Pre-|False Function|Postfix,-Post</Function>`

6.2.11.2 Culture functions

Some functions need language or culture information to define how a string is interpreted and formatted (for example, dates and numbers).

The technical implementation is based on the .NET *CultureInfo* class and the culture is identified by the name as defined in the *National Language Support (NLS) API* in column *Culture Name*. Typical values are:

- de-DE: German (Germany)
- en-GB: English (Great Britain)
- en-US: English (USA)
- fr-FR: French (France)

If no culture is set the local machine setting is used for input and output. For explicitly setting the culture information proceed any function calls with the `SetLanguages` function:

`<Function>SetLanguages,en-US,de-DE|FormatDate,s</Function>`

Additional information

Refer to .NET documentation for additional information:

- Reference Standard date and time format strings : https://learn.microsoft.com/en-us/dotnet/standard/base-types/standard-date-and-time-format-strings
- Reference Custom date and time format strings : https://learn.microsoft.com/en-us/dotnet/standard/base-types/custom-date-and-time-format-strings
- Reference Standard numeric format strings : https://learn.microsoft.com/en-us/dotnet/standard/base-types/standard-numeric-format-strings
- Reference Custom numeric format strings : https://learn.microsoft.com/en-us/dotnet/standard/base-types/custom-numeric-format-strings
- Regular Expression Language - Quick Reference : https://learn.microsoft.com/en-us/dotnet/standard/base-types/regular-expression-language-quick-reference
- National Language Support (NLS) : https://learn.microsoft.com/en-us/windows/win32/intl/national-language-support

6.2.12 Removing local files

The **Remove local files** functionality is designed to help you clean up the working directory by removing unnecessary local files generated during Inventor operations.

- **Configuration file**: `XPlmArasInventorTransaction.xml`

The integration provides 3 `Remove Local Files` Methods in the Aras Innovator ScriptEngines

- `arasRemoveLocalFiles`
- `arasRemoveLocalFiles2`
- `arasRemoveLocalFiles3`

There are two ways for removing the local files; through ExitEvents or the UI command.

The default method is `arasRemoveLocalFiles3` during the ExitEvent and `arasRemoveLocalFiles2` from the UI

In the `RemoveLocalFiles1` transaction , you can define specific file extensions that should be always deleted, as well as those that should be checked before deletion.

When a file is deleted all other associated files (like CLB files) will also be deleted.

The **Remove local files** features is not intended to only remove files that are already stored in Innovator. It is designed to clean up the local working directory, with exceptions only in rare, defined cases where files should remain in the working directory folder.

Method	Description
`arasRemoveLocalFiles`	Deletes all files in LocalWorkDir, LocalCheckoutDir and LocalCheckinDir without checking
`arasRemoveLocalFiles2`	Deletes all files in LocalCheckoutDir and LocalCheckinDir Deletes files in LocalWorkDir by using the Transaction `RemoveLocalFiles1`. Files open in session and modified files will NOT be deleted.
`arasRemoveLocalFiles3`	Deletes all files in LocalCheckoutDir and LocalCheckinDir Deletes files in LocalWorkDir by using the Transaction `RemoveLocalFiles1`. Files open in session (and not modified) will be deleted. Modified files will NOT be removed.

Modified here refers to modified locally in comparison to Innovator and not just locally modified.

Removal logic

`arasRemoveLocalFiles2` method introduces logic to determine whether files can be safely removed;

- If the CAD file is loaded, do not remove the file.
- If a CLB file exists and model stamps match, remove the local CAD files.
- If files cannot be removed due to access issues or ownership, a message is displayed warning of the issue.

IF a CAD file cannot be deleted, the corresponding CLB file is also not deleted.

The following are examples of use cases when files are not removed;

- Access problems (for example, the user is not the owner).
- Timestamp differences between CAD and CLB files.
- CAD file is modified by mass recalculation (for example, after **Update BOM**)
- Different timestamps due to
  - activation of **Update BOM** in the *Save dialog*.
  - mismatches between model stamp and modification date.
  - local storage of CAD files

7 Update & removal

7.1 Modifying, repairing or removing installation

Complete these steps to modify, repair or remove an existing installation. When modifying, you can add or remove components, while repairing reinstalls a complete installation to resolve file or other conflicts.

About this task

During a modification, no existing files are overwritten and only missing files are added. During a repair, existing files are overwritten and components registered again. This corresponds to a complete installation reset.

Files with the prefix `customer_` remain unaffected.

You can install, modify, repair, or uninstall also in silent-mode. See Silent-mode installation (p. 136) for more information.

Procedure

1. Close all open applications related to the integration.
2. Start the current installer `Setup-*.exe` with administrator rights. If this file is no longer available, start `Setup-*.msi` directly from the directory `%ProgramData%\XPLM Solution GmbH\packages`. Alternatively, you can also start the installer from Windows:
   a) In the system settings, go to *Apps & Features* and search for the entry **XPLM Setup for ...**.
   b) Select the entry and click **Modify**.
   ➔ Visual C++ runtimes are checked/installed again and the installation wizard appears.
3. Click **Next** to start the wizard.
   ➔ The step *Modify, repair or remove installation* appears.
4. Modify installation:
   a) Click **Modify**.
      ➔ The step *Installation path* appears.
   b) In this step, you cannot change the installation path, but applying overlay packages or making backups are possible. Click **Next**.
   c) In each step, select components as required and click **Next** until you reach the step *Ready to install*.
   d) Click **Install**.
      ➔ Installation starts.
   e) After completion, click **Finish**.
   ➔ The installation is modified.
5. Repair installation:
   a) Click **Repair**.
      ➔ The step *Installation path* appears.
   b) In this step, you cannot change the installation path, but applying overlay packages or making backups are possible. Click **Next**.
      ➔ The step *Ready to install* appears.

c) Click **Install**.
➔ Installation starts.

d) After completion, click **Finish**

➔ The installation is repaired.

6. Remove installation:

a) Click **Remove**.
➔ The step *Remove of the installation* appears.

b) Click **Remove**.
➔ Uninstallation starts.

c) After completion, click **Finish**.

➔ The installation is removed.

Result

You know how to modify, repair or remove an installation.

Related links
Working with overlay packages (p. 143)

7.2 Updating installation

Complete these steps to update an existing installation with a new version.

Before you start

It's good practice to always back up the existing installation directory. You can do this manually before the update, or during installation of the new version.

About this task

XPLM strongly recommends using appropriate services for an update. This ensures that existing functionality and modifications are correctly transferred to the new product. Access https://support.xplm.com and file a ticket for assistance.

You can install, modify, repair, or uninstall also in silent-mode. See Silent-mode installation (p. 136) for more information.

Procedure

1. Close all open applications related to the integration.
2. Start the new installer `Setup-*.exe` with administrator rights.
➔ Visual C++ runtimes are checked/installed again and the installation wizard appears.
3. Click **Next** to start the wizard.
➔ The installer detects an existing installation and shows a message.
   - If the existing installation is compatible with the installer, you can proceed in the wizard. When installation begins, outdated components are removed first and current ones installed.
   - If the existing installation is not compatible with the unified installer, it is first removed completely and then installed from scratch.

   Files with the prefix `customer_` remain unaffected.

4. Click **Next**.
   ➔ The step *License agreement for end-users* appears.
5. Accept the license agreement and click **Next**.
   ➔ The step *Installation path* appears.
6. If an existing and compatible installation was found, you cannot change the installation path in this step, but applying overlay packages or making backups are possible. Click **Next**.
7. In each step, select components as required and click **Next** until you reach the step *Ready to install*.
8. Click **Install**.
   ➔ Installation starts.
9. After completion, click **Finish**.

Result

The installation is updated. Carefully compare and merge the changes from the backup with the new files. Start the product and verify everything works as expected.

Related links
Working with overlay packages (p. 143)

8 Troubleshooting

8.1 Troubleshooting integration problems

About this task

In case of integration problems, proceed as follows:

Procedure

1. Close all related programs.
2. Activate logging in corresponding `*Connector.xml` files.
3. Restart the integration and reproduce the problem.
4. Access https://support.xplm.com and file a ticket with the logs attached. If you don't have an account yet, write to support@xplm.com.

8.2 License errors

If you encounter license issues that affect the integration, see the configured log file for more information. The following table will help you to solve license issues.

Reason	Description	Solution
No license file found	No license file was found in the directory `%xPlmRootDir%\xml` or the alternative directory defined in environment variable `XPlmLicenseDirectory`.	Put license file in this directory. If no license file is available, get a valid license file from XPLM.
Cannot read file	There is not enough memory to read the license file.	Upgrade RAM on the computer where the integration is installed.
Unable to read license	The license file cannot be read/ decrypted.	Get a valid license file from XPLM. Do not edit a license file.
Unable to retrieve license data	The content of the decrypted data does not correspond to an XPLM license content.	Get a valid license file from XPLM.
No domain data found	No domain entry was found in the license file.	Get a valid license file from XPLM with all applicable domain names. Domain names are case sensitive.
License is not valid for current domain <DOMAIN_NAME>	The customer domain does not exist in the license file.	
No MAC address found	No MAC address entry was found in the license file.	Get a valid license file from XPLM with all applicable MAC addresses. MAC addresses are case sensitive.
This system is not allowed	The customer MAC address does not exist in the license file.	
There is no license for <LICENSE_NAME>	The requested license is not part of the customer's license subscription.	Get a valid license file from XPLM with all integration components that are subscribed. License names are case sensitive.

Reason	Description	Solution
Unable to retrieve license information for <LICENSE_NAME>	There is no information about when the license expires.	Get a valid license file from XPLM.
License for <LICENSE_NAME> is expired on <YEAR>-<MONTH>-<DAY>	License is expired and no longer valid.	Resubscribe and get a valid license file from XPLM.

8.3 Solving issues in mixed environments

If you plan to use ECAD and MCAD integrations together on the same client computer, several installation and licensing issues may occur.

Naming conventions for integration versions:

- Before ECAD product version 1.0.0, the separate FlexLM licensing mechanism was used together with the license file `license.dat`. We call such ECAD integrations the **first generation**.
- Starting with ECAD product version 1.0.0, the separate local license mechanism, called License Client, is used together with the license file `*.lic`. We call such ECAD integrations the **second generation**.
- Starting with ECAD product version 1.2.0, the official MCAD installer is used, which already contains and installs License Client. We call such integrations the **third generation**.
- Starting from 2022, a unified installer based on the third generation is gradually being introduced for all XPLM products. We call such integrations the **fourth generation**. Integrations installed with this installer can run fully in parallel with other products on the same client computer, eliminate redundancies in certain core components, and provide improved uninstall and upgrade behavior.

  The integration described here corresponds to the fourth generation.

Different behavior in the licensing mechanism for ECAD and MCAD integrations:

- ECAD integrations use the Innovator license mechanism by default and check for valid feature licenses. If you use Innovator without an Aras subscription but as part of enterprise-free software, you must disable the Innovator license mechanism and use the XPLM license mechanism.
- By default, MCAD integrations always use the XPLM license mechanism first and check the license file for valid connector entries. Then, the feature licenses in Innovator are additionally checked. Thus, depending on the license model and mechanism used, you do not need to configure anything additional for MCAD integrations.

Issue	Solution
How can I identify an installed integration version?	■ First generation ECAD: □ In the installation directory of the integration, the license file `license.dat` exists. ■ Second generation ECAD: □ The environment variable `xPlmRootDir` exists and points to the installation directory of License Client where the license file `*.lic` is located. Typical locations for the license file in second-generation products are `%xPlmRootDir%\LicenseClient\xml` or `%xPlmRootDir%\xcl\xml`. □ The integration itself is not installed under the `xPlmRootDir` path but in another directory. □ In the installation directory of the integration, the file `uninst.exe` exists. ■ Third generation ECAD/MCAD: □ The integration is installed under the `xPlmRootDir` path. □ In the installation directory `xPlmRootDir`, the shortcut `Uninstall_*` exists. □ MCAD and certain ECAD installers are named `xxx_<version>_x64.msi`. □ ECAD installers in general are named `Integrate2_<PLM>_<ECAD>_<version & build>.exe` ■ Fourth generation ECAD/MCAD: □ The integration is installed under the `xPlmRootDir` path. □ In the installation directory `xPlmRootDir`, there is no shortcut `Uninstall_*`. □ MCAD installers are named `<PLM>_<MCAD>_<version>.7z.exe`. □ ECAD installers are named `<PLM>_<ECAD>_<version>.7z.exe`. □ The directory `%ProgramData%\XPLM Solution GmbH` exists. □ The registry entry *HKLM\SOFTWARE\XPLM Solution GmbH \{00000000-0000-0000-0000-000000000000}* exists.

8.4 Resolving common errors

Following table shows a list of errors that may occur and possible solutions to resolve these errors.

Error	Possible Reason(s) and Solution(s)
Installation errors	
IOM-DLL registration problems	**Reason:** If the installation runs successfully but the IOM-COM object could not be found, check if the runtime is on a local drive. **Solution:** When using a network drive for the runtime, configure the .NET-Security for running IOM.DLL from a network drive because it is not allowed by default.
ArasBrowserAuthentication connection errors	

Error	Possible Reason(s) and Solution(s)
Error *An error occurred while sending the request* is shown after clicking connect.	**Reason:** It is likely that TLS is only executed in Version 1.0 on the Machine. TLS 1.0 is deprecated. **Solution:** Via a known Workaround by Microsoft, TLS can be forced to version 1.2. SystemDefaultTlsVersions=1 has to be set at 2 locations ■ Computer\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft \.NETFramework\v4.0.30319 ■ Computer\HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node \Microsoft\.NETFramework\v4.0.30319 For more information visit Microsoft support
Error *Unknown error ....* OR *unknown error in module STDMETHODIMP CArasBAPI::connect*	**Reason:** Integration before version 4.0.9 is used with the install option Aras 19+ or newer + Property ArasBrowserAuthentication=true **Solution:** BrowserLogin (Property `ArasBrowserAuthentication`) is fixed for Aras 19,20,21,22 (Aras 19+). Now it uses Microsoft Edge instead of Internet Explorer . UI is almost the same Requirement on Client Machine: WebView2 Runtime **Workaround for previous Releases with Aras 19+** Copy the following files from an Aras19+ CD Image or from a recent XPLM Installer to the Aras** folder in the Solution Dir (example: C:\Program Files\XPLM Solution GmbH\bin\x64\Aras19) ■ Microsoft.Web.WebView2.Core.dll ■ Microsoft.Web.WebView2.WinForms.dll ■ Microsoft.Web.WebView2.Wpf.dll ■ WebView2Loader.dll (arm64 / x64 /x86)
If no error occurs, but an invisible windows is blocking the CAD in the back.	**Solution:** Make sure that WebView2 Runtime is installed. Make sure that the WebView2Load DLL + 3 Microsoft.Web.WebView2 DLLs are placed next to the IOM DLL as described above.

Error	Possible Reason(s) and Solution(s)
Server error: Command Bar Section (CommandBarSection) with Name 'CAD Exchange_TOC_Content', Location 'TOC' already exists.	**Reason:** since Aras Innovator 14, the CAD Exchange Table was no longer Part of the Administration section of the Aras Innovator Client. This Table is used for Load To CAD, Load History and Copy. Sometimes an admin may want to wipe the table to remove old entries. That was no longer possible as the Table existed, but was not visible in the UI. The import package was enhanced with several elements. If the enhanced CAD Exchange Package is imported to Aras Innovator 12, then Error messages appear because elements with the same name but different IDs already exist. **Solution:** To fix the CAD Exchange Package for Aras Innovator 12: ■ Make sure the error happens in the CAD Exchange Package. ■ Delete the following folders in CAD_Exchange Package: CommandBarItem, CommandBarSection, PresentationConfiguration ■ In `....\CAD_Exchange\Import\ItemType\CAD Exchange.xml` Comment out the `ITPresentationConfiguration Item` (line 119-124) ■ In `....\CAD_Exchange\Import\ItemType\CAD Exchange.xml` Comment out the `TOC Access Item` (line 113-118) If the enhanced CAD Exchange Package should be imported to Aras Innovator 12 SP18 or lower, then: ■ adjust `imports.mf` to use the `Import_V12` instead of `Import` ■ select the `Import_V12` in the Aras Innovator Import Tool
User interface errors	
Resolution and windows size changes	**Description:** While working with the integration it may happen that the Windows resolution changes or commands of the ribbon bar are no longer executable. **Solution:** **1.** Perform a right-click on the Windows Desktop and select **Display Settings** from the context menu. **2.** If not activated, select **Display** in the left-pane menu. **3.** Move to the right pane and click the **Advanced scaling settings** link under the *Scale and Layout* section. **4.** Now either **A.** toggle the switch under the *Let Windows try to fix apps so they're not blurry* to **Off** or **B.** change the size of text, apps and other items to **125** or. **C.** set *Scale and layout* to **100%**
Changes of the integration menu do not become effective	For changes of the integration menu it might be necessary to reset the ribbon (**Right-click the ribbon** > **Ribbon Appearance** > **Reset Ribbon**).
Functionality errors	

Error	Possible Reason(s) and Solution(s)
Error revising a CAD document from PLM	**Reason:** When versioning or revising a document from the Innovator interface (not from the Inventor integration), all children are removed in the Inventor structure. Revising a Inventor document from Innovator could be done e.g. within an ECO or DCO. Removing the Inventor structure destroys the CAD information. **Solution:** In the OOTB Innovator 9.3 installation, a method `PE_DeleteCADStructure` is called in the `onAfterVersion` trigger. This method should be deactivated in the trigger `onAfterVersion`.
CaxPackage increases CAD number on every update document	**Reason:** This Issue only occurs when a CaxPackage before 4.0.10 is used on an Innovator server and this CaxPackage is updated with a CaxPackage with version 4.0.10 or later (till 4.2.0). **Solution:** When using the ArasImportTool, Type - Merge must be used!
Error when creating Part and BOM with disabled or delegated BOM views.	**Reason:** This issue occurs when the assembly contains suppressed components, a delegated BOM or disabled/missing BOM views that prevent the integration from traversing the BOM structure correctly. In this cases, the integration cannot generate a valid BOM during the operation and an exception message is thrown. **Solution:** ■ Check the assembly for suppressed components and unsuppress components that should be part of the BOM creation. ■ Verify that a valid BOM view is available in Inventor. ■ Review delegated BOM settings and make sure the parent assembly is configured correctly and delegation is intentional for assemblies with delegated BOM. since release 4.7.0, the system no longer throws a low-level exception. Instead, a clear and actionable message explaining why the BOM cannot be created is displayed, allowing quick correction of the assembly setup.

9 Tools and add-ons

This chapter describes working with the tools and add-ons that can either be installed as an option or are installed by default.

9.1 Batch Queue Admin

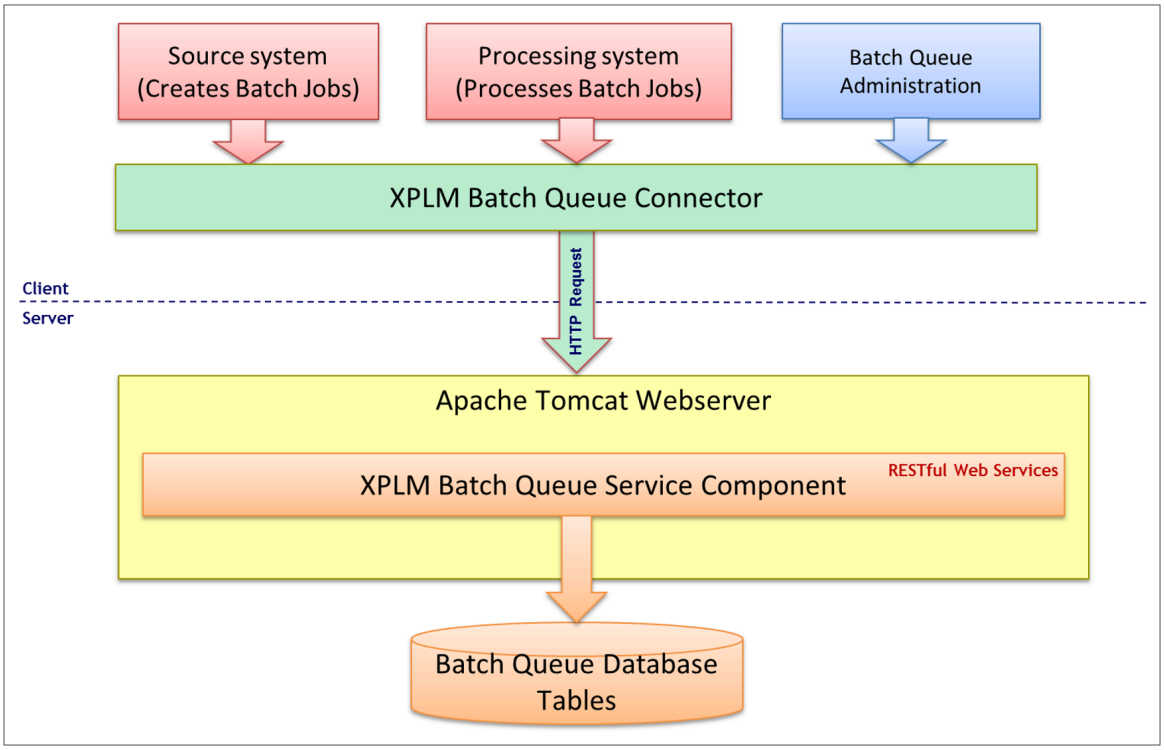
Please consult XPLM services for setup and configuration of this component.

9.1.1 Batch Queue Solution

The XPLM Batch Queue Solution is an entire generic solution for processing batch operations.

The XPLM Batch Queue Solution is an independent flexible extendable solution for described process. It will provide the tools for running batching process which can be freely defined. This means it is not restricted to only run viewable generation inside a Batch Client. It is also possible to define and implement Batch Clients for CAD independent customers' requirements.

Architecture



Components

- **XPLM Batch Queue Service Component:** This is the base component. It is a set of RESTful Web Services which are hosted in a webServer and hold data in a relational database (Batch Queue Database Tables).
- **XPLM Batch Queue Connector:** This is a library which provides functions for data exchange between XPLM Generic Batch Queue and Batch Queue consuming applications.
- **XPLM Batch Queue Administration:** Provides a means to manage batch jobs.
- **Source system:** This is the trigger for working with the Batch Queue. For example, it can be a PLM System which adds a job in the queue via the XPLM Batch Queue Connector during a release process. This process is the PUT-process of the queue and executed asynchronously by the GET-process of the queue.
- **Processing system:** This is the system responsible for pooling and handling the job. This process is the GET-Process of the queue and is executed asynchronously to the PUT-Process. There are two kinds of processing systems available.
  - **Script-Engine based processing systems** - These products will take use of the XPLM Batch Server as pooling service to a Batch Queue. In case of the XPLM Generic Batch Queue Solution the XPLM Batch Server uses the implementation of the XPLM Generic Batch Queue Handler.

- **Product integrated processing systems** -These products will own their pooling service and worker service in one solution. For example, this is usually the case for non COM based CAD Systems.

9.1.2 Batch Queue Admin

The Batch Queue Administration tool provides a means to manage batch jobs via a dialog.

The Batch Queue Administration tool allows:

- Viewing jobs and history entries
- Retrieving jobs and history entries according to certain search criteria
- Changing the status of jobs
- Editing configured fields for jobs and saving this data to the Batch Queue
- Deleting jobs and history entries

XPLM Batch Queue Administration dialog has two main tabs:

- Batch Queue tab - shows search criteria and hit lists for jobs stored in the queue table
- History tab - Using the standard Batch Queue service configuration, all modifying operations on Batch Queue jobs (INSERT, UPDATE, and DELETE) cause a new entry in the history table.
  - During INSERT and UPDATE the system will store a queue table entry. After the operations the system will create history entries.
  - During the operation DELETE the system will save the queue table entry as a history entry before deletion.

It is possible to follow up any change for a job entry in the history table. This leads to much more entries in the history table than in the queue table and makes a regular deletion of history entries necessary.

Functional overview

The menu bar contains the following functions

Function	Description
Search 	Pressing the button **Search** will start a searching by using the currently set search criteria. If the system found jobs or history entries corresponding to the entered search criteria, the search criteria table will be collapsed and the hit list will be shown You can define the search criteria by entering values based on the properties given in the column headers. The system will then display all jobs or history entries which apply to the entered values. It is possible to use several search conditions to find jobs. The following wild-card characters are supported within the search fields: ■ **%** - used to specify any number of arbitrary characters for example `Source Obj. Type = D%` ■ **_** - used to specify a single arbitrary character for example source `System = Ag_le` ■ **|** - used to specify several values for one field for example `Job Type = STEP|PDF`

Function	Description
Show history 	Shows the history of selected jobs. The system will automatically enter the JOB_IDs as search criteria in the history tab, start the history search and switch to the view of the history tab.
Save all changes 	Saves all changes made to the job definitions. Some columns are editable. You may modify the values in these columns to change the job definition depending on the job status. To edit a value, double click in the cell and enter the new value. In the standard configuration, jobs can be edited if the status is not **2. Working**. The following columns are editable: ■ Job Type ■ Job Priority ■ Proc. site ■ Target Obj. Type ■ Target Obj. Number ■ Target Obj. Version ■ Job Parameters  Press **Save all changes** after all necessary are made.
Restart job	Resets the status of the selected job(s) to `0 - Pending` if the current job status allows the transition. In the standard configuration, jobs can be restarted if they are in status `1 - Success`, `3 - Error` and `4 - Deferred`.
Defer job	Resets the status of the selected job(s) to `4 - Deferred` if the current job status allows the transition. In the standard configuration, jobs can be restarted if they are in status `0 - Pending`, `2 - Working` and `3 - Error`.
Delete job 	Deletes selected job(s) with a confirmation dialog if the current job status allows it. In the standard configuration, jobs can be deleted if they are in status `0 - Pending`, `1 - Success`, `3 - Error` and `4 - Deferred`. After deletion the deleted jobs can be found in the history table if writing history entries is not switched off for the action **Delete** in the Batch Queue service configuration.
Purge	Deletes selected job(s) or all loaded jobs if no job is selected with the status `1 - Success`. After deletion the deleted jobs can be found in the history table if writing history entries is not switched off for the action **Delete** in the Batch Queue service configuration.

Function	Description
Stop processing	Shows a list of configured processing systems for which the job processing can be stopped. For example: Selecting the menu entry for one processing system (for example SolidWorks) the XPLM Batch Queue Administration Tool will create a new job with type EXIT for this system. Whenever the program processing jobs for SolidWorks are executed, this specific job with the type EXIT will stop the program. In the example, the program must be restarted to start processing of SolidWorks jobs again.
Show/Hide search criteria 	This function expands or collapses the search criteria view. The search criteria row will either be hidden or displayed.
Delete History Entries for Job	Deletes the history entries for selected job(s) with a confirmation dialog.
Clear search criteria 	Removes entered values in the search fields.
Max.Hits	Shows the search limit. It is configured to **500** by default but you can change the value to any positive integer or zero (0). When the value is set to 0 or is empty, all hits that match the search criteria will be found and may take a very long time for the table entries to load and be displayed.

The functions are also available in the context menu (except for search functions).

Job status

A job can have one of the following statuses

1. Pending
2. Working
3. Success
4. Error
5. Deferred

All changing functions described above in Toolbar section can be enabled or disabled in the configuration depending on the job status. Status transitions to *Pending* (Function **Restart job**) and *Deferred* (Function **Defer job**) can also be enabled or disabled in the configuration depending on the current job status.

Job status standard configuration

Status	Allow Edit	Allow delete	Allow Purge
Pending	Yes	Yes	No
Success	Yes	Yes	Yes
Working	No	No	No
Error	Yes	Yes	No
Deferred	Yes	Yes	No

Job Status transitions

Only status *Pending* and *Deferred* are allowed as destination status (Status to).

Status change is performed using functions **Restart Job** (set status to *Pending*) and **Defer job** (set status to *Deferred*

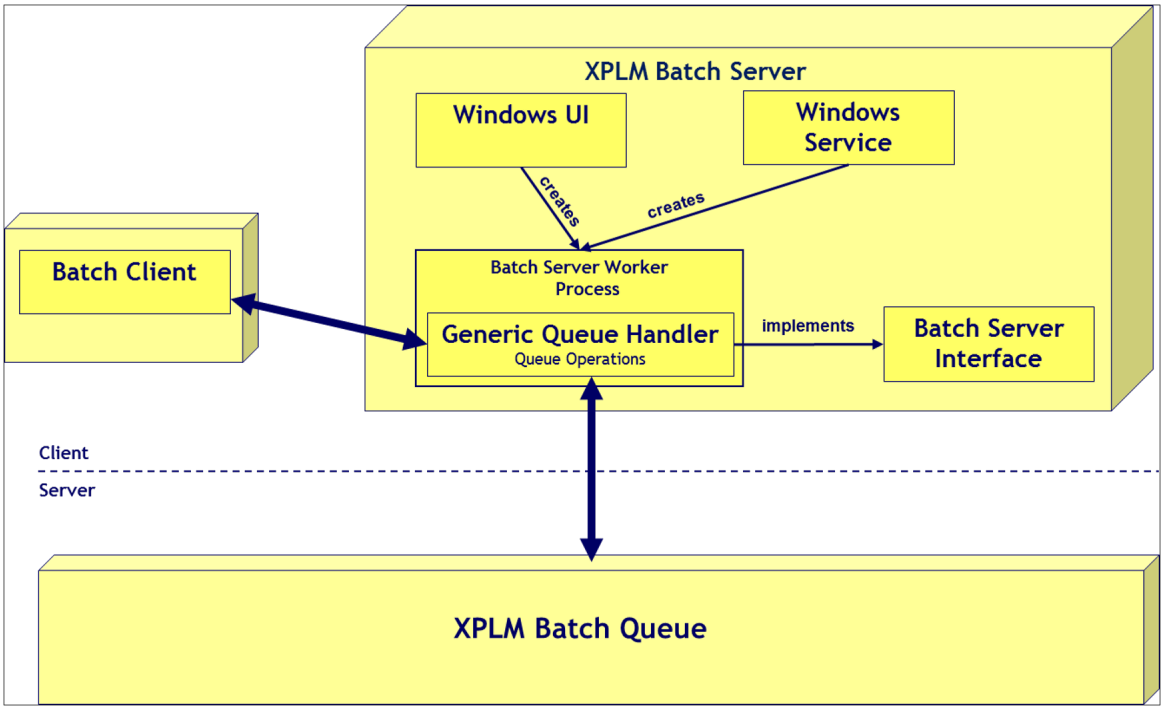
Status From	Status To
Pending	Deferred
Success	Deferred
Working	Deferred
Error	Pending
Deferred	Pending

9.2 Batch Server

The XPLM Batch Server is used in the XPLM Batch Queue Solution as a pooling tool for getting batch job requests from a Batch Queue and passes it over to a Batch Client tool for processing the batch job requests and transfer feedback back to the Queue.

Please consult XPLM services for setup and configuration of this component.

Architecture



XPLM Batch Server acts as communicator between the Batch Client and the Queue as a pooling tool (as kind of watch dog).

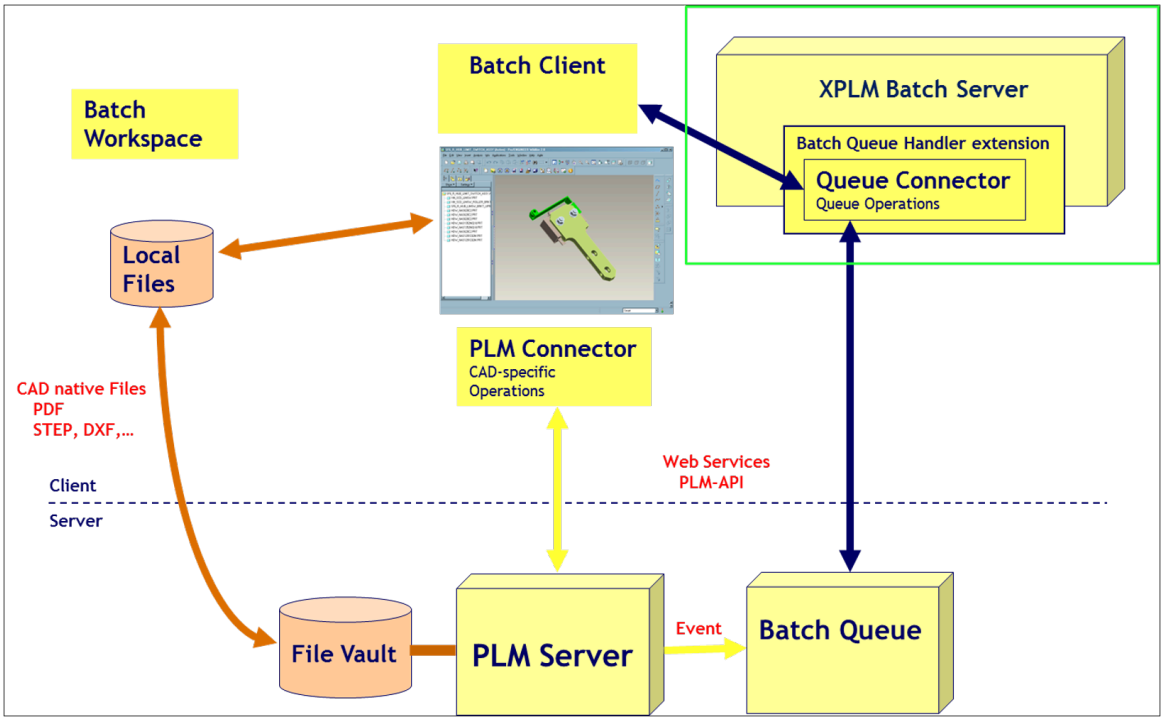
It consists of three components: a UI-Component, a common interface and tool library and a Batch Server Worker Component with the implementation of a Batch Queue Handler.

Batch Process

The Batch server process is based on an asynchronously execution.

The Source System, for example, a PLM-System does not notify client machines directly. It adds a batch job into the Batch Queue, for example, during a release process. Requests are asynchronously processed on a special client machine that is polling a job queue via a queue connector with configurable search criteria.

If a job is found, the XPLM Batch Server retrieves all relevant information from the Batch Queue and passes the job to Batch Client.



In Batch Mode the load is performed before that Task (for example PDF) is dispatched.

9.3 Conversion Server

9.3.1 Introduction

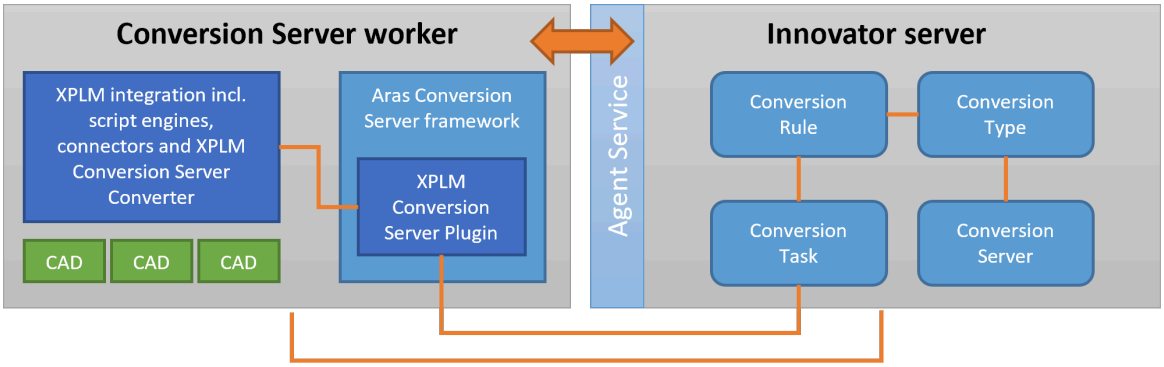
The XPLM Conversion Server Plugin for Innovator is a framework for file conversion processes by converting native CAD files into viewable formats. The plugin extends the default Conversion Server framework from Aras Innovator to work with the integration and enable accurate output, for example PDFs with updated title blocks.

Terms and definitions

Term	Definition
Innovator server	Defines a machine running Innovator.
Innovator database	Defines a database within Innovator.
Conversion Rule	Defines the rules and events in Innovator that define the conversion.
Conversion Type	Defines the name of the converter in Innovator that represents the converter DLL.
Conversion Task	Defines a Conversion Task in Innovator that represents one file being processed by a Conversion Server.
Conversion Server	Defines name, location (URL) and Conversion Types of a Conversion Server in Innovator.
Agent Service	Defines a a Windows service on the Innovator server, designed to handle other add-on services such as Conversion Server and Vault Replication in a clustered environment. It works by actively listening to http/https requests from Innovator on a dedicated port.
Conversion Server worker	Defines a machine running Aras Innovator Conversion Server framework to convert specific file types into other file formats.
Aras Conversion Server framework	Defines a conversion server installation on the Conversion Server worker.
XPLM Conversion Server Plugin	Defines the plugin extension from XPLM installed within the Aras Innovator Conversion Server framework.
XPLM integration	Defines the integration from XPLM including script engines, connectors and XPLM Conversion Server Converter.
XPLM Conversion Server Converter	Defines the helper-application `XPlmConversionServerConverter.exe` to control script engines and CAD applications.
CAD application	Defines a CAD application installed on the Conversion Server worker used to create accurate outputs.

How it works

The Aras Innovator Conversion Server framework can be installed on the Innovator server. In a productive environment, often a separate machine (a "worker") is used to limit the load on the Innovator server. This setup is also used in this description.



Actions on the Innovator server:

1. A file is checked-in into the Innovator vault.
2. This triggers an event in Innovator that checks if the file type is bound to a Conversion Rule.
3. The Conversion Rule contains the Conversion Type, which is referenced by the definition of the Conversion Server. Innovator now knows where the Aras Conversion Server framework is located that executes this Conversion Type.
4. A Conversion Task is created and processed as described in the Conversion Rule.
5. The Agent Service selects and prepares the Conversion Server based on the priorities defined in the vault.
6. A pre-process identifies the object to which the file belongs.

Actions on the Conversion Server worker:

1. The Aras Conversion Server framework receives the file as requested by the Agent Service and executes the Conversion Type.
2. The Aras Conversion Server framework defines the plugin to load and instantiates the converter.
3. The plugin starts the conversion and defines the script engine. The results are passed to the helper-application Conversion Server Converter. This application needs to run in the background to start the script engine and the CAD application required for the conversion. The plugin itself cannot start them.
4. The script engine starts the CAD application, updates title blocks and creates the output file, mostly a PDF.
5. The resulting file is passed to the plugin and the plugin passes it to Innovator.

Actions on the Innovator server:

1. Innovator checks-in the file into the vault.
2. Via the Agent Service, Innovator knows that the conversion is complete and starts a post-process to link the file with the object.
3. The conversion task is closed.

9.3.2 Setup overview

Step 1: Set up Conversion Server worker

Step	Task	Reference
1	Check prerequisites and organize installation media.	Prerequisites (p. 101)
2	Install and license Inventor.	See vendor documentation.
3	Install Conversion Server framework.	Installing Conversion Server framework (p. 102)
4	Install integration for Inventor.	Installing integration components (p. 104)
5	Install Conversion Server Plugin and import specific AML definitions.	Installing Conversion Server Plugin and importing AML definitions (p. 106)
6	Register the Conversion Server Plugin.	Registering plugin in Aras Conversion Server framework (p. 107)
7	Set up the integration components.	Setting up integration components (p. 110)
8	Set up the Conversion Server Converter.	Setting up XPLM Conversion Server Converter (p. 113)

Step 2: Set up Innovator

Step	Task	Reference
1	Check Agent Service on Innovator server.	Setting up Agent Service (p. 114)
2	Set up URL to Conversion Server worker in Innovator web client.	**1.** Get the IP address of the Conversion Server worker. **2.** Log in to the web client as administrator. **3.** Go to **Administration** > **File Handling** > **Conversion Servers**. **4.** Edit the default entry. **5.** Under **Url**, define the correct URL to the Conversion Server worker, for example *http://<IP address>/ConversionServer/ConversionService.asmx* **6.** Click **Done**.

9.3.3 Prerequisites

Requirements for Conversion Server worker

Components	Notes
Windows OS and hardware	For OS and hardware requirements, see the *Platform Specifications for Conversion Server* available in the Aras installation media. To later reference this machine correctly from Innovator, use a static IP address and not a dynamic one.
Aras installation media	The Aras Conversion Server framework is part of the main Aras installer. For supported versions, see System requirements (p. 14). In general, must match version as on the Innovator server in use.
XPLM integration installer `Aras-MCAD-*.7z.exe` XPLM Conversion Server Plugin `Aras-ConversionServer_*.7z.exe`	For download location, see Required installation media (p. 14).
Inventor installer	For supported versions, see System requirements (p. 14). Make sure, licenses are available.
Microsoft Visual C++ Redistributables (x64)	For required versions, see the *Platform Specifications for Conversion Server* available in the Aras installation media.
Microsoft .NET Hosting Bundle	
Microsoft .NET Framework 4.7.2	–

Requirements for Innovator server

- An administrator account for the web client is available.
- Access to the server OS and file system is given.
- The Agent Service is installed and configured. See the *Installation Guide* available in the Aras installation media for more information.
- Required feature licenses are installed.

Requirements for client machine running Inventor

Make sure the integration is installed and configured as described in this document.

9.3.4 Installation

9.3.4.1 Installing Conversion Server framework

The installation of the Conversion Server framework is described in *Conversion Server Setup Guide* available in the Aras installation media. To simplify the installation, the main steps are summarized. However, for troubleshooting it's best to have the Aras guide also available for consultation.

Procedure

1. Under Windows features, enable **Web server (IIS)** and related components.
2. Install Microsoft Visual C++ Redistributables, .NET Framework and .NET Hosting Bundle.
3. Restart worker machine.
4. From the Aras installation media, execute the main installer `InnovatorSetup.msi` and start the wizard.
   a) In step *Setup Type* select **Custom**.
   b) In step *Custom Setup*, enable **Conversion Server** and disable others.
   c) In step *OAuth Components*, select **Configure OAuth components later manually**.
   d) In step *Conversion Server*, change **Conversion Server name** or leave default and enter under **Application Server ULR** the correct URL to the Innovator server in use.
   e) Finish installation.
5. Restart worker machine.
6. Set up authentication and temp directory:
   a) On the Innovator server, go to the Innovator installation directory.
   b) In the directory `OAuthServer`, open the file `OAuth.config`.
   c) Search for `OAuthServer.pfx` and note the password used.
   d) In the same directory, go to `App_Data\Certificates`.
   e) Copy the files `ConversionServer.pfx` and `OAuthServer.cer` to the Conversion Server worker.
   f) On the Conversion Server worker, go to the Innovator installation directory.
   g) In the directory `ConversionServer`, edit the file `OAuth.config`.
   h) Enter the same password as used on the Innovator server in line `<certificate filePath="App_Data/Certificates/ConversionServer.pfx" password="password goes here"></certificate>` and save the file.
   i) Copy the files `ConversionServer.pfx` and `OAuthServer.cer` to directory `ConversionServer\App_Data\Certificates`.
   j) Finally, in directory `ConversionServer`, create the directory `temp` and assign the role *Everyone* with full read/write access.
7. Test basic setup:
   a) Start the IIS Manager console and check that the website **Sites** > **Default Web Site** > **ConversionServer** is available. If you have changed the name in the installer, the correct name must be shown.
   b) Stop and start the IIS root node to properly load all websites.
   c) Enter the URL *http://localhost/ConversionServer/ConversionService.asmx* in a browser window.
      ➔ An XML schema declaration appears.

Result

Installation of Conversion Server framework is complete.

9.3.4.2 Installing integration components

Install the required components.

Before you start

The Aras Conversion Server framework is set up and running.

Procedure

1. Copy the installer archive `Aras-MCAD-*.7z.exe` to the Conversion Server worker.
2. Close all open applications related to the integration.
3. Extract the archive and start `Setup-*.exe` with administrator rights.
4. Install any Visual C++ runtimes that you are prompted for.
   ➔ The runtimes are installed, and the installation wizard appears.
5. Click **Next** to start the wizard.
   ➔ The step *License agreement for end-users* appears.
6. Accept the license agreement and click **Next**.
   ➔ The step *Installation path* appears.
7. Enter the installation directory and click **Next**.
   - The default installation directory `C:\Program Files\XPLM Solution GmbH` is protected by Windows and only administrators can modify files in it.

     To avoid later file access problems, it's good practice to always select a writable location not protected by Windows.
   - If another product is already installed, you cannot change the installation directory. The product installed first defines this path for other products.
   - If you have an overlay package you want to apply, enable the option **Apply custom files after installation** and enter the path to the overlay. Overlays contain customized files that overwrite the installed files as the last step. See Working with overlay packages (p. 143) for more information.
   - To backup the existing installation, enable the option **Backup current installation**. Define the backup scope first. Then click **Browse** and select a backup directory.

     Always create a backup when you update or change an installation. This makes it easier to compare and merge the files later.

   ➔ The step *MCAD components* appears.
8. Select the application(s) to be integrated. If required, select additionally a version or other settings. Click **Next**.
   ➔ The step *Tool components* appears.
9. Select the option **Conversion Server** and click **Next**.
10. To start installation, click **Install**.

    In installer versions 24.3.3.606+, a progress dialog is shown with detailed information on each MSI currently installed. If there are installation issues, the process stops and the error is shown in red in the lower dialog section. Click **Save to log** and send the log file to support@xplm.com for further investigation.
    For older installers, a common CMD window with the progress is shown. Do not click into this window or installation cannot proceed. To continue if you clicked in, press **Enter** or **Esc**.
11. To close the wizard after installation, click **Finish**.

Result

Installation is complete and the environment variable `xPlmRootDir` points to the installation directory. You can find log files for all installed components in the directory `%ProgramData%\XPLM Solution GmbH\log`.

9.3.4.3 Installing Conversion Server Plugin and importing AML definitions

Install the required plugin components and import AML definitions.

Before you start

The Aras Conversion Server framework is set up and running.

Procedure

1. Install plugin files over the Aras Conversion Server framework:
   a) Stop the IIS root node.
   b) Extract the package `Aras-ConversionServer_*.7z.exe`.
      ➔ In the extracted directory, two directories for older and newer Innovator versions are listed.
   c) Go to the directory that fits your Innovator version and extract the archive `XPlmArasConversionServerPlugin_x64.zip`.
      ➔ In the extracted directory, the directories `ConversionServer` and `Imports` are listed.
   d) Copy the folder `ConversionServer` to the installation directory of the Aras Conversion Server framework, overwriting existing.
   e) Start the IIS root node.

   ➔ Required plugin DLLs are available to the Aras Conversion Server framework.
2. Import AML definitions to Innovator:
   a) From the extracted `XPlmArasConversionServerPlugin_x64` directory, import the database definition package `Imports\imports.mf` using the *Package Import Export Utilities* into the Innovator database in use.

   ➔ Integration-specific conversion rules and other settings are available in the database.

Result

Installation of the XPLM Conversion Server Plugin is complete.

9.3.4.4 Registering plugin in Aras Conversion Server framework

Complete this procedure to register the plugin in the Aras Conversion Server framework.

Procedure

1. Go to the installation directory of the Aras Conversion Server framework and edit the file `ConversionServerConfig.xml`.
2. In the section `sectionGroup` at the top, add the following tag to register the plugin:

```
...
<configSections>
...
  <sectionGroup name="ConverterSettings">
    <section name="XPlmArasConversionServerPlugin"
 type="XPlmArasConversionServerPlugin.Configuration.XPlmArasConverterConfiguration,
 XPlmArasConversionServerPlugin" />
  </sectionGroup>
</configSections>
...
```

3. In the section `Converters`, add the tag `Converter` to register the converter. This tag defines the DLL for the converter and the class within the DLL that performs the conversion.

```
<ConversionServer>
  <Converters>
    <Converter name="XPlmArasVaultFileConverter"
 type="XPlmArasConversionServerPlugin.XPlmArasVaultFileConverter,
 XPlmArasConversionServerPlugin" />
    ...
  </Converters>
</ConversionServer>
```

4. In the section `ConverterSettings`, add the tag `XPlmArasConversionServerPlugin` and within one or several job definition. The following code shows examples for the common CAD applications in the integration scope. Adjust them as required.

```
<ConverterSettings>
    <XPlmArasConversionServerPlugin>
      <JobDefinitions>
        <JobDefinition ID="1" CreationSystem="SOLIDWORKS" JobType="PDF"
 FunctionModule="arasBatch2"
 ScriptEngine="XPlmSolidWorksArasScriptEngine.CScriptEngine"/>
        <JobDefinition ID="2" CreationSystem="INVENTOR" JobType="PDF"
 FunctionModule="arasBatch2"
 ScriptEngine="XPlmInventorArasScriptEngine.CScriptEngine"/>
        <JobDefinition ID="3" CreationSystem="INVENTOR" JobType="TIFF"
 FunctionModule="arasBatch2"
 ScriptEngine="XPlmInventorArasScriptEngine.CScriptEngine"/>
        <JobDefinition ID="4" CreationSystem="INVENTOR"
 JobType="PDF:TIFF" FunctionModule="arasBatch2"
 ScriptEngine="XPlmInventorArasScriptEngine.CScriptEngine"/>
        <JobDefinition ID="5" CreationSystem="INVENTOR" JobType="JPEG"
 FunctionModule="arasBatch2"
 ScriptEngine="XPlmInventorArasScriptEngine.CScriptEngine"/>
        <JobDefinition ID="6" CreationSystem="INVENTOR" JobType="STEP"
 FunctionModule="arasBatch2"
 ScriptEngine="XPlmInventorArasScriptEngine.CScriptEngine"/>
        <JobDefinition ID="7" CreationSystem="NX" JobType="PDF"
 FunctionModule="arasBatch2"
 ScriptEngine="XPlmNXArasBatchScriptEngine.CScriptEngine"/>
        <JobDefinition ID="8" CreationSystem="AUTOCAD" JobType="PDF"
 FunctionModule="arasBatch2"
 ScriptEngine="XPlmAutoCADArasScriptEngine.CScriptEngine"/>
        <JobDefinition ID="9" CreationSystem="AUTOCAD" JobType="STEP"
 FunctionModule="arasBatch2"
 ScriptEngine="XPlmAutoCADArasScriptEngine.CScriptEngine"/>
        <JobDefinition ID="10" CreationSystem="AUTOCAD" JobType="BMP"
 FunctionModule="arasBatch2"
 ScriptEngine="XPlmAutoCADArasScriptEngine.CScriptEngine"/>
        <JobDefinition ID="11" CreationSystem="AUTOCAD" JobType="TIF"
 FunctionModule="arasBatch2"
 ScriptEngine="XPlmAutoCADArasScriptEngine.CScriptEngine"/>
        <JobDefinition ID="12" CreationSystem="OTHER" JobType="TIF"
 FunctionModule="arasBatch2" ScriptEngine="XPlmBlockerScriptEngine.CScriptEngine"/>
        <JobDefinition ID="13" CreationSystem="CREO" JobType="PDF"
 FunctionModule="arasBatch2"
 ScriptEngine="XPlmProEArasBatchScriptEngine.CScriptEngine"/>
      </JobDefinitions>
    </XPlmArasConversionServerPlugin>
</ConverterSettings>
```

Each job definition must have a unique `ID` and the required target `ScriptEngine` to run the Conversion Task with defined options. `JobeType` and `CreationSystem` are defined by the Conversion Rule and passed to the plugin via the Conversion Task

5. Save and close the file.

Result

The Aras Conversion Server framework is set up.

9.3.4.5 Setting up integration components

Complete this procedure on the Conversion Server worker to set up the integration components for the conversion.

Enabling Innovator legacy login

Configuration file: `%xPlmRootDir%\xml\XPlmArasConnector.xml`

For the Conversion Server Plugin to work, the legacy login for Innovator with defined connection parameters is required. By default, the Innovator login is used.

Example

```
<Field AdvancedLevel="1">
  <Name>EnableArasLogging</Name>
  <Value>true</Value>
</Field>
```

Defining connection to Innovator

Configuration file: `%xPlmRootDir%\xml\XPlmArasLogonParameter.xml`

Define the connection parameters for the Innovator server in use. Required parameters:

- DATABASE
- URL
- USER
- PASSWORD

Defining connection to Conversion Server worker

Configuration file: `%xPlmRootDir%\xml\XPlmArasConversionServerPlugin.xml`

Define IP address and port for the Conversion Server worker used by IIS. In most cases, the localhost IP address 127.0.0.1 is sufficient, but sometimes it may be required to enter the real IP address of the machine.

Example

```
<Field>
  <Name>ArasConversionServerConverterWatcherIP</Name>
  <Value>127.0.0.1</Value>
</Field>
<Field>
  <Name>ArasConversionServerConverterWatcherPort</Name>
  <Value>6653</Value>
</Field>
```

Alternative communication protocol (conversionServerConverter)

Configuration file: `%xPlmRootDir%\xml\XPlmArasConversionServerPlugin.xml`

The communication between the plugin and the ConversionServerConverter is done via TCPIP on the defined port (usually 6653). Since release 4.3.0, there is an additional channel for the plugin to communicate with the ConversionServerConverter via transfer files in case the defined port (6653) is blocked or affected by firewalls or routing issues. This is activated using the parameter `ArasConversionServerConverterConnectProtocol`.

Example

```
<Field>
```

```
  <Name>ArasConversionServerConverterConnectProtocol</Name>
  <Value>file</Value>
</Field>
```

Allowing Conversion Server framework to delete files

Configuration file: `%xPlmRootDir%\xml\XPlmInventorArasConnector.xml`

In the following configuration files, set the shown properties to `false`. This allows the Conversion Server framework to delete files. Otherwise an error is raised and the Conversion Task will fail.

Configuration file	Property
`XPlmCatiaConnector.xml`	`CatiaArasChangeReadOnlyOnLockUnlock`
`XPlmNXArasConnector.xml`	`NXArasChangeReadOnlyOnLockUnlock`
`XPlmSolidEdgeArasConnector.xml`	`SolidEdgeArasChangeReadOnlyOnLockUnlock`
`XPlmSolidWorksArasConnector.xml`	`SolidWorksArasChangeReadOnlyOnLockUnlock`

Example

```
<Field>
  <Name>CatiaArasChangeReadOnlyOnLockUnlock</Name>
  <Value>false</Value>
</Field>
```

This setting is not required for AutoCAD, BricsCAD, Creo Elements, Creo Parametric, Inventor and Revit.

Activating post actions

Transaction file: `%xPlmRootDir%\xml\XPlmArasInventorTransaction.xml`

Activate the exit handler `PostAction_initApplication` for the following customer script engines:

- XPlmAutoCADCustomerScriptEngine.CScriptEngine
- XPlmBricsCADCustomerScriptEngine.CScriptEngine
- XPlmCatiaCustomerScriptEngine.CScriptEngine
- XPlmInventorCustomerScriptEngine.CScriptEngine
- XPlmNXCustomerScriptEngine.CScriptEngine
- XPlmProECustomerScriptEngine.CScriptEngine
- XPlmRevitCustomerScriptEngine.CScriptEngine
- XPlmSolidDesignerCustomerScriptEngine.CScriptEngine
- XPlmSolidWorksCustomerScriptEngine.CScriptEngine

Transaction file	Description
`XPlmArasAutoCADTransaction.xml`	Uncomment the whole `transaction` block to activate.
`XPlmArasBricsCADTransaction.xml`	Uncomment `table` block only to activate.
`XPlmArasCatiaTransaction.xml`	Uncomment the whole `transaction` block to activate.
`XPlmArasInventorTransaction.xml`	Uncomment the whole `transaction` block to activate.
`XPlmArasNXTransaction.xml`	Uncomment `table` block only to activate.
`XPlmArasProETransaction.xml`	Uncomment `table` block only to activate.
`XPlmArasRevitCOMTransaction.xml`	Uncomment `table` block only to activate.

Transaction file	Description
XPlmArasSolidDesignerTransaction.xml	Uncomment the whole `transaction` block to activate.
XPlmArasSolidWorksTransaction.xml	Uncomment the whole `transaction` block to activate.

This action ensures that the local workdir information (one for each ConversionTask) is correctly transferred to the customer script engine.

9.3.4.6 Setting up XPLM Conversion Server Converter

On the Conversion Server worker, the helper-application Conversion Server Converter must run to process conversions.

Procedure

1. Go to the directory `%xPlmRootDir%\bin\x64` and start the executable `XPlmArasConversionServerConverter.exe`.
2. Right-click the icon in the tray area and select **Start**.
3. **Optional**: Add the executable to a Windows logon task or a batch script in the Windows start-up folder to start it automatically.

   Important here is to add the parameter `/start` to activate it after the start, for example `%xPlmRootDir%\bin\x64\XPlmArasConversionServerConverter.exe /start`.
4. **Optional but recommended**: Use a more robust mode for the converter. Set in file `%xPlmRootDir%\xml\XPlmArasConversionServerPlugin.xml` the option `UseWorkerProgram=true`.
   - false (default): The Conversion Server Converter executes the script engine itself. The script engine is started once and then processes all jobs one after the other. **Disadvantage**: If the process crashes, everything is blocked as there is only this one process.
   - true: Instead of executing the script engine directly, an XML document is created. A separate process called `XPlmTaskWorker.exe` is then started. Each task runs in its own process with a newly started script engine. **Advantage**: If a process crashes, this only affects this one task, and the rest continues to run.

Result

XPLM Conversion Server Converter is active.

9.3.4.7 Setting up Agent Service

This procedure is for reference only, as the Agent Service will most likely already be configured.

Before you start

If Agent Service is not installed, you must install and configure it first. See the Aras documentation for more information.

Procedure

1. Go to the Innovator server.
2. In the installation directory `AgentService`, edit the file `conversion.config`.
3. Uncomment the tag `cycle` and change as follows:

```
<configuration>
...
  <Innovator>
    <Conversion>
      <ProcessingCycles>
        <cycle DB="<db name>" User="<db user>" Password="<user password>"
 Interval="20000" MaxBatch="1" />
      </ProcessingCycles>
    </Conversion>
  </Innovator>
</configuration>
```

   - `DB`: Defines the Innovator database in use.
   - `User`: Defines the database user. The user must be a member of the *Conversion Manager* group and must have permissions to claim Inventor items that were created by other users.
   - `Password`: Defines the user's password.
   - `Interval`: Defines how often the Conversion Server checks for conversion tasks to be processed. Leave the default 20000 (milliseconds).

     `MaxBatch`: Defines the number of conversion tasks to be processed at the same time. The default is 5. To process only one task at the same time, set `MaxBatch="1"`.
4. Save and close the file.
5. Restart the Agent Service using **Control Panel** > **Administrative Tools** > **Services**.

Result

The Agent Service is set up.

9.3.5 Troubleshooting

Find here information about common errors that may occur and possible solutions to resolve them. When errors occur, it's also helpful to enable logging.

Common errors

Error	Possible Reason(s) and Solution(s)
Conversion tasks are not created.	**Solution**: In Innovator, check if one or more Conversion Rules are assigned an event type with a server method already added (under Rule Events, Task Event Templates).
Error message: `No connection could be made because the target machine actively refused it <ip address>:6653`	**Solution**: ■ On the Conversion Server worker, check if the service *XPlmArasConversionServerConvert* is running. ■ In the file `%xPlmRootDir%\xml\XPlmArasConversionServerPlugin.xml`, check if the configured port `ArasConversionServerConverterWatcherPort` is free to use.

Enabling logging

You can enable logging for each integration component individually.

Component	File
Conversion Server Plugin	`%xPlmRootDir%\xml\XPlmArasConversionServerPlugin.xml`
Innovator connector	XPlmArasConnector.xml (p. 126)
Inventor connector	XPlmInventorConnector.xml (p. 121)
Script engine	XPlmInventorArasConnector.xml (p. 122)

9.4 Encryption Helper

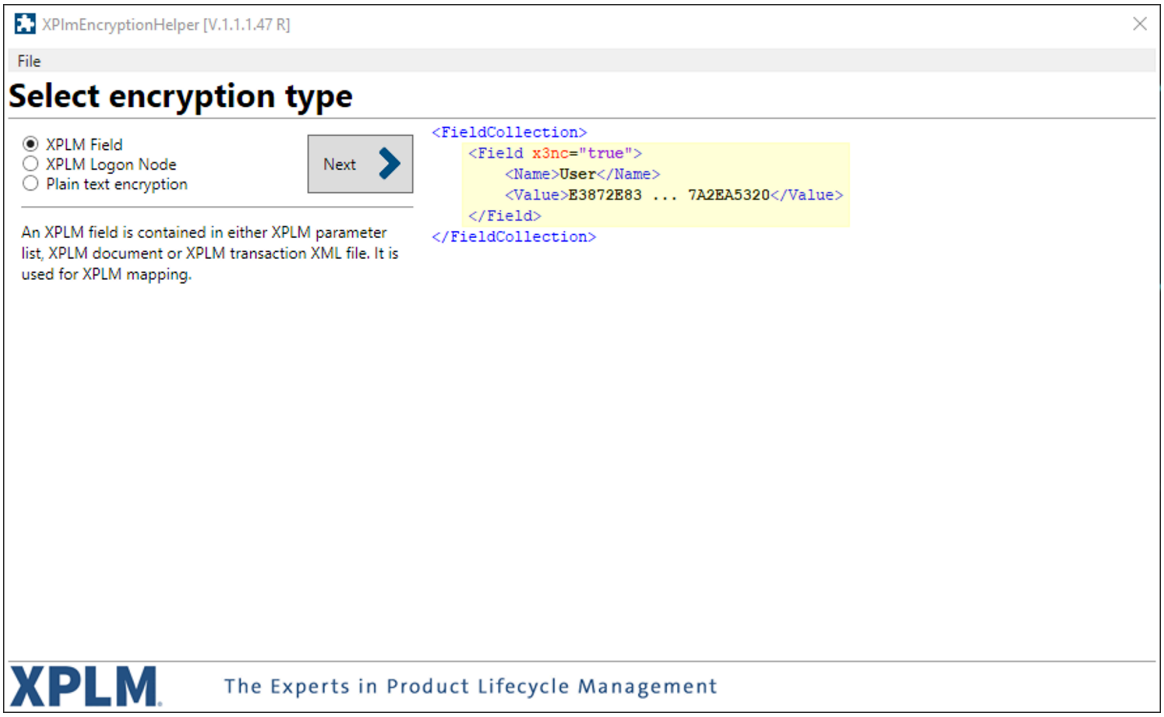
9.4.1 Introduction

The Encryption Helper is used to encrypt fields that must not be read in plain text, for example passwords.

You can encrypt required passwords and other information to prevent the value from appearing as plain text in configuration files. This is true for most configuration files, at least for login parameters and for regular transactions written in the current data model.

Make sure that you keep the original password in a safe place if you want to encrypt it. It is not possible to recover the original value after encryption.

Figure 1: User interface



9.4.2 Usage

The application is installed from the integration installer in the step *Tool components*. After installation, a shortcut to start the application is placed on the desktop.

About this task

The Encryption Helper can encrypt a defined field, a logon node or just any plain text. It provides examples on how to use the encrypted information.

Procedure

1. Start Encryption Helper via the shortcut on the desktop.
2. Select an encryption type and click **Next**. Click **Back** to select another type.
3. **XPLM Field**: Enter the name of the field and its value, and click **Encrypt**.
4. **XPLM Logon Node**: Enter the name of the field and its value, and click **Encrypt**.
5. **Plain text encryption**: Enter the plain text and click **Encrypt**.

Result

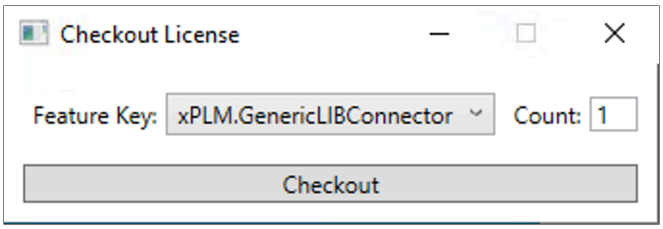
The entered information is encrypted. Click **Copy** to copy the information to the clipboard and use it in the desired file.

9.5 License Test Tool

9.5.1 Introduction

You can use the License Test Tool to check out a feature license from Innovator and verify in Innovator if the license was consumed.

Figure 2: User interface



9.5.2 Usage

The application is installed from the integration installer in the step *Tool components*. After installation, a shortcut to start the application is placed on the desktop.

Before you start

Make sure a valid integration license file `*.lic` is installed in directory `%xPlmRootDir%\xml`.

Procedure

1. Go to the directory `%xPlmRootDir%\xml` and edit the file `XPlmArasCheckoutLicenseTool.xml`.
2. Enter the connection information for Innovator as follows:
   - URL: Defines the server URL for Innovator.
   - DATABASE: Defines the Innovator database.
   - USER: Defines the user name.
   - PASSWORD: Defines the user's password.
3. Save the file.
4. Start License Test Tool via the shortcut on the desktop.
5. Select a license from the list, define a quantity and click **Checkout**.
6. Log in to Innovator and click the user icon in the top-right corner.
7. Select **Licenses** > **License Manager** and check if the license with the correct quantity was consumed.

   The consumed license may also appear as part of an all-features bundle.
8. To check in the consumed licenses, select the license(s) and click **Terminate Selected Sessions**.

Result

The feature licenses required for the integration are correctly consumed.

Troubleshooting

If the feature license cannot be consumed, check the feature tree in Innovator and make sure the selected license exists.

10 Reference information

10.1 Main configuration files

10.1.1 XPlmConnector.xml

This file contains general integration settings. Only the important settings that are likely to be changed are listed. Do not change others.

Location: `%xPlmRootDir%\xml\XPlmConnector.xml`

Settings marked with * are important for initial setup.

	Name	Description
	Language	Defines the UI language in menus and dialogs. Make sure the language code is defined in the corresponding resource file and set it here accordingly. ■ EN (default): English ■ DE: German
*	LocalWorkDir	Defines the default local work directory used by the integration to exchange data. Make sure this path is writable. A common solution is to add the user directory to the path, for example `%USERPROFILE%\caxtmp\`. Also modify all the mentioned sub-directories under this path as well.
	LocalCheckoutDir	Defines the default local checkout directory.
	LocalCheckinDir	Defines the default local checkin directory.
	RemoteCheckoutDir	Defines the default remote checkout directory.
	RemoteCheckinDir	Defines the default remote checkin directory.
	StandardPartDir	Defines the default standard part directory.
	CopiedPartCheckoutDir	Defines the default checkout directory for copied parts.
*	StandardPartCheckoutDir	Defines the default checkout directory for standard parts. Make sure this path is writable. A common solution is to add the user directory to the path, for example `%USERPROFILE%\cadlib\`.
*	TemplateCheckoutDir	Defines the default checkout directory for templates. Make sure this path is writable. A common solution is to add the user directory to the path, for example `%USERPROFILE%\templates\`.
	UseNewDialogs	Defines if new dialogs are to be used. Do not change the default setting unless instructed to do so. ■ true (default): Use new dialogs. ■ false: Use old dialogs.
	AbsoluteClasspath	Defines locations of `.jar` files. The definition must include the absolute path to the file. Separate multiple file definitions by `;`.

	Name	Description
	RelativeClasspath	Defines locations of `.jar` files. The definition must include the relative path to the file starting from the installation directory `%xPlmRootDir%`. Separate multiple file definitions by `;`.
	LicenseServerHost	If set up, defines the address to the server running License Monitor. Uncomment to enable.
	LicenseServerPort	If set up, defines the port of the server running License Monitor. Uncomment to enable.
	LicenseServerProtocol	If set up, defines the protocol (http/https) of the server running License Monitor. Uncomment to enable.
	JniOptions	Defines options for the Java-to-C++ bridge.

10.1.2 XPlmInventorConnector.xml

This file contains settings for the Inventor connector.

Location: `%xPlmRootDir%\xml\XPlmInventorConnector.xml`

Option	Description
EnableInventorLogging	It is a configuration option that determines whether logging is enabled or disabled in the application. ■ true: Logging is enabled and the connector will record relevant events, messages, or errors to a log file ■ false (default): Logging is disabled, and no logging information will be generated or stored.
InventorLogFile	Gives the path and name to the log file generated if logging is enabled for the Inventor connector. ■ C:\caxtmp\log\Inventor.log (default)
InventorLogLevel	Determines the level of detail and importance of the information that gets recorded in the log file generated if logging is enabled (higher number indicates more log messages). ■ `10` (default)
InventorCreateBitmapPreview	Controls whether a preview file should be created for the current document in Innovator. ■ true: Bitmap previews are created during save. ■ false (default): No bitmaps are created during save.
InventorDefaultSaveName_2	Default save name of unsaved Inventor assembly files. ■ Assembly .iam (default)
InventorDefaultSaveName_3	Default save name of unsaved Inventor drawing files. ■ Drawing .idw (default)
InventorUpdateProperties LatestGeneration	■ true (default): The properties of the latest generation are used. ■ false: The properties are pulled from the same generation of the CAD document that is currently in session.
InventorBomName	Value of the BOM structure within Inventor. ■ ModelData (default)
InventorCustomPropertiesName	A parameter that can be used within transactions, pointing to the custom file properties section. This setting depends on the language of Inventor installation. ■ Inventor User Defined Properties (default) Should not be changed!
InventorTemplateDir	Path to the template files. ■ C:\XPLM\Templates (default)

10.1.3 XPlmInventorArasConnector.xml

This file contains settings for the script engine used in this integration.

Location: `%xPlmRootDir%\xml\XPlmInventorArasConnector.xml`

Name	Description
EnableInventorArasLogging	Determines whether logging is enabled or disabled. ■ true: Logging is enabled. ■ false (default): Logging is disabled.
InventorArasLogFile	Path to the log file for the Inventor connector. ■ C:\caxtmp\logInventorAras.log (default)
InventorArasLogLevel	Describes the log level (higher number indicates more log messages). ■ 1: Only Errors are stored to the log file ■ 5: Errors and warnings are stored to the log file ■ 10 (default): Errors, warnings and top level functions are stored to the log file ■ 100: Errors, warnings, top and second level functions plus additional information are stored to the log file ■ 1000: Full logging
InventorArasLogType	Defines the log type. ■ SimpleAppendTextLog ■ SharedAppendTextLog (default) ■ SimpleXmlAppendLog ■ SharedXmlAppendLog ■ SimpleHtmlAppendLog ■ HtmlPageLog
InventorArasIncludeDate	■ true: Date and Time information is added to the log file. This helps to analyze the performance of the integration. ■ false (default): No Date/Time information is added to the log file. With that log files can be compared with each other.
InventorArasUseSingletonLog	If set to ■ true (default): One log file for the whole integration. ■ false : Each Component (Connector, ScriptEngine...) logs into a separate log file
InventorMenuFiles	Defines the menu definition XML file used by the integration. ■ XPlmInventorArasAddin.xml (default) Should not be changed.

Name	Description
InventorDefaultSaveMode	Pre-selection for the save mode. ■ 1 (default): Modified ■ 2: All ■ 3: Locked
InventorArasGetItemRepeatConfig	Retrieves a resolved structure of a CAD item with detailed metadata. ■ Avoid line breaks in the select statement inside the CDATA block to prevent attribute selection issues. ■ Do not use `select=*`, it may affect performance and unintentionally include large metadata sets like viewable file attributes.
InventorArasSynchronizeStandardParts	Synchronizes standard parts from Innovator. Retrieves items marked as `is_standard=1` and `is_current=1`.
InventorArasGetRelatedParts	Retrieves parts related to selected CAD file using the PE_GetRelatedParts action.
InventorArasDocument DocumentItemRelationItemType	■ CAD Structure (default)
InventorArasDocument FileItemRelationItemType	■ *(empty)*
InventorArasBatch DocumentFileRelationItemType	■ *(empty)*
InventorArasDocument PartRelationItemType	■ blank: File is saved under **Viewable File** ■ CADFiles: File is saved in ribbon **Files** ■ *(empty)*
InventorArasPart DocumentRelationItemType	■ Part CAD (default)
InventorArasPartClassification	Value for the classification for a Inventor part. ■ Mechanical/Part (default)
InventorArasAssemblyClassification	Value for the classification for a Inventor assembly. ■ Mechanical/Assembly (default)
InventorArasCopyAction	■ xplm_copyCAD (default) ■ copyDocument
InventorArasLockNewItems	Defines whether new created CAD documents are claimed in Innovator after initial save or copy actions. ■ true (default): Initially saved items are claimed to the current user. Standard parts are not claimed. ■ false

Name	Description
InventorArasUpdateOnLoadRecursive	Defines the objects for which properties are updated after load if check box **Update Properties** in the *Load dialog* is selected. ■ true (default): The post action after LOAD updates the properties of all loaded objects ■ false: The post action after LOAD just updates the properties of the top object
InventorDestroySaveProgressWindow	Defines whether the save progress user interface is automatically closed. This UI element shows, which CAD file is stored to which Innovator item and whether the action was successful. ■ true (default): The save progress dialog closes automatically. ■ false: The save progress dialog does not close automatically after the save process has finished. User must manually close the dialog in this case.
InventorArasCheckMember OnPLMDocumentExist	If set to `true`, existing instances of family tables are recognized by their file name and missing CLB files are added to the local workspace. ■ true (default) ■ false
InventorUpdatePartForLocked DocumentOnly	Defines whether unclaimed parts are considered during Item and BOM creation. ■ true: **Create Part** and **Update BOM** actions only work for objects claimed by current user. ■ false (default)
InventorArasBulkBomOnSave	Defines whether new BOM algorithms integrated with version 4.4 are used ■ true (default) ■ false
InventorArasBomDialogEnabled	Defines whether the new *BOM dialog* appears during BOM creation process. ■ true (default) ■ false
InventorArasBomDialogShowExternParts	Defines whether non-CAD objects are displayed within the *BOM dialog*. ■ true (default) ■ false

Name	Description
InventorUpdatePropertiesLatestGeneration	Defines from where metadata of an Innovator object containing a 3D object are retrieved. ■ true: the system inquires the metadata from the newest generation of the Innovator object. ■ false (default): the system inquires the metadata of the Innovator object related to the current generation.
InventorArasDisableOldGenerations OnLoad	If set to `false`, it deactivates the load handling capability for assemblies with components of different generations. ■ true (default) ■ false

10.1.4 XPlmArasConnector.xml

This file contains settings for the Innovator connector.

Location: `%xPlmRootDir%\xml\XPlmArasConnector.xml`

Name	Description
EnableArasLogging	Defines an option to enable logging for the Innovator connector. ■ true: Enable logging. ■ false (default): Disable logging.
ArasLogFile	Defines the path and name of the log file. Check it this path is writable and keep it short.
ArasLogLevel	Defines the log level. A higher number (10-1000) writes more log information.
ArasLogType	Defines the log type. Several types and formats are available: ■ SimpleAppendTextLog ■ SharedAppendTextLog ■ SimpleXmlAppendLog ■ SharedXmlAppendLog ■ SimpleHtmlAppendLog ■ HtmlPageLog Simple*Log: This type allows only one process to add log messages to a file. Existing files are overwritten on create, otherwise a new one is created. Shared*Log: This type extends the simple log type and allows multiple processes to add log messages to a file. Existing files are not overwritten on create and logs are attached. Otherwise a new file is created. HtmlPageLog: Behaves like the simple log type. Log messages are split when the size of the file exceeds a certain limit. Each log page links to the previous page so that it can be browsed in a web browser. Requires the additional option *LOG_SIZE* to define the number of log messages within a single HTML page.
ArasUseLegacyLogin	Defines whether to use a silent login behaviour with username and password defined in file `XPlmArasLogonParameter.xml`, or the Aras login dialog. Product-specific connector files may override this setting with another default. ■ true: Use silent login with defined username and password, or another authentication method (see below). ■ false (default): Use Aras login.
ArasWindowsAuthentication	Defines whether to use Windows authentication with Innovator. ■ true: Enable Windows authentication. ■ false (default): Disable Windows authentication. Do not enable *ArasBrowserAuthentication* when using Windows authentication.

Name	Description
ArasBrowserAuthentication	Defines whether to use a separate browser window for entering the login information. ■ true: Open separate window. ■ false (default): Use current window.
ArasCertificateAuthentication	Defines whether to use certificate authentication with Innovator. ■ true: Enable certificate authentication. ■ false (default): Disable certificate authentication.
ArasTokenAuthentication	Defines whether to forward the certificate to the Innovator server and receive a token back. ■ true: Enable token authentication. ■ false (default): Disable token authentication.
ArasRefreshToken Authentication	Defines whether to use Windows authentication in XPlmWebHost-based products. ■ true: Enable Windows authentication. ■ false (default): Disable Windows authentication. Do not enable any other of the authentication options when using this method.
ArasHttpServer ConnectionTimeout	Defines the timeout in seconds for any call to Innovator.
ArasStartClient	Defines the path to a specific browser executable. If nothing is defined, the default browser is used.
ArasUnlockItemsOnExit	Defines whether items should be unlocked when exiting. CAD documents are not unlocked if they were already locked in Innovator before being loaded into Inventor. ■ true: Items are unlocked. ■ false (default): Items are not unlocked.
ArasCopyStandardParts	Defines whether to copy standard parts. ■ true: Parts are copied. ■ false (default): Parts are not copied.
ArasProjectItemType	Defines the item type for project.
ArasQuotationItemType	Defines the item type for quotation.
ArasStandardNamesItemType	Defines the item type for standard names.
ArasSAPMaterialType	SAP only: Defines the material type.
ArasSAPItemCateg	SAP only: Defines the item category.
ArasSAPBOMRelationship	SAP only: Defines the BOM relationship.
ArasSAPClassification Relationship	SAP only: Defines the classification relationship.
ArasSAPBOMHeadItemType	SAP only: Defines the item type for the BOM header.
cax_*	Defines properties that are set on the Innovator object.
ArasModelItemPartNumberField	Defines the part number field for a model item.
ArasModelItem DocumentNumberField	Defines the document number field for a model item.

Name	Description
ArasDeleteLoadFrom ClientOnConnect	Defines whether objects added to the load queue in Innovator should be deleted when connecting to Innovator. ■ true: Delete load queue. ■ false (default): Don't delete load queue.
ArasDiscussionPanelWidth	Defines the default width of the *Discussion Panel*.
ArasDiscussionPanelHeigth	Defines the default height of the *Discussion Panel*.
ArasDiscussionPanelUpdate	Defines the update behavior of the *Discussion Panel*. ■ true (default): The *Discussion Panel* is updated and shows the conversation regarding to the currently active model, whenever a PLM known model is opened/loaded in Inventor. Furthermore it is updated when using the functions **Copy** or **Assign Document**. ■ false: The *Discussion Panel* is not updated.
ArasRelationshipDeleteFlags	Defines whether to delete relationships on update in Innovator. ■ 1 (default): Delete relationships. ■ 0: Keep relationships.
ArasGenericCopyAction	Defines the generic copy action.
ArasGetCADStructureAML	Defines an AML code to read the CAD structure.
ArasGetPartStructureAML	Defines an AML code to read the Part structure.
ArasGetItemAccessAction	Defines the access action.
ArasHideLogin DialogForAuthLogin	Defines whether to hide the login dialog for any alternative authentication method. ■ true: Hide login dialog. ■ false (default): Show login dialog.
AMLPackageVersionToCheck	Defines the package version to check.
ArasGetAMLPackageVersion	Defines an AML code to read the package version.
ArasGetXPConfiguration WriteToTempOnConnect	Defines a low-code feature to save Innovator settings in the temp directory on connect. ■ true (default): Enable feature. ■ false: Disable feature.
ArasGetXPConfiguration Properties	Defines an AML code to read configuration properties stored in Innovator.
ArasGetXPConfiguration Resources	Defines an AML code to read configuration resources stored in Innovator.
CustomDialogColors	Defines a custom color theme to use. Only applicable for MCAD integrations. The theme *ColorTheme_Gray* is the default for Aras-MCAD integrations.
ArasFeaturelicenseTimeout	Defines the time (in seconds) the system waits for a response when checking out the feature license before timing out. With each Invoke, the system checks locally whether the feature license needs to be renewed. ■ 3600 (default)

10.1.5 XPlmArasLogonParameter.xml

This file contains the connection information for Innovator.

Location: `%xPlmRootDir%\xml\XPlmArasLogonParameter.xml`

Settings marked with * are important for initial setup.

Default login parameters are defined between `<logon>` blocks. You can create as many of these blocks as there are different connections and name them using the option `ENVIRONMENT`. Environment selection is only supported for integrations that use a login dialog. Other integrations use the first `<logon>` block in this file.

Since release 4.5.0, define only the URL to Innovator in this file and use the **Connection Parameters** function in the integration menu to select the corresponding environment. The **Connect** function brings up the official Aras login dialog for this environment and allows for further authentication. See User Guide for more information.
If the integration requires another authentication method without the Aras dialog, you still have to define database and user credentials in this file and define in file XPlmArasConnector.xml (p. 126) the setting `ArasUseLegacyLogin=true` together with other settings.

	Parameter	Value
*	ENVIRONMENT	Defines the environment, for example *Development* or *Production*.
	COMPUTERNAME	Defines the name of the client computer. Do not change.
	COMPUTERUSER	Defines the user logging in on the client computer. Do not change.
	DATABASE	**Optional**: Defines the Innovator database.
*	URL	Defines the URL of the Innovator server.
	USER	**Optional**: Defines the user name.
	PASSWORD	**Optional**: Defines the user password. Do not enter security-relevant information such as passwords in plain text. To encrypt such strings and enhance security, always use the tool Encryption Helper (p. 116). When using encrypted strings, you must use the option `x3nc="true"`.

10.1.6 XPlmCopyArasConfiguration.xml

This file contains configuration settings for the *Copy dialog*.

Location: `%xPlmRootDir%\xml\XPlmCopyArasConfiguration.xml`

The configuration file is read once during connection to Innovator. You must restart Inventor to apply any changes made in the configuration file.

Configuration options

- **Transaction:** `Config`

Name	Description
EnableLogging	Defines an option to enable logging for the XPlmComLog. ■ true: Enable logging. ■ false (default): Disable logging.
LogFile	Defines the path and name of the log file. Check it this path is writable and keep it short. Default: empty
LogLevel	Specifies the log level of the XPlmComLog ■ 1: Error ■ 10 (default): Warning ■ 100: Info ■ 200 Debug
LogType	Defines the log type. Several types and formats are available: ■ SimpleAppendTextLog ■ SharedAppendTextLog ■ SimpleXmlAppendLog ■ SharedXmlAppendLog ■ SimpleHtmlAppendLog ■ HtmlPageLog Simple*Log: This type allows only one process to add log messages to a file. Existing files are overwritten on create, otherwise a new one is created. Shared*Log: This type extends the simple log type and allows multiple processes to add log messages to a file. Existing files are not overwritten on create and logs are attached. Otherwise a new file is created. HtmlPageLog: Behaves like the simple log type. Log messages are split when the size of the file exceeds a certain limit. Each log page links to the previous page so that it can be browsed in a web browser. Requires the additional option *LOG_SIZE* to define the number of log messages within a single HTML page.
CopyMethodCall	The Aras AML to copy the documents.Used to customize the server method call.

Name	Description
LoadStructureResolution	Via this field, the resolution to load the structure of the root document from Innovator can be set. The Innovator method `PE_GetResolvedStructure` is used. ■ AsSaved (default) ■ Released ■ Current
GetRelatedDrawingsAML	The Innovator AML to query the related drawings of the selected CAD document.
GetRelatedPartsAML	The Innovator AML to query the related parts of the selected CAD document.
GenerateNewNumberAML	The AML to get a new document number for the copy. To disable the behaviour, set an empty value or remove this field.
DisableDrawingNumberCreation	If set to `true`, the default drawing number creation is disabled. ■ true ■ false
DisablePartNumberCreation	If set to `true`, the default part number creation is disabled. ■ true ■ false
CheckIfStructureIsModified	If set to `true`, the CAD document structure is checked if it is modified, and the user gets informed. ■ true ■ false
RenameFileNames (Structure)	When a document with an `authoring_tool` of this structure is copied, then a new `external_id` for the copied document is created if the value is `true`. ■ true ■ false
RenameFileNamesNegate (Structure)	Same as `RenameFileNames`, but the value is negated. ■ true ■ false
IdentifierPath (Structure)	Sets the path to the field which is used to display the identifier of a document for each dialog context.
ExcludeChilds	If set to `true`, also child elements of excluded standard parts and templates are greyed out in the *Copy* dialog. ■ true (default) ■ false

Client User exits

There are several user exits available. They work in the same way as other user exits in the integration.


Ensure the correct customer script engine name is set.

The following user exit transactions are available for configuration;

Name	Description
OnBeforeInvokeDialog	Is invoked before the dialog is called and can be used to exclude for example standard parts from the *Copy* dialog.
OnAfterInvokeDialog	Is invoked after the dialog is closed and can be used to modify selected documents.
In both user exits above, the document structure which is displayed in the dialog is the `pInput` document. In addition, the current dialog context is given as an attribute of the `pInput` document `DialogContext`.	
OnBefore_UpdateRelatedDrawings	Is invoked before the related drawings are queried and can be used to implement an own logic to query related drawings.
OnAfter_UpdateRelatedDrawings	Is invoked after the related drawings are queried and can be used to exclude drawings from the dialog by setting the exclude flag or to manipulate the data of them.
OnBefore_UpdateRelatedParts	Is invoked before the related parts are queried and can be used to implement an own logic to query related parts.
OnAfter_UpdateRelatedParts	Is invoked after the related parts are queried and can be used to exclude parts from the dialog by setting the exclude flag or to manipulate the data of them.
In the four user exits above, the new selected CAD document in the MainTLV is the `pInput`. In addition, the current dialog context is given as an attribute of the `pInput` document `DialogContext`.	
QueryNewDrawingNumber	Can be used to implement an own logic to query the new number for the copied drawing.
QueryNewPartNumber	Can be used to implement an own logic to query the new number for the copied part.
PreActionOnOKNotify	Can be enabled to make plausibility checks when the user clicks on commit. With this user exit the commit action can be cancelled and the dialog stays open.

In the `QueryNewDrawingNumber` and `QueryNewPartNumber` UserExits, the drawing/part document is the `pInput`. In addition, the current dialog context is given as an attribute of the `pInput` document `DialogContext`. The related CAD document is the value of the field with the name `CopyDialogRootDoc`. From there, all related drawings/parts are set in field `CopyDialogRelatedDrws` or `CopyDialogRelatedPrts`. The listing below shows how to get them.

```
Dim root As IXPlmDocument = TryCast(pInput.ParameterList.FieldCollection.
 getItemByName("CopyDialogRootDoc").Value, IXPlmDocument)

Dim relatedDrws As IXPlmDocumentCollection = TryCast(root.ParameterList.FieldCollection.
 getItemByName("CopyDialogRelatedDrws").Value, IXPlmDocumentCollection)

Dim relatedParts As IXPlmDocumentCollection =
 TryCast(root.ParameterList.FieldCollection.
 getItemByName("CopyDialogRelatedPrts").Value, IXPlmDocumentCollection)
```

Server method user exits

The business logic for copying documents, drawings and parts in Innovator is done via the server method `xplm_copy_doc` on the Innovator server.

You can customize this server method using the following four UserExits;

- PreAction_UpdatePropertiesDocument
- PostAction_Update PropertiesDocument
- PreAction_UpdatePropertiesPart
- PostAction_UpdatePropertiesPart

Do NOT edit any other source code of the method except the mentioned UserExits.

Updating the server method via the **ArasImportTool** replaces the code hence all UserExits will be replaced too. To keep the customized code, please make a backup copy before the update and replace the empty UserExits with the old one from the backup.

Method call

You can customize the server method call with the field `CopyMethodCall` by adding the following parameters to the AML template

Parameter	Description
usesqltoupdatecopiedproperties	When an item in Innovator is copied, a property update is performed afterwards. An AML call is used to do this. To use SQL instead, set this parameter to `true`. ■ true ■ false (default)
updatekeyednameaftersqlupdate	Only relevant when `usesqltoupdatecopiedproperties` is set to `true`. When an Innovator item is copied, no new `keyed_name` is computed. To create a new one, an empty AML property update is performed on the copied item. To disable this update, set this parameter to `false`. ■ true (default) ■ false
removepartbom	Removes BOM from a copied part. ■ true (default) ■ false
removepartalternate	Removes alternate parts from a copied part. ■ true (default) ■ false
removepartaml	Removes all manufacturer parts from a copied part. ■ true (default) ■ false
removepartdocument	Removes all documents from a copied part. ■ true (default) ■ false
removepartgoal	Removes all goals from a copied part. ■ true (default) ■ false

Parameter	Description
removepartrequirement	Removes all requirements from a copied part. ■ true (default) ■ false
removepartreqdoc	Removes all requirements documents from a copied part. ■ true (default) ■ false

10.1.7 CustomDialogColors.xml

This file contains color settings for integration dialogs. HTML names and HEX values are supported.

Location: `%xPlmRootDir%\xml\CustomDialogColors.xml`

Name	Description
XPLM_Main_1	Specifies the colors used for certain dialog and UI elements
XPLM_Main_2	Specifies the colors used in the background of grid headers
XPLM_Highlight_1	Defines the colors applied to UI elements when they are in the hover-state.
XPLM_Highlight_2	Defines the colors applied to rows in the property grid when they are in the hover-state.
XPLM_Selected_1	Specifies the colors used for enabled filters in the grid fields
XPLM_Main_TitleBar	Specifies the colors used in the background and borders of dialog title bars.
XPLM_Main_Button_TitleBar_Background	Specifies the colors used in the background of UI elements in the dialog title bar
XPLM_Main_Button_TitleBar_Foreground	Specifies the colors used in the foreground of UI elements in the dialog title bar
XPLM_Main_Header_Foreground	Specifies the color used for the font of grid headers.
XPLM_Main_Header_Background	Specifies the colors used in the background of grid headers
XPLM_Main_Header_Hover	Defines the color applied to grid headers in the hover-state
XPLM_Main_Header_Selected	Specifies the color used for grid headers in the selected-state
XPLM_Main_Border	Specifies the color used in the borders of ribbon sections, property grid (inner), tabs, check-boxes and so on.
XPLM_Main_Border_Dark	Specifies the color used in the outer borders of the property grid, content window, and search fields.

10.2 Silent-mode installation

You can also install this XPLM product in silent-mode. In silent-mode, you can easily install the product from a batch file without showing the installer GUI. Alternatively, you can start the installer with a batch file in GUI mode and have options already preset. This allows a controlled installation by the user or by other automated installation routines.

The installer components are of type Windows Installer (MSI) and require corresponding parameters for silent-mode installation.

The Visual C++ runtimes are normal executables. Always install all x64/x86 runtimes that come with the installer package.

Understanding installer structure

When you start an installation using the installer, required files are copied first to the directory `%ProgramData%\XPLM Solution GmbH` and are executed from there.

```
XPLM Solution GmbH
├──cmd
├──log
└──packages
```

- `cmd`: Contains the batch files `Setup-*_admin.bat` and `Setup-*_user.bat`.
  - The file `Setup-*_user.bat` contains the copy commands for the required MSIs from the original location to the directory `%ProgramData%\XPLM Solution GmbH\packages`.
  - The file `Setup-*_admin.bat` installs the individual MSIs from this new location with the parameters as defined in the installer.
- `log`: Contains log files of each installed component.
- `packages`: Contains copies of all MSIs used for installation, modification or uninstallation.

Use the definitions in the file `Setup-*_admin.bat` as the basis for a silent-mode installation. The command line calls already contain the required component MSIs and parameters as selected in the installer.

General command line calls

Installing Visual C++ runtimes:

```
vcredist_*.exe /quiet
```

Uninstalling Visual C++ runtimes:

```
vcredist_*.exe /quiet /uninstall
```

Installing or modifying MSIs:

```
msiexec /i <name>.msi /quiet <parameter>=<value>
```

Repairing MSIs:

```
msiexec /i <name>.msi /quiet INSTALLMODE=Restore
```

Uninstalling MSIs:

```
msiexec /x <name>.msi /quiet REMOVE_SECURE=1
```

Using preset options in `Setup-*.exe`:

```
Setup-*.exe <parameter>=<value>
```

Using preset options in `Setup-*.msi`:

```
msiexec /i Setup-*.msi <parameter>=<value>
```

Creating a batch file for silent-mode installation

This example is intended as a guideline for creating an installation script in silent mode. It assumes that the MSIs used for the installation are stored on a network share.

1. On a clean admin machine, create a new batch file, for example `silent.bat`.
2. Extract the main archive and start the file `Setup-*.exe` with administrator rights.
3. Select required components and settings, and finish installation.
4. Copy the entire contents from the extracted archive to a network share, for example `\\myShare`.
5. Add to the batch file the following commands for installing the C++ runtimes, for example:

```
REM *** install c++ runtimes ***
\\myShare\vcredist_14.42.34438.0_x64\vcredist_14.42.34438.0_x64 /quiet
\\myShare\vcredist_14.42.34438.0_x86\vcredist_14.42.34438.0_x86.exe /quiet
```

6. Add to the batch file the following *robocopy* command for copying the MSIs to the client computer, for example:

```
REM *** copy from network share to client ***
robocopy "\\myShare\packages" "C:\ProgramData\XPLM Solution GmbH\packages"
```

7. Go to the directory `%ProgramData%\XPLM Solution GmbH\cmd`.
8. Open the file `Setup-*_admin.bat` and copy to the batch file all command line calls for installing the MSIs:

```
REM *** installing msi ***
msiexec /i "C:\ProgramData\XPLM Solution GmbH\packages\Core_*.msi" /passive ←
  CALLED_BY=Setup-* ←
  INSTALLDIR="C:\Program Files\XPLM Solution GmbH\" ←
  BATCH_ADMIN="C:\ProgramData\XPLM Solution GmbH\cmd\Setup-*_admin.bat" ←
  GUI_LOG_FILE="C:\ProgramData\XPLM Solution GmbH\log\Setup-*_gui.log" ←
  JAVA_JNI=0 ←
  JAVA_JNI_X86="" ←
  JAVA_JNI_X64="" ←
...
```

In the above example, line breaks were inserted to show readable content. Realign lines so that each `msiexec` call is on one line. You can further clean-up each call by deleting the information marked red, as it is not required in your batch file.

9. Uninstall the installed product first. Start `Setup_*.exe/msi` and click **Remove**.
10. To test, run the batch file `silent.bat` with administrator rights. Check that the installer is executed without GUI and that all components are installed.

Creating a batch file for GUI-mode installation with preset options

This example is intended as a guideline for creating a batch script for GUI-mode with preset options. It follows a similar approach as silent-mode.

1. Create a new batch file, for example `preset.bat` and edit.
2. Repeat steps 2-6 from the above silent-mode procedure.
3. Add to the batch file the following command for calling `Setup-*.msi` with GUI and preset options:

```
REM *** preset options in gui ***
"msiexec /i "C:\ProgramData\XPLM Solution GmbH\packages\Setup-*.msi" ←
  INSTALLDIR="C:\Program Files\XPLM Solution GmbH\" ←
  <parameter>=<value> ←
  <parameter>=<value> ←
  <parameter>=<value> ←
...
```

   In the above example, line breaks were inserted to show readable content. Realign the one `msiexec` call on one line.
4. Replace `<parameter>=<value>` with the prefixed parameters from section Parameters: Silent-mode & GUI-mode (p. 140) and the normal parameters for enabling products from section Parameters: GUI-mode (p. 142).
5. Uninstall the installed product first. Start `Setup_*.exe/msi` and click **Remove**.
6. To test, run the batch file `preset.bat` with administrator rights. Check that the installer shows the preset options as defined, and finish installation.

Parameters: General information

- If no parameters are defined, default settings apply. In the following tables, default settings are underlined.
- If you use parameters in the scope of `Setup_*.exe/msi`, use them with the provided prefix, for example COR_JAVA_JNI.

  You cannot use the setup files `Setup_*.exe/msi` for silent-mode installation. For this you must use the individual component MSIs. You can only use parameters in scope of `Setup_*.exe/msi` to preset options when installing in GUI-mode.
- Use parameters without prefix to define settings in the scope of individual component MSIs, for example `Core_*.msi`.
- Use the following parameters to control either GUI-mode or silent-mode installation:
  - `none`: GUI-mode
  - `/quiet`: Silent-mode without GUI
  - `/passive`: Silent-mode with additional progress indication

Parameters: Silent-mode & GUI-mode

The following parameters apply to silent-mode as well as to GUI-mode with preset options.

Table 1: Component MSI & Setup EXE/MSI

Prefix	Parameter	Value	Description and use
	INSTALLDIR	Requires a valid path.	Defines the path of the installation directory. Available in all MSIs.
	INSTALLMODE	Change | Restore	Defines the installation mode after the installation is already completed and the installer is restarted. ■ Change = Modify ■ Restore = Repair Available in all MSIs.
	REMOVE_SECURE	0 | 1	Enables uninstallation. This corresponds to **Remove** in the installer. Available in all MSIs.
COR_BAT_	SERVER	0 | 1	Enables Batch Server. Available in Core-BatchServer MSI and all Setup EXE/MSI using this component.
COR_BAT_	ADMIN	0 | 1	Enables Batch Queue Admin. Available in Core-BatchServer MSI and all Setup EXE/MSI using this component.
COR_BAT_	SERVER_TYPE	INTERFACE | SERVICE	Defines if Batch Server should run with GUI or as service. Available in Core-BatchServer MSI and all Setup EXE/MSI using this component.
COR_COL_	CLIENT	0 | 1	Enables Collaboration Server. Available in Core-CollaborationServer MSI and all Setup EXE/MSI using this component.
COR_COL_	SERVER	0 | 1	Enables Collaboration Client. Available in Core-CollaborationServer MSI and all Setup EXE/MSI using this component.
COR_COL_	SERVER_TYPE	INTERFACE | SERVICE	Defines if Collaboration Client should run with GUI or as service. Available in Core-CollaborationServer MSI and all Setup EXE/MSI using this component.
ARS_	VERSION	Requires the version as shown in the installer.	Defines the Innovator version. Available in Aras MSI and all Setup EXE/MSI using this component.

Prefix	Parameter	Value	Description and use
INV_	VERSION	Requires the version as shown in the installer.	Defines the Inventor version. Available in Inventor MSI and all Setup EXE/MSI using this component.

Parameters: GUI-mode

The following parameters apply exclusively to the presetting of options in GUI-mode and not to silent-mode.

Table 2: All MSIs

Parameter	Value	Description and use
BACKUP_FILES	Requires a valid path.	Defines a location for the backup.
BACKUP_TYPE	FULL | CONFIG	Defines the backup scope. ■ FULL = full backup of `%xPlmRootDir%`. ■ CONFIG = backup of configuration directory only, for example `%xPlmRootDir%\xml`.
CUSTOM_FILES	Requires a valid path.	Defines the path to an overlay package with custom files to be copied after installation.
CUSTOM_FILES_TEMP	Requires a valid path.	Defines the path to where an overlay package is copied first before it is copied to its final destination. It's a good practice to set this path to `%ProgramData%\XPLM Solution GmbH\custom_files`.

Table 3: Setup EXE/MSI

Parameter	Value	Description and use
ARS_BAT	0 | 1	Enables Batch Server. Available in all Setup EXE/MSI using this component.
ARS_TST	0 | 1	Enables License Test Tool. Available in all Setup EXE/MSI using this component.
ARS_COV	0 | 1	Enables Conversion Server. Available in all Setup EXE/MSI using this component.
ARS_EXP	0 | 1	Enables File Export Tool. Available in all Setup EXE/MSI using this component.
COR_ENC	0 | 1	Enables Encryption Helper. Available in all Setup EXE/MSI using this component.

Table 4: Setup-Aras-MCAD EXE/MSI

Parameter	Value	Description
ARS_ACD	0 | 1	Enables AutoCAD Mechanical.
ARS_INV	0 | 1	Enables Inventor.
ARS_PRO	0 | 1	Enables Creo Parametric.
ARS_REV	0 | 1	Enables Revit.
ARS_SDE	0 | 1	Enables Creo Elements.
ARS_SED	0 | 1	Enables Solid Edge.
ARS_SNX	0 | 1	Enables NX.
ARS_CBS	0 | 1	Enables CADBAS Checkout Tool.

10.3 Working with overlay packages

An overlay package contains custom files with modified configuration. In the installer, you can select an option to apply an overlay as the last step of the installation process, copying the custom files over the installed files.

Supported overlay features in the installer

The following features are available in installer versions 24.3.3.606+. For older installers, the legacy behaviour applies. This behaviour has some limitations, see column **Legacy** for more information.

Feature	Description	Legacy
Support for directories and archives	An overlay can be referenced in the form of an archive or as a directory. Supported archive formats are ZIP, 7z or 7z.exe.	Only directories and no archives can be referenced.
Support for local directories or shares on the network	You can reference an overlay from the local disk or from a network share.	
Support for multiple directories	By default, an integration is installed in one directory, which is referred to with the environment variable `xPlmRootDir`. Certain integrations require multiple installation directories which you can select in the installer, or are set in the background. You can reference these paths in the overlay, so that the content is copied to the correct location when the overlay is applied. To reference multiple installation directories in an overlay structure, create the following directories below your overlay `<ROOT>` directory: ■ `<ROOT>\INSTALLDIR`: Resolves to the main installation directory `%xPlmRootDir%`. The directory `INSTALLDIR` is not required if everything is installed in one directory but must always be added in case of multiple installation directories. ■ `<ROOT>\<PREFIX>_PATH`: Resolves to another installation directory you can select in the installer. See Silent-mode installation (p. 136) for installation parameter.	

Feature	Description	Legacy
Support for custom scripts	When applying an overlay, you can also run a script with custom actions. For this, add the script `custom.bat` in the directory `<ROOT>\script`. If the integration requires multiple installation directories, add the installation parameter again as first directory in the overlay structure. This parameter defines the directory where the script is executed, for example: ■ `<ROOT>\INSTALLDIR\script\custom.bat` The directory `INSTALLDIR` is not required if everything is installed in one directory but must always be added in case of multiple installation directories. ■ `<ROOT>\<PREFIX>_PATH\script\custom.bat` When applying the overlay, the script is copied to the directory `script` in the target location as defined by the overlay structure and executed from there. In installer versions 24.4.4.628+, you also have access to all environment variables in your script that were written during the installation.	

<table>
<tr><th>Feature</th><th>Description</th><th>Legacy</th></tr>
<tr><td>Support for automatic detection of overlays</td><td>Overlay archives or directories are automatically recognized if they are located next to or in the extracted setup directory and contain the string overlay in any written form in their name, for example:
Example 1 (overlay as 7-Zip archive next to extracted setup)<pre><EXTRACTED INSTALLER DIR>
myOverlay.7z</pre>
Example 2 (extracted overlay next to extracted setup)<pre><EXTRACTED INSTALLER DIR>
myOverlay</pre>
Example 3 (overlay as ZIP archive in extracted setup)<pre>myOverlay.zip
packages
vcredist_*_x64
vcredist_*_x86
Setup-*.exe</pre>
Example 4 (extracted overlay in extracted setup)<pre>myOverlay
packages
vcredist_*_x64
vcredist_*_x86
Setup-*.exe</pre>
If the overlay exists as shown in the above examples, and the installer is started for the first time, the option Apply custom files after installation is automatically enabled with the correct path.

When the installer is restarted, it checks whether the applied overlay is still in its original location and automatically enables the option with the correct path. For example, put the overlay to a network share and it is always referenced from this location, see also best practices.</td><td>Only directories and no archives can be auto-detected. The overlay directory must be named <code>custom_files</code>.</td></tr>
</table>

Best practices with overlays

- To create an overlay archive, complete these steps:
  1. Create a new overlay `<ROOT>` directory on disk.
  2. Create relevant sub-directories as in the original location, for example `<ROOT>\xml` and copy modified files. If you have multiple installation directories, add the following directories below `<ROOT>` first:
     - `INSTALLDIR`
     - `<PREFIX_PATH>`
  3. Create an archive from the content within the directory `<ROOT>` and not from the directory `<ROOT>` itself. Add the string *overlay* in the archive name to allow auto-detection by the installer.
     Correct

     ```
     <ROOT>
       <overlay content> => create archive
     ```

     Incorrect

     ```
     <ROOT> => create archive
       <overlay content>
     ```
  4. Test the archive. Extract it using option *Extract here* and check if just the content appears, without the `<ROOT>` directory.
  5. Always add a changelog file to the overlay package that shows the modified scope with date and description. A version counter for the entire overlay in this file is also helpful. This way, you can quickly check whether the correct overlay content has been applied.
- To apply an overlay archive in the installer, enable the option **Apply custom files after installation** and navigate to the archive. Make sure to select **Archive Files** in the selection dialog. If you have placed the overlay for auto-detection, the option is automatically enabled with the correct path.
- To apply an overlay directory in the installer, extract the archive first. Then enable the option **Apply custom files after installation** and navigate to the extracted overlay directory. Make sure to select **Directories** in the selection dialog. If you have placed the overlay for auto-detection, the option is automatically enabled with the correct path.
- To apply an overlay manually, extract the archive first. Then copy the modified files to the correct location.
- For deployment to clients, it is best to store the overlay on a network share. This allows you to update modified configuration on a regular basis and apply the overlay via your preferred method.
- When applying an overlay, its content is copied first to the directory `%ProgramData%\XPLM Solution GmbH\custom_files\<DATE-TIME>` for backup purposes. The content is then copied to the directories as defined by the overlay structure. Existing files in the target location are overwritten.

How to apply an overlay in silent-mode

There are no parameters for applying an overlay package in silent-mode, because the copying process is not triggered directly by an MSI. You can only use certain parameters to preselect options in the installer. But, you can use *Robocopy* commands to copy the overlay content from a network share directly to the target directory.

1. Copy the overlay to a network share, for example `\\myShare\myOverlay`.
2. In your own batch file for silent-mode, add the copy operation to the target directory as the last step, for example:

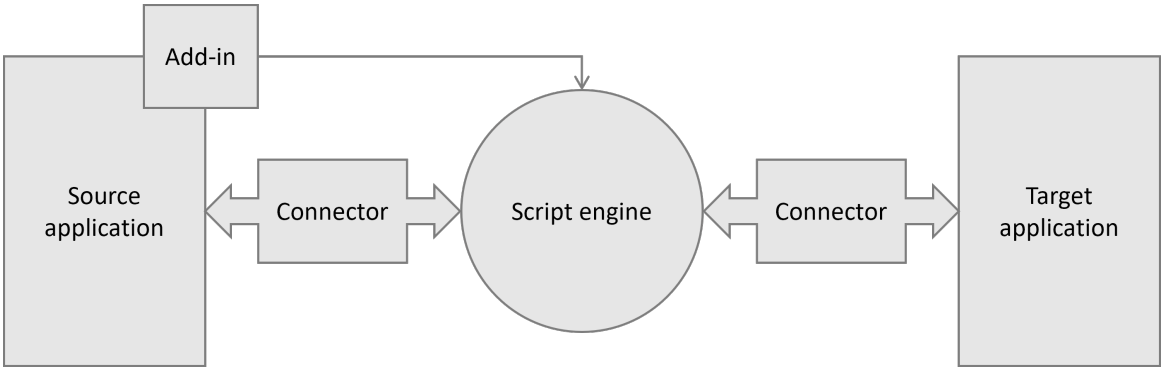
```
...
REM *** apply overlay ***
robocopy "\\myShare\myOverlay" "C:\Program Files\XPLM Solution GmbH" /E
```

3. Execute your batch file as administrator on a clean client computer.
4. Check that the overlay was applied correctly and that the integration is working.

10.4 Integration components and related files

The integration consists of several software components that interact with each other. Read this chapter to familiarize yourself with these components.

The following components represent a classical integration setup. The setup can be more complex or use additional components for specific integrations. However, most of the components described below are always part of the setup in one way or another.



- The **connector** is a software library that provides access to an application through its API. It implements various functions based on the application API and defines a standardized interface to the script engine. Typical connector functions include for example *getActiveDocument*, *getOpenDocuments*, *Open*, *Save*, or *Close*.
- The **script engine** is a software library that provides the integration logic between applications and connectors. It does not directly access related applications, but communicates with them through the connectors. This architecture ensures that the script engine can support different application versions without needing to change the integration logic. Main functions of the script engine include for example *Connect*, *Save*, *Checkin/Checkout*, or *Load*.
- The **add-in** provides the integration functions in the source application. The add-in communicates with the script engine, which in turn communicates with the connector.
- The integration consists in general of the following configuration files:
  - `XPlmConnector.xml`: Contains global configuration options and lists other dependent files, for example connector and resource files.
  - Transaction files: Contain the data mapping between applications. Data mapping is essential for exchanging data using the integration.
  - Connector files: Contain options for the connectors and the script engine.
  - Dialog files: Contain the configuration of the integration dialogs.
  - Resource files: Contain the language-specific text strings used in dialogs and add-in.
  - Add-in file: Contains the configuration of the add-in.

10.4.1 Data mapping and transaction files

A key concept of the data exchange between applications is the data mapping. Read this chapter to familiarize yourself with these topics.

Data mapping

In each integration, object information is exchanged between connected applications. In the most simple form, a property or the content of a database field is transferred from the source application to the target application. The technical mapping and its process within the integration are commonly referred to as data mapping.

Data mapping is based on transaction files written in the XML syntax. For more complex structure transformations, also XSLT stylesheets can be used.

In both cases, the process flow is the same:

- The source data is read from the source application and provided internally in an *XPlmDocument* container.
- This source data is translated either via a transaction or a stylesheet into an object that can be understood by the target application. The translation is done by the script engine, but also by connectors.

Transaction files

Transactions are independent, unified data containers that define the data mapping. Each file can contain multiple transactions. A transaction is identified by the element `<Aliasname>`. It further consists of the element structures `<FieldCollection>`, `<StructureCollection>` and `<TableCollection>`. These structures contain values that are important for the underlying definition of the data mapping.

```
<Transaction>
  <Aliasname>MyTransaction</Aliasname>
  <Import>
    <Parameter>
      <FieldCollection>...</FieldCollection>
      <StructureCollection>...</StructureCollection>
      <TableCollection>...</TableCollection>
    </Parameter>
  </Import>
</Transaction>
```

There are a few points to keep in mind when working with transactions. Within the transaction itself, all information is treated as strings, which means that unless specific functions are used, there is no parsing or validation of the content.

Related links
Overwriting transaction structures (p. 155)

10.4.2 XPLM data types

The data types listed here are used in transaction files, but also in connector files. They represent the internal data model of the integration on which the entire data exchange is based.

Field

Defines the smallest configuration unit. It describes a key/value pair and has its origin in a field of the source application, for example a database field or a property. The key is defined in the *Name* element, the value in the *Value* element .

```
<Field>
  <Name>MyField</Name>
  <Value>Value</Value>
</Field>
```

FieldCollection

Groups several *Field* elements. Within a *FieldCollection* element, field names should be unique. There can be only one *FieldCollection* element in an XML file.

```
<FieldCollection>
  <Field>
    <Name>Field1</Name>
    <Value>Value1</Value>
  </Field>
  <Field>
    <Name>Field2</Name>
    <Value>Value2</Value>
  </Field>
</FieldCollection>
```

Structure

Defines a named *FieldCollection* element. It is identified by the *Name* element and contains a *FieldCollection* element. Thus, it represents a collection of properties that are not combined in the simple FieldCollection element for technical reasons or for better readability of the source data. In addition, *Field* elements with the same name can be used in different structures.

```
<Structure>
  <Name>Structure1</Name>
  <FieldCollection>
    <Field>
      <Name>Field1</Name>
      <Value>Value1</Value>
    </Field>
    <Field>
      <Name>Field2</Name>
      <Value>Value2</Value>
    </Field>
  </FieldCollection>
</Structure>
```

StructureCollection

Groups several *Structure* elements but can also contain only one structure or be empty. There can be only one *StructureCollection* element in a transaction.

```
<StructureCollection>
  <Structure>
    <Name>Structure1</Name>
    <FieldCollection>...</FieldCollection>
  <Structure>
  <Structure>
    <Name>Structure2</Name>
    <FieldCollection>...</FieldCollection>
  <Structure>
</StructureCollection>
```

FieldCollectionTable

Contains one or more *FieldCollection* elements. It is used exclusively within the *Table* element, which provides a table-like mapping of data. Unlike a table definition in a relational database, the columns, in this case the *Field* elements, can be different in each row, although this will rarely be the case in practice. In this container, each *FieldCollection* element corresponds to a table row.

```
<FieldCollectionTable>
  <FieldCollection>...</FieldCollection>
  <FieldCollection>...</FieldCollection>
</FieldCollectionTable>
```

Table

Groups several *FieldCollectionTable* elements and is identified by the element *Name*.

```
<Table>
  <Name>Table1</Name>
  <FieldCollectionTable>...</FieldCollectionTable>
  <FieldCollectionTable>...</FieldCollectionTable>
</Table>
```

TableCollection

Groups several *Table* elements. There can be only one *TableCollection* element in a transaction.

```
<TableCollection>
  <Table>
    <Name>Table1</Name>
    <FieldCollectionTable>...</FieldCollectionTable>
   </Table>
   <Table>
      <Name>Table2</Name>
      <FieldCollectionTable>...</FieldCollectionTable>
   </Table>
</TableCollection>
```

10.4.3 Data referencing

The data types describe the source data format, whereby the target data format is semantically identical.

Basic rules for data referencing

The referencing of the individual fields is done with a special notation. This type of description is subject to certain restrictions and follows some basic rules:

- Referencing is always done at the field level, which means that the source of the data can be very broad, but the target is always a *Field* element.
- Referencing data in a table can only be made with a specific row number or with the help of a search expression. A dynamic reference is not possible.
- Partially qualified references are not supported, which means the name of an element must always be specified, case sensitive.
- If the referenced *Field*, *Structure*, or *Table* element does not exist, an empty string is used as the result.

Reference to fixed value

In the simplest form, there is no reference to the source data, but a fixed value is specified.

```
<Field>
  <Name>LANGUAGE</Name>
  <Value>DE</Value>
</Field>
```

Name: Refers to the target field.

Reference to field

To reference the value from a field in a *FieldCollection*, three parameters are specified instead of a fixed value:

- *Type*: Contains *XPlmDocument* or *ParameterList* and refers to the default object used in the integration source code.
- *Subtype*: Contains the name of the parent element, in this case *FieldCollection*.
- *Attribut*: Contains the name of the *Field* element from which the value is to be read.

```
<Field>
  <Name>DESCRIPTION</Name>
  <Type>XPlmDocument</Type>
  <Subtype>FieldCollection</Subtype>
  <Attribut>T_MASTER_DAT.PART_NAME</Attribut>
</Field>
```

Reference to field in StructureCollection

To reference the value from a field in a *StructureCollection*, four parameters are specified:

- *Type*: Contains *XPlmDocument* or *ParameterList* and refers to the default object used in the integration source code.
- *Subtype*: Contains the name of the parent element, in this case *StructureCollection*.
- *Structurename*: Contains the name of the structure.
- *Attribut*: Contains the name of the *Field* element from which the value is to be read.

```
<Field>
  <Name>DOCUMENTNUMBER</Name>
  <Type>XPlmDocument</Type>
  <Subtype>StructureCollection</Subtype>
  <Structurename>EDB-ART-DOC-RLI</Structurename>
  <Attribut>T_DOC_DAT.DOCUMENT_ID</Attribut>
</Field>
```

Reference to field in TableCollection

To reference the value from a field in a table and a defined row in a *TableCollection*, five parameters are specified:

- *Type*: Contains *XPlmDocument* or *ParameterList* and refers to the default object used in the integration source code.
- *Subtype*: Contains the name of the parent element, in this case *TableCollection*.
- *Table name*: Contains the name of the table.
- *Row*: Contains the number of the row from which the field is to be read.
- *Attribut*: Contains the name of the *Field* element from which the value is to be read.

```
<Field>
  <Name>URL</Name>
  <Type>XPlmDocument</Type>
  <Subtype>TableCollection</Subtype>
  <Tablename>EDB-ART-SLI</Tablename>
  <Row>5</Row>
  <Attribut>T_MASTER_DAT.UNIT</Attribut>
</Field>
```

10.4.4 Field mapping

The transfer of values alone is not sufficient in practice, since there is usually no conversion of different types in the source and target connectors. This means that all values in a data structure are not typed and must therefore be treated as simple strings. It can be assumed that the data types of the applications involved are usually not identical and treat types such as dates, logical values or numbers differently. For this reason and certain properties it is necessary to convert the content before further use.

For this purpose there is a possibility to convert the content with the help of certain functions, which is called field mapping. The basic principle is that in each field reference in the data mapping an additional element *Function* can be used, in which one or more sequentially acting functions for the conversion can be entered. The input of the function always consists of the referenced value and the specific function parameters, the result of the function call is then taken over for the data mapping.

Simple function call

The function name and its comma-separated parameters are written to the *Function* element. In this example, the referenced value has a prefix added.

```
<Field>
  <Name>DESCRIPTION</Name>
  <Type>XPlmDocument</Type>
  <Subtype>FieldCollection</Subtype>
  <Attribut>T_MASTER_DAT.PART_NAME</Attribut>
  <Function>Prefix,PRE-</Function>
</Field>
```

In another example, one string is replaced by another.

Spaces after the comma are considered part of the parameter!

```
<Field>
  <Name>DESCRIPTION</Name>
  <Type>XPlmDocument</Type>
  <Subtype>FieldCollection</Subtype>
  <Attribut>T_MASTER_DAT.PART_NAME</Attribut>
  <Function>Replace,-,/</Function>
</Field>
```

Multiple function call

Multiple functions can also be used:

- The functions are executed one after the other.
- The result of each function is used as input for the following function (sequential execution).
- If the execution of a partial function runs into an error, it is skipped and the result of the last successfully executed partial function is used as input for the subsequent function.

```
<Field>
  <Name>DESCRIPTION</Name>
  <Type>XPlmDocument</Type>
  <Subtype>FieldCollection</Subtype>
  <Attribut>T_MASTER_DAT.PART_NAME_GER</Attribut>
  <Function>Replace,#,-|Postfix,-POST</Function>
</Field>
```

10.4.5 Overwriting transaction structures

You can overwrite and thus customize certain configuration files that contain transaction structures.

To overwrite a transaction, copy the original file and add the prefix `Customer_`. This so called customer file may or may not contain a transaction from the original file. When executing transactions, the integration checks whether a corresponding customer file exists. If it exists, any transaction listed in that file is executed instead of the transaction defined in the original file. If no transaction is defined in the customer file, the transaction from the original file is used.

Always overwrite complete transaction structures. You cannot overwrite a subset of a transaction, for example a single *FieldCollection* element, because these XML structures cannot be identified without the corresponding transaction.

Unfortunately, overwriting transactions does not work for all integration components. First try overwriting the transaction in a customer file. If it does not work, modify the transaction in the original file. Make sure to backup all your modified files before an update.

After modifying a transaction, always restart Inventor. This will reload all configuration files used by the integration.